

Title Page

T H E E M B O D I E D L L M A R E N A
An Open-Source Framework for Benchmarking Every Large Language Model Through Human-Robot Interaction with Pepper
-- Built on OmniLLM --
A Master's Thesis & Engineering Reference
Akshita-sr
DIBRIS, University of Genoa * 2026
Prof. Antonio Sgorbissa's Human-Robot Interaction Lab

Copyright & Licence

This book documents the **OmniLLM** open-source project (MIT License, copyright © 2026 Akshita-sr) and the **Embodied LLM Arena** research study that the project supports.

The book is written as:

1. an **educational reference** for the project owner, future collaborators, and the open-source community,
2. a **technical foundation** for a Master's thesis in Human-Robot Interaction submitted at the University of Genoa, DIBRIS, under the supervision of Prof. Antonio Sgorbissa, and
3. a **deployment guide** for the real Pepper robot located in Prof. Sgorbissa's lab at DIBRIS.

The source code is freely available at:

<https://github.com/Akshita-sr/OmniLLM>

Dedication

To everyone who ever looked at a chatbot and asked —

"Yes, but what would it be like to talk to one with a body?"

Acknowledgements

This work is conducted at the **Department of Informatics, Bioengineering, Robotics and Systems Engineering (DIBRIS), University of Genoa**, in the Human-Robot Interaction laboratory directed by **Prof. Antonio Sgorbissa**. The real Pepper robot used in the experimental study is hosted by that lab.

The OmniLLM software framework is built upon the following open-source projects, to whose authors and maintainers the author owes a profound debt:

- **LangGraph** and **LangChain** (Harrison Chase et al.) — for the agent graph runtime
- **LiteLLM** (BerriAI) — for the unified LLM provider gateway
- **ChromaDB** — for the local vector database underlying the RAG pipeline
- **OpenAI Whisper** — for local speech-to-text
- **Hugging Face sentence-transformers** — for semantic embeddings
- **NAOqi SDK** and **Choregraphe** (SoftBank / Aldebaran Robotics) — for Pepper's middleware and simulator
- **Flask**, **ReportLab**, **xhtml2pdf**, **DejaVu Sans/Mono** — for the AI server and this book's typesetting

Preface — How to Read This Book

This book is unusual in one important way: **it is written for three different readers at once.**

When you began this project, you (the author) were one of those readers. By the time you finish reading the book, you will have walked through the perspective of all three — and you will be able to *talk* to the project, the code, and the research community fluently in any of the three "languages."

The Three Readers

Reader	Who they are	What they need from the book
The Owner	You — the person who built this and may have forgotten parts of it after a month	Quick-take recap boxes, file paths, "why did we do X this way?" justifications, fast orientation
The Beginner	A friend, intern, or junior developer who knows a little Python and wants to understand the system from zero	Plain-English explanations, mini-primers on jargon, worked examples, Python concepts explained at the point of first use, no skipping steps
The Academic	A thesis examiner, an HRI researcher, an LLM engineer evaluating the work for citation or replication	Literature context, formal evaluation methodology, hypotheses, why the design is methodologically defensible

The Layered Chapter Structure

Every chapter from Part II onwards follows the same **three-layer structure** so that you can read it the way that fits your purpose today:

+-----+
AT A GLANCE (Owner's Recap)
One-paragraph summary, file paths, the "why we built it
this way" justification.
+-----+
THE WALK-THROUGH (Beginner's Path)
Plain language, jargon defined on first use, examples you can
actually run, Python concepts explained where they appear.
+-----+
ACADEMIC CONTEXT (Researcher's Sidebar)
Literature, citations, research justification, "why this
approach is methodologically defensible."

How to Use This Book

If you are...	Read...
Re-learning your own project after a break	Skim the <i>At a Glance</i> boxes; dive into the modules you've forgotten
Setting up the project on a new machine	Part IV (Setup, Execution, Building from Scratch) front-to-back
Onboarding a collaborator	Hand them Parts I, II, and the relevant module chapter from Part III
Writing a thesis or paper based on this work	Read the <i>Academic Context</i> sidebars; consult Part V for the experimental design
Connecting to the real Pepper at DIBRIS for the first time	Part IV -> Chapter "Day Zero — Real Pepper at the Lab"
Trying to understand a single Python file	Find it in the master file table in Part III, Chapter 9; each module has its own deep-dive chapter
Just curious	Read the Preface, then skip to Chapter 1

A Note on Notation

Throughout this book:

- **Bold** marks a term being introduced for the first time. The first introduction is always a definition — not just a use of the word.
- `code-style monospace` is used for filenames, command lines, function names, and code identifiers.

Block-quoted text contains direct quotes from the codebase or short excerpts that are too important to paraphrase.

- (Tip) marks a rule of thumb or shortcut that experienced users will appreciate.
- (!) marks a warning or known gotcha — usually a thing that bit the author in the past.
- (Academic) marks material aimed primarily at the thesis examiner; you may skim it on a first read.
- (Beginner) marks an inline mini-primer on a piece of Python or systems jargon; experienced developers may skip these blocks.

What This Book Does Not Cover

To keep the book finite, the following are **not** included in detail:

- Generic Python language tutorials. Appendix A provides a focused primer on the parts of Python you will see in this codebase, but for a complete tutorial we point you to the official Python documentation.
- Generic Linux / Windows administration.
- The internal implementation details of LangGraph, LiteLLM, ChromaDB, or the NAOqi C++ middleware. We use these as black boxes and focus on how OmniLLM uses them. References to the upstream documentation are given where relevant.
- The full text of the OmniLLM source code (it lives in the repository). The book quotes excerpts; the code is the source of truth.

A Word on the May 2026 Upgrade

The book you are reading is the **second edition** of the OmniLLM book. Between the first edition (168 pages, written in March 2026) and this edition, the project underwent a substantial upgrade in preparation for the live experimental study at DIBRIS. The major changes folded into this rewrite are:

- **A new robot bridge topology.** The first edition assumed Pepper polls the AI server (Python 2.7 -> Python 3.x). The May 2026 upgrade adds a *reverse* topology where the Python 3.x AI server drives Pepper through a small HTTP bridge server running on Python 2.7. Both topologies are now first-class.
- **Three trigger modes for live conversation:** `text` (typed input — works on virtual Pepper too), `touch` (head-touch + 5-second audio capture, real Pepper only) and `vad` (continuous voice-activity detection, real Pepper only).
- **Face-tracking** via NAOqi's ALTracker is now integrated as a first-class interaction primitive, not an experimental extra.
- **Per-condition model resolution** moved from the LangGraph nodes into `omnillm/server/app.py`, fixing a bug where Condition B (fixed local Llama) silently ran the cloud default model.
- **Multilingual routing rerouted** from `gemini-flash` to `claude-haiku` after the Google free tier started rate-limiting Condition T4 mid-experiment.
- **Knowledge base rewritten** for the **University of Genoa / DIBRIS / Sgorbissa lab** context. The previous generic IRAI-Lab content is preserved under `knowledge_base/_legacy_irai/` for reference.
- **A two-run pilot** with the author as participant P000 (2026-05-20) revealed the silent-routing bug above and validated the rest of the pipeline end-to-end on Choregraphe's virtual Pepper.

If you have read the first edition, treat this one as a complete replacement. Section structure and chapter numbering have changed.

Table of Contents

Part I — Motivation & Scope

- **Chapter 1** — What Is OmniLLM, in One Sentence and in 1,000 Words
- **Chapter 2** — Why a Physical Humanoid Robot? The Motivation Behind Pepper
- **Chapter 3** — What Happens When the Robot is Absent — the Three Operating Modes
- **Chapter 4** — Scope, Hypotheses, and the Embodied LLM Arena Study

Part II — Architecture, Flow, & Tech Stack

- **Chapter 5** — The Three-Layer System Architecture
- **Chapter 6** — The Lifecycle of a Single Question, End to End
- **Chapter 7** — The LangGraph Agent Pipeline, Node by Node
- **Chapter 8** — The Tech Stack — Every Library, and Why That One
- **Chapter 9** — Two Topologies for Driving Pepper (Polling vs. Bridge Server)

Part III — Comprehensive Script & File Breakdown

- **Chapter 10** — The Master File Table — Every Script, Its Purpose, Its Execution Order
- **Chapter 11** — Core LLM Modules: `gateway.py`, `router.py`, `consensus.py`, `evaluator.py`, `scorer.py`, `cli.py`
- **Chapter 12** — HRI & RAG Modules: `hri/classifier.py`, `hri/language_detector.py`, `hri/agent_graph.py`, `hri/experiment.py`, `rag/pipeline.py`
- **Chapter 13** — Robotics Modules: `robotics/bridge.py`, `robotics/pepper.py`, `robotics/gesture_planner.py`, `robotics/whisper_stt.py`
- **Chapter 14** — Server & NAOqi Bridge: `server/app.py`, `server/naoqi_bridge_server.py`, `server/naoqi_client.py`
- **Chapter 15** — Utilities: `utils/experiment_logger.py`, `utils/questionnaire.py`, `utils/cost_tracker.py`, `utils/export.py`
- **Chapter 16** — The Python 2.7 ↔ Python 3.x Bridge Problem (and Three Different Solutions to It)

Part IV — Setup, Execution, & Building from Scratch

- **Chapter 17** — Installing OmniLLM From Zero (Windows, macOS, Linux)
- **Chapter 18** — Scenario A — Running Without Any Robot (Text-Only)
- **Chapter 19** — Scenario B — Running With Choregraphe's Virtual Pepper
- **Chapter 20** — Scenario C — Running With the Physical Pepper Robot

- **Chapter 21** — Day Zero — The Real-Pepper Deployment at DIBRIS, Step by Step
- **Chapter 22** — Building This Project From Scratch — A 13-Week Plan for a Replicator

Part V — Conducting Experiments & Logging

- **Chapter 23** — The Experimental Design — Five Conditions, Four Task Types, One Hypothesis Set
- **Chapter 24** — Running a Single Subject Session (Step by Step)
- **Chapter 25** — Counterbalancing and Why It Matters (Latin-Square Designs Explained)
- **Chapter 26** — The Data Logging Pipeline — From Microphone to CSV
- **Chapter 27** — Questionnaires — Likert, Godspeed, Pairwise, Observer (Concrete Examples)
- **Chapter 28** — Ethics, Consent, GDPR, and the UniGE Process
- **Chapter 29** — Insights From the May 2026 Pilot — What the Two P000 Runs Actually Showed
- **Chapter 30** — Data Analysis with pandas and Jupyter — A Beginner-Friendly Workflow

Part VI — Future Scope & Improvements

- **Chapter 31** — Current Limitations — Honest Assessment
- **Chapter 32** — Short-Term Improvements (Next 3 Months)
- **Chapter 33** — Medium-Term Research Directions (Next 12 Months)
- **Chapter 34** — Long-Term Vision — Where Embodied LLM Research Is Going

Appendices

- **Appendix A** — Python Primer — Just the Parts You Need for OmniLLM
- **Appendix B** — Complete Glossary
- **Appendix C** — Complete Terminal Command Reference (Windows PowerShell + macOS/Linux Bash)
- **Appendix D** — Troubleshooting Reference (Symptom -> Cause -> Fix)
- **Appendix E** — End-to-End Flow Diagram (One Page)
- **Appendix F** — File Index — Every File in the Repository, One Line Each
- **Appendix G** — External Resources — Papers, Libraries, Repositories (Annotated)
- **Appendix H** — Pepper-LLM Integration Survey — The State of the Field, May 2026
- **Appendix I** — The Pepper Platform Reference — Hardware, NAOqi, Choregraphe, the Five Bridge Patterns
- **Appendix J** — AI-Stack Library Rationale — What, Why, Alternatives, Where Used
- **Appendix K** — Walkthroughs, Feature Catalogue, and Execution Plans (Line-by-Line Code Journey, Worked Session, Eight Axes, Complete Feature Reference, One-Month Plan)

Part I — Motivation & Scope

Before you can fairly read a single line of code, you need to understand what the project is trying to be, why it exists at all, and what it deliberately is not. Part I answers exactly those three questions.

Chapter 1 — What Is OmniLLM, in One Sentence and in 1,000 Words

At a Glance (Owner's Recap)

OmniLLM is an open-source Python framework that lets you compare, route between, and orchestrate every major Large Language Model — and then plug whichever one is winning today into a Pepper humanoid robot. It started as a multi-LLM benchmark workbench (gateway, smart router, LLM-as-judge, ELO leaderboard) and grew a second body: a LangGraph-based Human-Robot Interaction pipeline that gives Pepper a brain assembled out of any combination of GPT-4o, Claude Haiku, Gemini 2.5, DeepSeek, Llama 3, Qwen 2.5, Mistral 7B, and twelve more models, with optional **RAG** grounding against a domain knowledge base — currently the DIBRIS / Sgorbissa lab — and full per-interaction logging for experimental study.

Files you most often touch:

File	Role
omnilm/gateway.py	The single async door to every LLM provider
omnilm/router.py	Six strategies for picking the right model for a given query
omnilm/hri/agent_graph.py	The nine-node LangGraph pipeline that is "Pepper's brain"
omnilm/server/app.py	The Flask HTTP bridge between Pepper (Python 2.7) and the AI stack (Python 3.x)
config/models.yaml	The 19-model registry — add a new LLM here, no Python changes needed

The Walk-Through (Beginner's Path)

Let's unpack that one-sentence definition piece by piece, because each word in it carries a specific design decision.

"A Large Language Model"

A **Large Language Model (LLM)** is an artificial-intelligence system that reads and produces natural-language text. The most familiar examples are *ChatGPT* (which is a product wrapper around the **GPT** family of models from OpenAI), *Claude* (from Anthropic), *Gemini* (from Google DeepMind), and *Llama* (from Meta). Each LLM is, internally, a very large neural network with billions to trillions of parameters that has been trained on a substantial fraction of the public internet plus assorted licensed and curated corpora. From the outside, an LLM looks like a function `f("user message") -> "assistant message"` — but every provider implements that function with their own API, their own pricing, their own latency profile, and their own strengths and weaknesses. GPT-4o is famously good at reasoning. Claude is famously good at long-context document understanding. Gemini is famously good at multimodal input. Mistral and Llama are famously cheap and open-weight. **Nobody is best at everything**, and the model that is best for *your* particular question today may be different from the one that is best for the next question.

"Every Major Large Language Model"

OmniLLM currently registers **19 models from 6+ providers**. That registry is plain YAML (`config/models.yaml`), and adding a new LLM is a seven-line YAML change with zero Python edits required. As of the 2026-05-20 deployment, the active registry includes:

- **OpenAI:** GPT-4o, GPT-4o-mini, GPT-3.5-turbo
- **Anthropic:** Claude Sonnet, Claude Haiku
- **Google:** Gemini 2.5 Pro, Gemini 2.5 Flash, Gemini Flash
- **DeepSeek:** DeepSeek V3
- **Ollama (local, free, no API key):** Llama 3:8b (aliased `llama3-8b-local`), Llama 3.2:3b, Qwen 2.5:7b (aliased `qwen3-8b-local`), Mistral 7B, Phi-3, plus a handful of fine-tuned variants

The key word in *every major LLM* is **abstraction**. OmniLLM never talks to a provider's SDK directly. Instead it relies on **LiteLLM**, a Python library that translates a uniform interface (`litellm.completion(model="...", messages=[...])`) into the actual HTTPS POST that each provider expects. The benefit is that OmniLLM's own code only has to deal with one calling convention, regardless of whether the model lives in OpenAI's data centre, in Anthropic's, or in `ollama serve` running on `localhost:11434` on your laptop.

"Compare, Route Between, and Orchestrate"

These three verbs map to three different patterns in the codebase:

Verb	What it does	Code path
Compare	Run the same prompt against many models in parallel, score each output with an LLM-as-Judge, and update an ELO leaderboard	<code>omnilm/evaluator.py</code> + <code>omnilm/scorer.py</code>

Route	Pick <i>one</i> model dynamically based on a strategy (cheapest / fastest / best-quality / best-for-task)	<code>omnilm/router.py</code>
Orchestrate	Fan out to N models in parallel and merge their answers via voting, weighted-sum, or judge-LLM synthesis	<code>omnilm/consensus.py</code>

The three patterns are not mutually exclusive — a single experimental condition can use them all. **Condition D** in the Embodied LLM Arena, for example, asks three models (GPT-4o-mini + Claude Haiku + Gemini Flash) the same question, then uses GPT-4o-mini *itself* as the judge to synthesise a final answer.

"Plug Into a Pepper Humanoid Robot"

Pepper is a 1.2-metre-tall, plastic-shelled humanoid robot manufactured by SoftBank Robotics (formerly Aldebaran Robotics) since 2014. It has:

- 20 degrees of freedom (DOF) for arm, head, and torso movement
- omnidirectional wheels on a triangular base (no legs — it cannot walk, but it can roll)
- 4 microphones on the head (used here for speech input)
- a 3-D depth sensor + 2 RGB cameras
- a touchscreen tablet on its chest
- coloured LEDs in its eyes (used here for emotional cues)
- a Linux-based on-board computer running the **NAOqi** middleware

The on-board computer is locked to **Python 2.7** because NAOqi's official SDK was last released in that era and has never been ported. This is the single most important fact about programming Pepper in 2026: **the modern AI stack you want to use does not run on the robot**. OmniLLM solves this by splitting itself into two processes that talk over HTTP, but we will return to that surprising amount of plumbing in Chapter 9 and Chapter 16.

"Open-Source Python Framework"

OmniLLM is MIT-licensed and lives on GitHub. The whole project is roughly **9,500 lines of Python plus 250 pages of documentation (this book)**. The codebase is deliberately structured as a *library* (`omnilm/` is a normal `pip` install-able package), a *CLI* (`omnilm` is a console script registered in `pyproject.toml`), and a *deployment* (`omnilm/server/app.py` is a Flask app you run with `python -m omnilm.server.app`). All three modes share the same gateway / router / RAG / agent-graph backbone.

The Core Capabilities Table

This is the master "what does OmniLLM actually do" table — pin it to your wall.

Capability	What it does	Why it exists
------------	--------------	---------------

Unified Gateway	One async function call reaches any of 19 registered models from 6+ providers (cloud + local Ollama).	Without this, every script in the repo would carry per-provider try/except spaghetti.
Smart Router	Six strategies (BEST_QUALITY, LOWEST_COST, LOWEST_LATENCY, BEST_VALUE, LOCAL_PREFERRED, TASK_TYPE) that pick the right model per query. Learns from past evaluation results.	An LLM that is "best for one query" is not best for every query. Dynamic selection wins.
Cost Tracker	Real-time USD spend per model, per session, persisted to disk.	Cloud LLM bills can blow up silently mid-experiment. We want to see it.
Consensus Engine (LLM Council)	Fan out to N models in parallel, then synthesise via majority vote, confidence-weighted, or judge-LLM.	Implements Condition D of the Embodied LLM Arena (ensemble vs. single model).
LLM-as-Judge	Three research-backed patterns: Referenceless (G-Eval), Reference-Based, Pairwise (with position-bias swap-and-aggregate).	Open-ended HRI dialogues have no gold-standard answer. We need an LLM judge to score "naturalness" automatically.
ELO Leaderboard	Chatbot-Arena-style ratings, with per-category leaderboards (overall, reasoning, embodied_hri, ...).	Lets us produce <i>the</i> central deliverable of this thesis: a leaderboard of LLMs ranked by their suitability as social-robot brains.
RAG Pipeline	ChromaDB-backed retrieval over our own documents (TXT, CSV, PDF), with optional faithfulness scoring. Falls back to keyword search if ChromaDB is unavailable.	Grounds the robot's answers in factual data about the DIBRIS lab and Prof. Sgorbissa's research instead of generic LLM trivia.
LangGraph Agent Pipeline	Whisper STT -> language detect -> task classify (T1–T4) -> RAG / direct LLM / multilingual -> robot action plan -> log. Nine nodes.	The pipeline is the robot's brain. Every spoken interaction flows through this graph.

Robotics Bridge	Abstract RobotBridge with a concrete PepperBridge over HTTP. Auto-detects the robot. Falls back to stub mode when no robot is reachable — so the entire pipeline runs on a laptop with no robot at all.	Lets you develop the AI stack without ever booting Pepper. Critical for fast iteration.
Three Trigger Modes	text, touch, vad — the participant can type, head-touch, or just speak.	Mandatory for the experimental study: we need different conversational starts for different participants and different scenarios.
Face Tracking	NAOqi ALTracker follows the participant's face during conversation, increasing perceived attentiveness.	Increases experimental ecological validity and reduces "is it talking to me?" confusion.
Plugin Architecture	Adding a new LLM is 7 lines of YAML, zero Python changes.	Models change every two weeks in this field. Hard-coded provider lists become technical debt overnight.
278+ Test Suite	All pytest, all mocked, no API keys required. Currently 289 tests pass.	The whole codebase can be CI'd on a laptop with zero LLM cost.

Key Concepts in 60 Seconds (Beginner's Glossary)

If any of these are unfamiliar, here is the smallest possible definition you need to follow the rest of the book. The full glossary lives in Appendix B.

Concept	One-line definition
API key	A secret password that lets your code call a cloud LLM provider's HTTPS service.
Token	A small chunk of text — roughly one short English word or a few characters — that LLMs charge per million.
Ollama	A free local LLM runtime that listens on <code>http://localhost:11434</code> and runs open-weight models on your own machine. No API key.
LiteLLM	A Python library that wraps 100+ LLM providers behind one identical interface.

LangGraph	A Python library by the LangChain team for building stateful, branching agent pipelines as directed graphs of nodes.
ChromaDB	A small embedded vector database. Used here as the storage for RAG document chunks.
RAG	Retrieval-Augmented Generation — search a corpus of documents, then ask the LLM to answer using the retrieved snippets as grounding.
LLM-as-Judge	The practice of using one LLM to grade the output of another. Surprisingly effective; well-studied since 2023.
ELO	A chess-derived rating system in which beating a higher-ranked opponent earns more points. +100 $\approx$ a 64% win rate.
Consensus / Council	Asking many models the same prompt in parallel and merging the answers.
HRI	Human-Robot Interaction — the academic research field this project lives in.
NAOqi	Pepper's on-board middleware operating system. Locked to Python 2.7.
Choregraphe	SoftBank's graphical IDE for programming Pepper. Ships a <i>virtual robot</i> (a simulator) that runs entirely in software with no real hardware required.

Academic Context (Researcher's Sidebar)

OmniLLM's design positions it squarely at the intersection of three active research streams:

1. **LLM evaluation and benchmarking.** Existing text-based benchmarks (MMLU [Hendrycks et al., 2021], HumanEval [Chen et al., 2021], GSM8K [Cobbe et al., 2021], LiveBench [White et al., 2024]) measure capabilities in isolation. Chatbot Arena [Chiang et al., 2024] introduced large-scale human pairwise ranking but remains text-only. OmniLLM's ELO scorer is directly inspired by Chatbot Arena's methodology and extends it to embodied interaction.
2. **LLM-as-Judge methodology.** G-Eval [Liu et al., 2023] established that GPT-4-class judges correlate strongly with human ratings on natural language generation tasks. Zheng et al. (2023) demonstrated that LLM judges can replicate human preferences in chatbot evaluation with >80% agreement. OmniLLM's `evaluator.py` implements all three patterns (referenceless, reference-based, pairwise with position-bias correction).
3. **Pepper-LLM integration.** The very first Pepper+LLM systems used a *single* model wired through a *single* speech pipeline (Hafez 2024; Mauliana et al. 2025). OmniLLM's contribution is to make the model *interchangeable* and to route between models dynamically based on task type — bringing the multi-model orchestration practice of the LLM benchmarking community into HRI for the first time.

For a complete annotated bibliography see Appendix G.

Chapter 2 — Why a Physical Humanoid Robot? The Motivation Behind Pepper

At a Glance (Owner's Recap)

You could run every script in this repository against a terminal and never own a robot. The reason we run it through Pepper anyway is that **the body changes the conversation, and the conversation is what we are trying to study**. Five Pepper-specific properties motivate the embodiment: (1) Pepper is the most-cited humanoid robot in HRI literature, giving us a comparable baseline; (2) Pepper's social presence shifts how participants perceive an LLM's answers along dimensions like *trust*, *naturalness*, and *competence* that text alone cannot measure; (3) Pepper has a built-in gesture library, LEDs, a tablet, and microphones that map cleanly onto the affective and informational channels of natural human conversation; (4) Pepper is "good enough" hardware — not too expensive, not too fragile, deployed in hundreds of universities; (5) Pepper's NAOqi SDK forces the Python 2.7 ↔ 3.x split that, far from being a nuisance, is a fair model of how AI stacks will be deployed in real-world embodied systems for years to come.

The Walk-Through (Beginner's Path)

Why Bother With Embodiment At All?

Every modern LLM benchmark we just cited in the previous chapter is conducted on text. You type a prompt, the LLM returns a string, a metric scores the string. Done. So why complicate things with a physical robot?

The honest answer is that **a chatbot in a browser and a chatbot inside a robot are different products even when the underlying LLM is identical**. Consider the same exchange in two media:

- **Text on a screen:** "Hello Pepper, how are you today?" -> "Hello! I'm feeling bright and cheerful today, thank you! How about you?"
- **Real robot in a room:** A 1.2-metre humanoid turns its head to face you, its eye-LEDs glow soft green, it raises one arm in a wave, and a synthesised voice with mild prosodic variation says the same sentence.

The information content is identical. The **interaction** is not. The robot makes eye contact, holds gaze, has a body posture, and exists in your physical space. Decades of HRI research (see Bartneck et al. 2009, Fox & Gambino 2021, Bonarini 2020 — full citations in Appendix G) have established that participants:

- attribute more **agency** to embodied systems than to disembodied ones, even when both are running identical software (Horstmann & Krämer 2022);
- form trust in the embodied system through both verbal and non-verbal channels (Etemad-Sajadi et al. 2022);
- forgive errors more readily when an embodied robot apologises (Hoffmann et al. 2020);
- treat the conversation as more *social* and less *transactional* (Sugiyama 2021);
- experience more emotional engagement and report more enjoyment (Betriana et al. 2022; De Carolis et al. 2021).

If LLM-A and LLM-B produce equally accurate answers but participants rate LLM-A as more *trustworthy* when those answers come out of a robot, that gap is invisible to text benchmarks. The **Embodied LLM Arena** is designed precisely to measure that gap.

Why Pepper Specifically, and Not Another Robot?

The HRI research community has many humanoid platforms. The author's choice of Pepper is driven by:

1. **Citation density.** Pepper is, by a wide margin, the most-published humanoid robot in HRI between 2014 and 2026. The library accompanying this thesis (see Appendix G) contains over 80 peer-reviewed studies that use Pepper specifically. Choosing Pepper means our results are directly comparable to a deep prior literature.
2. **Availability.** Prof. Antonio Sgorbissa's HRI lab at DIBRIS, University of Genoa, owns a working Pepper. The unit is physically accessible to the author for the experimental study. (Several papers from the same lab — Chiang, Bruno, Menicatti, Recchiuto, Sgorbissa 2019 — exemplify the institutional context.)
3. **Built-in social affordances.** Pepper ships from the factory with: - a large pre-built animation library (over 100 named gestures: `wave`, `nod`, `point_left`, `Explain_7`, `Hey_1`, etc.) - text-to-speech in 20+ languages via NAOqi's `ALTextToSpeech` and `ALAnimatedSpeech` - a chest-mounted touchscreen tablet (`ALTabletService`) for fall-back text I/O - eye LEDs controllable by hex colour (`ALLeds`) - a face-tracking system (`ALTracker`) that finds and follows human faces - 4 directional microphones with energy-thresholded VAD (`ALAudioDevice`) - basic person-detection and engagement modules out of the box (`ALEngagementZones`, `ALPeoplePerception`)

We use all of these. A bare-bones robot platform would force us to re-implement large chunks of social signal infrastructure before we could even start the LLM-comparison study. 4. **Affordable failure.** A new Pepper costs around €15,000 — expensive, but an order of magnitude cheaper than a humanoid robot like an iCub or an Atlas, and orders of magnitude cheaper than a research-grade dual-arm manipulator. If a participant trips over Pepper's base or a guest spills coffee on its tablet, the lab is not destroyed. 5. **The NAOqi / Python 2.7 constraint is realistic.** Pepper's middleware predates the modern AI stack by half a decade. We cannot upgrade it. Every choice we make in OmniLLM about *how the AI stack talks to the robot* has to confront this. In a sense the Python-2.7-vs-Python-3.11 boundary is a perfect miniature of the real-world deployment constraint that **AI moves faster than robot firmware**. By solving it explicitly here, we end up with a deployment pattern (HTTP bridge between an Old World and a New World) that generalises to any future legacy robot platform.

What Pepper is *Not* Good At

To set expectations honestly:

- **Pepper cannot walk.** It rolls on omnidirectional wheels and falls over on stairs.
- **Pepper's depth sensor is mediocre** in bright daylight (Bauer et al. 2019, Bauer et al. 2019b).
- **Pepper's microphones are noisy** at conversational range beyond 2 metres.
- **Pepper's on-board CPU cannot run modern LLMs.** Anything heavier than a 1B-parameter quantised model is out of the question. This is the central reason we need an off-board Python 3.x AI server.
- **NAOqi 2.5.5.5 on Windows 11 has a loopback bug** that requires binding `ALBroker` to `127.0.0.1` explicitly when connecting to Choregraphe's virtual robot. Real Pepper (running NAOqi natively on Linux) does not have this bug. (We will revisit this in Chapter 16.)

Academic Context (Researcher's Sidebar)

The research literature on Pepper falls into roughly four buckets:

1. **Pepper as receptionist / front-desk** (Gardecki et al. 2018; Stommel et al. 2022; De Gauquier et al. 2018; Carros et al. 2022). Question-answering dialogue is central; most use one rule-based or NLU pipeline; OmniLLM extends this by making the dialogue brain LLM-driven and multi-model.
2. **Pepper for education / therapy** (Lehmann & Rossi 2019; Pigureddu & Gena 2023; Castellano et al. 2022; Forbrig 2019). Tasks are highly structured. OmniLLM's open-ended T3 (Social Conversation) condition aligns with this literature.
3. **Pepper navigation / vision** (Alhmiedat et al. 2023; Bista et al. 2021; Ardón et al. 2019; Perera et al. 2017). Not the focus of this thesis but informs our T2 (Navigation) task design: we ground "where is Room 305?" in the RAG-indexed lab map rather than asking Pepper to actually locomote there.
4. **Pepper + LLM** (Hafez 2024; Mauliana et al. 2025; Rahimi et al. 2025; Dogan et al. 2025). The youngest cluster — almost all post-2024 — and the one this thesis pushes forward. To the author's knowledge, **no prior published work has compared multiple LLMs as interchangeable backends in the same Pepper deployment**. This gap is the entire raison d'être of the Embodied LLM Arena.

The four-task taxonomy (T1 Information Retrieval, T2 Navigation, T3 Social Conversation, T4 Multilingual) follows the standard HRI dialogue-act categorisation used by Hsieh et al. (2017) and Gross & Krenn (2023), translated into terms relevant for a knowledge-grounded campus deployment.

Chapter 3 — What Happens When the Robot is Absent — the Three Operating Modes

At a Glance (Owner's Recap)

OmniLLM has been designed so that **the absence of a physical robot is a first-class operating mode, not a fallback or a hack**. Three modes are supported, and all three exercise the same Python 3.x AI pipeline (LangGraph -> RAG -> LLM -> action plan) end-to-end:

Mode	Robot	Microphone	Speakers	Gestures	Used for
A — No Robot	None	None (text-only via curl or CLI)	None	Printed JSON	All AI-stack development; CI; the 278 pytest tests; participant pre-screening
B — Virtual Pepper (Choregraphe)	Software simulator	None (text trigger only)	"Robot Dialog" panel in Choregraphe GUI	Animations attempted; many report "behavior not installed" — see below	Live demos when no real robot available; this book's diagrams; the 2026-05-20 pilot
C — Real Pepper	Hardware	4 head microphones, VAD or head-touch trigger	Pepper's stereo speakers	Full NAOqi animation library	The actual experimental study at DIBRIS

The two non-real modes both exist for a single reason: **iteration speed**. You should not need a €15,000 robot booted, networked, and woken up to test whether the LangGraph node graph still routes correctly after a code change. Most of OmniLLM's development happened in Mode A on a laptop.

The Walk-Through (Beginner's Path)

Mode A — Pure Text, No Robot at All

This is the default operating mode if you just clone the repo, install dependencies, and run things. There are three sub-modes within Mode A:

(A1) The CLI:

```
omnillm ask "What time does the lab open?" -m openai-gpt4o-mini
omnillm route "Where is Room 305?" --strategy TASK_TYPE
omnillm council "What is consciousness?" --strategy synthesis
```

```
omnillm evaluate --output results/eval_2026.json
omnillm leaderboard --category embodied_hri
```

The CLI is your fastest path to exercising the gateway / router / consensus / evaluator / scorer subsystems without any HRI infrastructure.

(A2) The Flask AI server + curl:

Start the server:

```
python -m omnillm.server.app --host 127.0.0.1 --port 5000
```

Then talk to it with curl (Windows PowerShell):

```
curl -X POST http://127.0.0.1:5000/interact `
-H "Content-Type: application/json" `
-d '{"text": "Where is Room 305?", "participant_id": "P001", "session_id": "s1", "condition": "C", "rag_enabled": true}'
```

You get back a JSON RobotAction — `speech`, `gesture`, `emotion_led`, and `metadata` — exactly what Pepper would receive over HTTP. No robot needed.

(A3) Programmatic, from Python:

```
import asyncio
from omnillm.gateway import LLMGateway
from omnillm.rag.pipeline import RAGPipeline
from omnillm.hri.agent_graph import build_hri_graph

async def main():
    gw = LLMGateway()
    rag = RAGPipeline(gateway=gw)
    rag.index_directory("knowledge_base/")
    graph = build_hri_graph(gateway=gw, rag=rag)
    result = await graph.ainvoke({
        "utterance": "What time does the lab open?",
        "participant_id": "P001",
        "session_id": "s1",
        "condition": "A",
        "rag_enabled": True,
    })
    print(result["response_text"])      # the spoken text
    print(result["robot_action"])      # the full JSON action plan

asyncio.run(main())
```

This is what the test suite does. All 289 tests run in Mode A.

Mode B — Choregraphe's Virtual Pepper

Choregraphe is SoftBank's official IDE for Pepper development. It ships with a *virtual robot* — effectively a full Pepper simulator with a rendered 3-D body, working text-to-speech, a placeholder set of gestures, and a faked sensor stream. It is everything an embodied developer needs short of an actual robot, and SoftBank distributes it free for research.

Choregraphe runs only on Windows (officially) and works by listening on a local TCP port that changes every launch — it picks a random port in the 40000–60000 range. When you launch Choregraphe you must:

1. Pull down **Connection -> Connect to...**
2. Select the AKSHITA virtual robot in the list

3. Read off the **port number** in the connection dialog (e.g. 49959)
4. Pass that port to OmniLLM via `--robot-port 49959`

The `make_pepper_bridge()` helper in [omnillm/robotics/pepper.py](#) will probe `127.0.0.1:49959` (and a list of recently-seen ports) automatically, so in practice you rarely need the `--robot-port` flag once you have run the system once on a given machine.

(!) **Two well-known Choregraphe limitations:**

1. **No real microphone or camera.** The virtual robot's `ALAudioRecorder` returns silence; its `ALVideoDevice` returns a black frame. This means you must use `--trigger text` (typed input) for any conversational test on the virtual robot — `--trigger touch` and `--trigger vad` cannot work.
2. **Animations may report "behavior not installed."** Choregraphe ships a *placeholder* animation library that does not include the full set of named animations real Pepper carries (`animations/Stand/Emotions/Positive/Enthusiastic_1`, `animations/Stand/Gestures/Explain_7`, `animations/Stand/Gestures/Hey_1`, etc.). When you send Pepper a gesture command in Choregraphe, NAOqi returns `behavior not installed`. **This is expected, not a bug.** Pepper still speaks the text and changes its LED colour; only the gesture is silently dropped. The 2026-05-20 pilot run with the author as P000 logged exactly these errors — see Chapter 29.

(Beginner) **Why this gesture limitation does not break the experiment.** The pilot ran the full 20-interaction matrix (5 conditions × 4 task types) on Choregraphe's virtual robot and produced fully usable speech, latency, RAG, and routing data even when gestures failed. The experimenter sees `gesture: false` in `robot_result.executed` but the `speech`, `model_id`, `condition`, `latency`, and `faithfulness` fields are all populated correctly. We can therefore *develop* and *pilot* in Mode B before ever booting real Pepper, then run the actual study in Mode C with no code changes — only a configuration flip.

Mode C — The Physical Pepper at DIBRIS

This is the mode that matters for the thesis. Pepper lives in Prof. Sgorbissa's HRI lab. The connection sequence is:

1. **Press the chest button once.** Pepper says its IP aloud over the speaker (e.g. "One nine two, dot one six eight, dot one, dot one hundred"). Call this `PEPPER_IP`. The same IP appears in the tablet's *About -> Network* screen.
2. **Make sure laptop and Pepper are on the same Wi-Fi LAN.** Pepper does not need internet access; only LAN reachability to your laptop. `ping <PEPPER_IP>` from a PowerShell prompt should succeed.
3. **Start the AI server** on your laptop, bound to `0.0.0.0` so Pepper can reach it:

`powershell python -m omnillm.server.app --host 0.0.0.0 --port 5000` 4. **Start the NAOqi bridge** — the small Python 2.7 HTTP server that lives near (or on) Pepper:

`powershell C:\Python27\python.exe omnillm\server\ naoqi_bridge_server.py ` --robot-ip <PEPPER_IP> --robot-port 9559 ` --bind 0.0.0.0 --bridge-port 6000` 5. **Run the participant session driver:**

`powershell .\env\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py ` --participant P001 ` --server http://127.0.0.1:5000 ` --bridge http://127.0.0.1:6000`

The complete walk-through (with expected output and a symptom->cause->fix table) is [Chapter 21: Day Zero](#).

Academic Context (Researcher's Sidebar)

The decision to support a no-robot mode with the *same* code path that drives a real robot has a methodological pay-off: every result reported in the thesis is **reproducible by any reader who downloads the repo, even if they do not own a Pepper**. They will not get the embodied questionnaire data (which intrinsically requires a body), but they will get identical text outputs, identical RAG faithfulness scores, identical latency profiles, and identical cost tracking from the AI server. This is the open-science equivalent of providing both the Jupyter notebook *and* the raw data alongside a paper.

The "fall back to stub mode" design of `make_pepper_bridge()` is borrowed from established robot-software practice — most robotics frameworks (ROS, NAOqi itself, the Webots simulator) support a "simulated" vs "real" toggle. OmniLLM goes a step further by making the toggle automatic: the same `await bridge.execute_action(action)` call works in all three modes; only the bridge's `mode` attribute changes ("`server`" / "`naoqi`" / "`stub`"). For a code-level walkthrough see Chapter 13.

Chapter 4 — Scope, Hypotheses, and the Embodied LLM Arena Study

At a Glance (Owner's Recap)

OmniLLM is the *engineering substrate* for a single research study: the **Embodied LLM Arena**. The study has three formal hypotheses (H1, H2, H3), four task types (T1–T4), five experimental conditions (A–E), one robot (Pepper), one knowledge base (DIBRIS / Sgorbissa lab), and ~15 participants run individually over a ~one-month window. The leaderboard output of the study (per-condition mean Likert scores + pairwise ELO ratings) is the central scientific contribution.

The Walk-Through

The Research Gap

As established in Chapters 1–2, existing LLM benchmarks are entirely text-based. The handful of Pepper-LLM studies that exist (Hafez 2024; Mauliana et al. 2025) use a *single* LLM and report a single set of HRI metrics, with no inter-model comparison. **No prior work has:**

1. compared multiple LLMs as interchangeable backends for the *same* social robot;
2. applied dynamic smart routing between LLMs *during live HRI*;
3. tested whether embodied rankings agree with text-only rankings.

The Embodied LLM Arena fills exactly this gap.

The Three Hypotheses

#	Hypothesis	How we test it
H1	Embodied HRI rankings differ significantly from text-only benchmark rankings.	Compare per-condition mean Likert score (this study) against MMLU + Chatbot-Arena rank for the same model. Spearman correlation; expect $p < 0.7$.
H2	Dynamic smart routing (Condition C) produces higher participant satisfaction than any single fixed model.	Pairwise win-rate of C vs A and C vs B in the final preference question. Expect C to win >55% of comparisons.
H3	RAG-augmented responses (A vs E) are rated more accurate and trustworthy across all backends.	Within-subject contrast: A.accuracy minus E.accuracy. Expect positive mean difference; one-sided t-test.

The Four Task Types (T1–T4)

Task	Name	Example prompt	Why a robot matters	Routing
T1	Information Retrieval	<i>"What time does the lab open?"</i>	Pepper greets a visitor and explains while gesturing. Tests RAG faithfulness directly.	-> RAG pipeline -> LLM
T2	Navigation / Guidance	<i>"Where is the Pepper room at DIBRIS?"</i>	Pepper <i>physically points</i> with its arm and shows a map on its tablet. Tests gesture-speech synchrony.	-> RAG + Gesture Planner -> LLM
T3	Social Conversation	<i>"Hello Pepper, how are you today?"</i>	Physical presence transforms a vacuous-sounding exchange into something perceived as warm. Tests naturalness.	-> Direct LLM, no RAG
T4	Multilingual	<i>"Ciao Pepper, dove si trova la stazione di Brignole?"</i>	Embodiment makes multilingual feel more immersive (cf. Carolis et al. 2021). Tests cross-language gracefulness.	-> Language detector -> claude-haiku

Participants are explicitly told they may **ask in any language and may switch languages mid-conversation**. The language detector and multilingual routing handle this automatically.

The Five Experimental Conditions (A–E)

Condition	LLM	RAG	What it isolates
A	GPT-4o-mini (fixed cloud baseline)	ON	Cloud-baseline upper bound
B	Llama 3:8b via Ollama (fixed local)	ON	Free / offline baseline. Tests whether a local model is "good enough."
C	OmniLLM smart-routed (different LLM per task type)	ON	The effect of dynamic routing itself
D	Consensus council (GPT-4o-mini + Claude Haiku + Gemini Flash, synthesised)	ON	Ensemble vs single model
E	GPT-4o-mini, RAG disabled (control)	OFF	RAG's contribution, all else held constant

Condition E is the **control** for H3: A and E differ only in whether RAG is enabled, so any difference in accuracy/trust ratings is attributable to RAG. (The May 2026 pilot caught a routing bug where Condition B was silently running Condition A's model; this is documented in detail in Chapter 29.)

Participants: How Many, How Often, Which Conditions

This is the question the README and the first edition of the book never quite answered. The answer is below — and the reasoning is worth knowing because it will come up if you ever defend the design choice.

Target N = 15 participants. Recruited over a ~one-month window in summer 2026, run individually at the lab (one participant per session, no overlap).

Each participant experiences ALL FIVE conditions in a single ~30-minute session. The pilot confirmed that 20 interactions (5 conditions × 4 task types) takes ~25 minutes including questionnaires — comfortably within the participant's attention window. This is the **within-subjects, fully-crossed design**.

Design choice	Why
All 5 conditions per participant (not 3 of 5)	Maximises statistical power (each participant is their own control). The <code>experiment.py</code> docstring describes a 3-of-5 design as an alternative for studies where session length must be ≤15 minutes; in our case the pilot showed 30 minutes is acceptable.

N = 15 (not 30 or 50)	Pragmatic — DIBRIS is a single-robot lab. A power analysis for paired t-test of A vs E accuracy at $\alpha=0.05$ power=0.80 medium effect size $d=0.5$ gives $n \approx 27$, but with a within-subjects 5-condition repeated-measures ANOVA the same effect requires only $n \approx 11$. We over-provision to 15 to allow for two drop-outs.
~One-month spread	Avoids fatigue and lets us recruit through department mailing lists rather than a single batch. Also: if the model API costs or the robot itself behave differently across days, the temporal spread reduces a single-day artefact.
Counterbalanced condition order	Latin-square; see Chapter 25 for the explicit assignment table.
Pilot participant (P000 — the author)	Not counted in the final N. The two P000 runs on 2026-05-20 served only to debug the system. Their data is excluded from the analysis.

(!) **If you change N or the design, document it in [omnillm/hri/experiment.py](#)** — the docstring should always state the canonical design, not the alternatives.

Ethics, Consent, GDPR

A live human-subjects study at the University of Genoa requires:

- a **consent form** signed by each participant before any data is recorded;
- a **GDPR data-handling notice** explaining what data is collected, how long it is stored, who can access it, and how to request deletion;
- **anonymisation at logging time**: participants are referred to by code (P001, P002, ...) in all stored files. The mapping from code to real identity is held by the experimenter on paper, separately from the digital data, and destroyed at the end of the study;
- **ethics committee approval** (Comitato Etico di Ateneo). At the time of writing, the protocol has been drafted and is being prepared for submission. The book reflects best-practice defaults; readers running their own version of this study elsewhere must obtain their own institutional approval.

The exact templates the author uses are reproduced in Chapter 28.

What is *Not* in Scope

To keep the project finite:

- **No fine-tuning of any LLM**. All models are used out-of-the-box. The thesis benchmarks them as-shipped.
- **No autonomous locomotion**. Pepper stays put (or rolls only on command in T2). We do not study navigation as a robotics-research task.
- **No vision-based interaction**. Face detection and tracking are used purely for engagement signal (look-at-the-speaker). We do not extract emotion, identity, or object recognition from the camera.

- **No multi-party dialogue.** One participant at a time. Group-interaction effects are documented in the future-scope chapter.
- **No deception or distress protocols.** The robot never lies about its nature, and the interactions are designed to be pleasant. (Compare with adversarial-prompting studies which are entirely out of scope.)

Academic Context (Researcher's Sidebar)

The experimental design follows the **within-subjects repeated-measures** paradigm standard in HRI (Mavrogiannis et al. 2019; Lo et al. 2019; Stommel et al. 2022 for similar Pepper-based survey studies). Counterbalancing addresses order effects (Bartneck et al. 2009 noted that early-exposure conditions tend to receive more generous ratings). A Latin-square assignment per condition $\times$ per task ensures every condition appears in every ordinal position across the $N=15$ participants. The full assignment table is given in Chapter 25.

The five-condition factorial is deliberately *modest*: it crosses model-source (fixed vs. routed vs. council) with RAG presence, not with prompt-style or speech speed or gesture density. The choice keeps the participant cognitive load tractable and the statistical model interpretable, at the cost of not exploring some interesting interactions (e.g. *gesture density* $\times$ *model strength*). Future-scope variants are discussed in Chapter 32.

Part II — Architecture, Flow, & Tech Stack

Part I told you what we are building and why. Part II tells you what the system looks like when it is running.

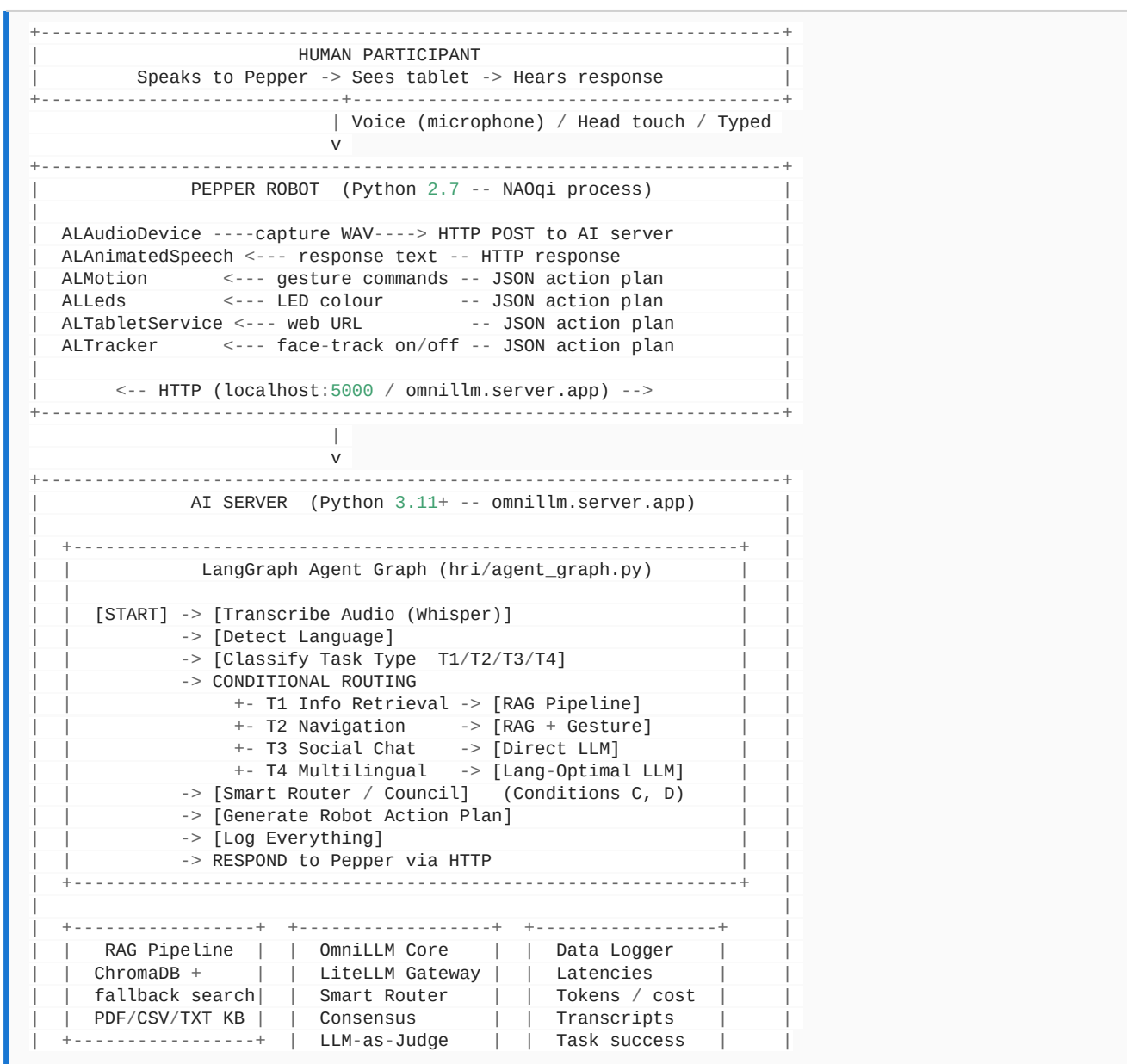
Chapter 5 — The Three-Layer System Architecture

At a Glance

OmniLLM lives across three concentric layers: a **Human Layer** (the participant talking, listening, looking), a **Robot Layer** (Pepper running NAOqi under Python 2.7), and an **AI Layer** (the modern Python 3.11+ stack on your laptop or a server). The three layers communicate exclusively over HTTP-and-JSON, which is what makes the Python-2.7/Python-3.x divide tolerable. The interaction itself flows in a single direction: human speech enters the Robot Layer, the Robot Layer marshals it to the AI Layer over HTTP, the AI Layer responds with a `RobotAction` JSON, and the Robot Layer enacts that action (speech + gesture + LED + tablet) back at the human.

The Big Picture

The system is most usefully drawn as three nested boxes with two HTTP arrows crossing the box boundaries:



		ELO Scorer		+-----+	
		+-----+			
				+-----+	
		LLM Backends (via LiteLLM)			
		GPT-4o-mini	Claude Haiku	Gemini 2.5 Flash	DeepSeek
		Ollama local: Llama 3:8b	Qwen 2.5:7b	Mistral:7b ...	
				+-----+	
				+-----+	

Why HTTP, and Why JSON?

Two pragmatic reasons.

Reason 1 — Python-version isolation. NAOqi's official Python bindings are compiled against Python 2.7. CPython 2.7 reached end-of-life on 2020-01-01. The modern AI stack (LangGraph 1.x, LangChain 0.3+, litellm, openai-python ≥ 1.0, chromadb ≥ 0.4, anthropic ≥ 0.30, etc.) requires Python 3.10+ and emits syntax errors on Python 2.7. The two runtimes cannot coexist in a single process. HTTP between two separate processes is the simplest portable boundary.

Reason 2 — Locational decoupling. The Python 2.7 process *can* live on the robot itself (Pepper's on-board CPU runs Linux and ships with Python 2.7 pre-installed), or it can live on a nearby laptop on the same LAN. The HTTP boundary is identical either way. This matters because: (a) running large LLM agents on Pepper's on-board CPU is infeasible — so the AI Layer always lives off-board; (b) running NAOqi *itself* off-board (i.e., talking from a laptop to Pepper's broker over the LAN) gives us the same effect as if NAOqi ran on the robot, *and* gives us the easy fallback to Choregraphe's virtual robot, which only listens on 127.0.0.1. So all our Python 2.7 entry points actually live on the laptop in our default setup.

JSON, meanwhile, is the only data format that travels lossless through every layer of the stack (Whisper output, LLM output, LiteLLM cost accounting, ChromaDB metadata, NAOqi's ALAnimatedSpeech annotated-text format, Pepper's tablet HTML). The choice was not exotic — but it was deliberate.

The Layer Responsibilities Table

Layer	Process	Language	Job	Cannot do
Human	(the participant)	n/a	Speak, listen, touch the tablet, watch	(the experimenter) writes their answers on the questionnaire
Robot Layer	<code>naoqi_client.py</code> or <code>naoqi_bridge_server.py</code>	Python 2.7	Drive Pepper's actuators and sensors via NAOqi services	Run a modern LLM; install a pip package newer than ~2019
AI Layer	<code>omnilm.server.app</code> (Flask)	Python 3.11+	Speech-to-text, language detection, task classification, RAG, LLM querying, gesture planning, logging	Talk to NAOqi directly

Academic Context

The HTTP-bridge pattern adopted here is well-precedented in robotics: ROS itself uses a publish-subscribe message bus (`roscore`) that is process-external for similar reasons (different nodes can be in different languages and on different machines). Pepper-specific bridges — for example *PePUT* (Ganal et al. 2023) and the QiSDK-based bridge in *InjectMeAI* (Ampadu et al. 2022) — adopt the same pattern.

The split has a side benefit for replicability: a reader can run the *entire* AI Layer on their own machine without the robot present, drop in their own Robot Layer (a different humanoid platform, ROS, or even a Unity simulator like *PePUT*), and reproduce most of the AI-side results. The HTTP contract acts as a published, language-independent API for "Pepper's brain."

Chapter 6 — The Lifecycle of a Single Question, End to End

At a Glance

Trace a single question from microphone to robot action. Two paths exist for the *capture* phase (head-touch capture vs. continuous VAD vs. typed text), but they all converge on the same `POST /interact` HTTP call. From there on, the path is identical for every interaction: Whisper STT -> language detection -> task classification -> conditional routing -> response generation -> action-plan generation -> logging -> HTTP response -> NAOqi enactment.

Step-by-Step Trace

The participant says: *"Where is the Pepper room at DIBRIS?"*

TIME	LAYER	EVENT
0.00	HUMAN	Says "Where is the Pepper room at DIBRIS?" out loud.
0.20	PEPPER	ALAudioDevice captures 1.8s WAV at 16 kHz mono. ALAudioRecorder saves /tmp/utterance.wav.
0.21	PEPPER (P2.7)	naoqi_client reads the WAV, base64-encodes it.
0.22	PEPPER (P2.7)	POST http://<server-ip>:5000/interact Body: {"audio": "...base64...", "participant_id": "P003", "session_id": "uuid", "condition": "C", "rag_enabled": true}
0.22	AI SERVER	Flask receives the request.
0.23	AI SERVER	Resolves Condition C -> effective model = (smart-routed). State dict built and passed to graph.ainvoke() .
0.23	GRAPH NODE 1	transcribe_audio: WhisperSTT.transcribe(audio_bytes)
0.65	GRAPH NODE 1	-> "Where is the Pepper room at DIBRIS?"
0.65	GRAPH NODE 2	detect_language: returns "en", confidence 0.92
0.66	GRAPH NODE 3	classify_task: hits "where" and "room" -> NAVIGATION (0.95)
0.66	GRAPH ROUTE	conditional edge picks "nav_rag" branch
0.66	GRAPH NODE 4	nav_rag: rag.query(utterance, model_id=None) ChromaDB top-3 chunks from university_map.txt + lab_info.txt
0.85	GRAPH NODE 4	LLM call (smart-routed -> gpt-4o-mini for navigation)
1.40	GRAPH NODE 4	-> "The Pepper room is at the end of the ground-floor corridor, on the right..."
1.40	GRAPH NODE 5	GesturePlanner picks gesture="point_right", led="#00AAFF" smart_router: condition is "C" but response_text is set , so no further routing - pass through.
1.41	GRAPH NODE 6	generate_action_plan: assembles the RobotAction dict.
1.41	GRAPH NODE 7	log_interaction: writes one InteractionRecord to disk.
1.42	AI SERVER	Returns JSON to Pepper.
1.42	PEPPER (P2.7)	Receives JSON , parses speech/gesture/emotion_led.
1.43	PEPPER	ALAnimatedSpeech.say("The Pepper room is...") ALMotion runBehavior("animations/Stand/Gestures/Right_1") ALLeds.fadeRGB("FaceLeds", 0x00AAFF, 0.3)
1.50	HUMAN	Hears the answer, sees the arm raise to the right , sees the eye-LEDs turn blue.
4.30	HUMAN	Response finished. Floor returns to participant.

(Beginner) **Why the `_start_time` field in `HRIGraphState`.** The state has a `_start_time` set in the transcription node (the first node that always runs). The `log_interaction` node at the very end computes `total_latency = (time.monotonic() - start) * 1000`. So the "end-to-end latency" we log includes Whisper STT, language detection, task classification, RAG retrieval, LLM generation, gesture planning, and JSON marshalling — but **not** the time the WAV took to travel from Pepper to the server, and **not** the time NAOqi takes to actually utter the speech. We measure compute, not the network or the actuator. This is a deliberate choice: the network and the actuators are dependent on the lab's Wi-Fi and Pepper's hardware, neither of which is the LLM's fault.

The State Dictionary

The graph passes one dictionary through every node. Each node *adds* fields; nothing is overwritten without intent. The schema is the `HRIGraphState` dataclass — but at runtime, LangGraph 1.x stores it as a dict, and we use a `_merge_state` wrapper to make every node behave as "the new state is `{**old, **node_output}`" rather than "the node return value replaces the state." This is the *single line of code* that fixed the empty-speech bug in the 2026-05 upgrade. See Chapter 12 for the deep-dive.

The most important fields after a full traversal:

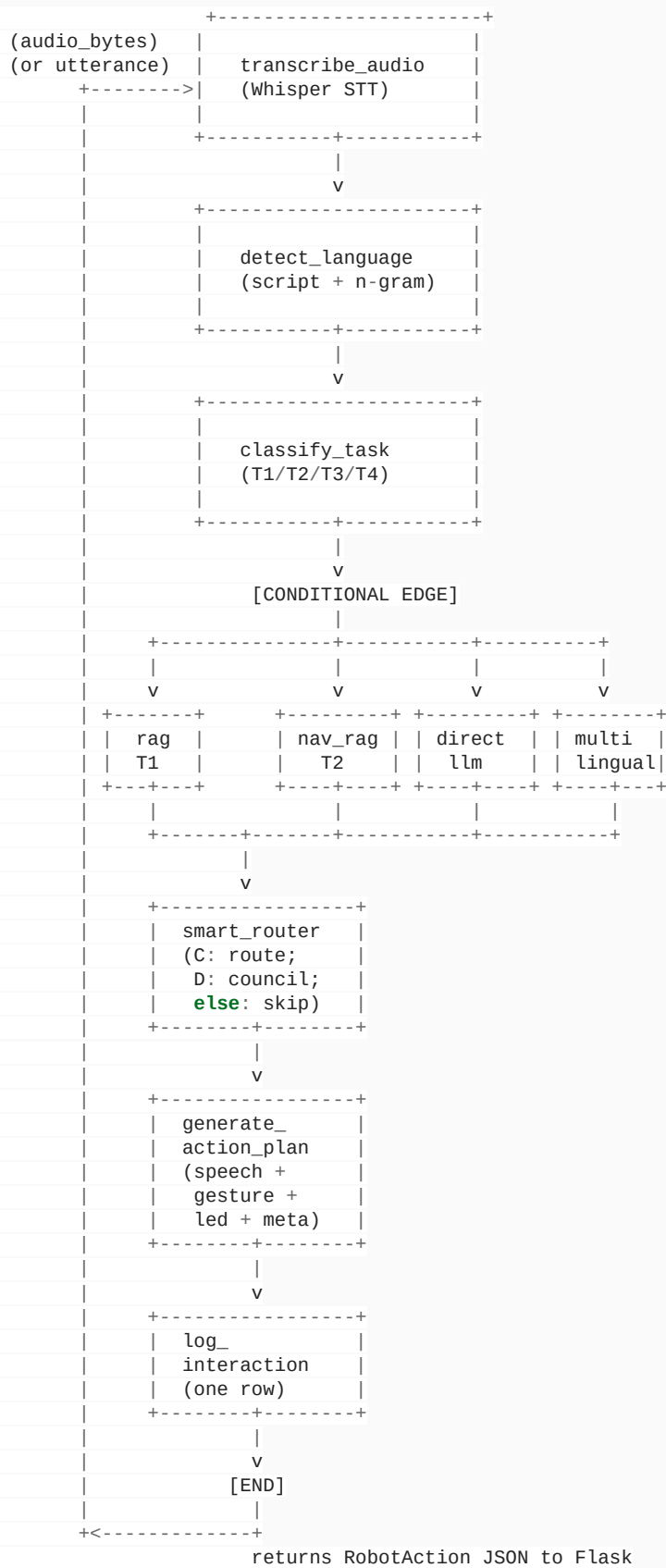
Field	Type	Set by node	Example value
<code>utterance</code>	<code>str</code>	<code>transcribe_audio</code>	"Where is the Pepper room at DIBRIS?"
<code>detected_language</code>	<code>str</code>	<code>detect_language</code>	"en"
<code>task_type</code>	<code>str</code>	<code>classify_task</code>	"navigation"
<code>task_confidence</code>	<code>float</code>	<code>classify_task</code>	0.95
<code>rag_context</code>	<code>str</code>	<code>nav_rag</code> (or <code>rag</code>)	(joined retrieved chunks)
<code>rag_chunks</code>	<code>list</code>	<code>nav_rag</code> (or <code>rag</code>)	(list of <code>DocumentChunk</code>)
<code>rag_faithfulness</code>	<code>float</code>	<code>nav_rag</code> (or <code>rag</code>)	0.87
<code>response_text</code>	<code>str</code>	LLM node	"The Pepper room is..."
<code>model_id</code>	<code>str</code>	LLM node	"openai-gpt4o-mini"
<code>gesture</code>	<code>str</code>	<code>GesturePlanner</code>	"point_right"
<code>led_color</code>	<code>str</code>	<code>GesturePlanner</code>	"#00AAFF"
<code>robot_action</code>	<code>dict</code>	<code>generate_action_plan</code>	full JSON for Pepper
<code>latency_ms</code>	<code>float</code>	<code>log_interaction</code>	1190
<code>input_tokens</code>	<code>int</code>	LLM node	412
<code>output_tokens</code>	<code>int</code>	LLM node	34
<code>cost_usd</code>	<code>float</code>	LLM node	0.0000687

Chapter 7 — The LangGraph Agent Pipeline, Node by Node

At a Glance

The brain of Pepper is a directed acyclic graph of nine asynchronous Python functions wired together by LangGraph. Every interaction creates one `HRIGraphState` dict, runs it through the graph, and gets back an updated dict. The graph is built by `build_hri_graph(gateway, rag, logger, default_model)` in [omnilm/hri/agent_graph.py](#).

The Graph, in Pictures



Per-Node Specification

#	Node name	Async function	Input fields	Output fields	When skipped
1	transcribe_audio	WhisperSTT.transcribe(audio_bytes, if audio_bytes; otherwise pass-through)	audio_bytes, utterance	utterance, _start_time	Text-only inputs skip Whisper entirely
2	detect_language	LanguageDetect(utterance)		detected_language	utterance is empty
3	classify_task	HRITaskClassifier()	utterance, detected_language	task_type, task_confidence	never
4a	rag (T1)	RAGPipeline.query(utterance, model_id=...)	utterance, model_id	rag_context, rag_chunks, rag_faithfulness, response_text, model_id, latency_ms	Routed only if task=info_retrieval
4b	nav_rag (T2)	RAGPipeline.query(utterance, model_id) + GesturePlanner.plan("navigation", gesture, led_color)	utterance, model_id	All RAG fields + gesture, led_color	Routed only if task=navigation
4c	direct_llm (T3)	gateway.query(messages) with social system prompt	utterance, model_id	response_text, model_id, input_tokens, output_tokens, cost_usd, latency_ms, gesture, led_color	Routed only if task=social_conversation OR Condition E
4d	multilingual_llm (T4)	llm tries lang-optimal model (Claude Haiku) first, falls back to GPT-4o-mini if first returns an error	utterance	Same as direct_llm	Routed only if task=multilingual

5	<code>smart_router</code>	For Condition C: re-routes the LLM call to the task-optimal model. For Condition D: fires a 3-model consensus. Otherwise pass-through.	All accumulated state	Possibly overwrites <code>response_text</code> , <code>model_id</code>	Conditions A, B, E pass through
6	<code>generate_action_plan</code>	Assembles <code>RobotAction</code> dict and assigns final gesture+LED if not set	All accumulated state	<code>robot_action</code>	never
7	<code>log_interaction</code>	<code>ExperimentLogger</code> writes one CSV row to disk	All accumulated state	<code>log_interaction_latency_ms</code> (end-to-end)	If logger not provided

Why a Graph, Not a Function Call Chain?

(Beginner) **Why bother with LangGraph at all?** The same logic could be a single `async def process_interaction(utterance: str) -> dict` with `if/elif` branches. Three reasons we use a graph:

1. **Inspectability.** LangGraph can render the graph as a Mermaid or Graphviz diagram at runtime. The Chapter 7 figure above was generated this way.
2. **Hot-swappable nodes.** Want to A/B-test a different task classifier? Swap one node in `build_hri_graph()` without touching any other code.
3. **State threading.** With a function chain you must thread variables through manually (`utterance`, then `(utterance, lang)`, then `(utterance, lang, task)`, ...). With LangGraph's `StateGraph(dict)` plus our `_merge_state` wrapper, the state grows naturally and any node can read any prior node's output.

The cost is a slightly heavier framework dependency. In return we get a brain shape that is easy to extend (e.g. adding a "safety filter" node before `generate_action_plan` would take ten lines of code).

Chapter 8 — The Tech Stack — Every Library, and Why That One

At a Glance

OmniLLM is intentionally a thin glue layer over very capable open-source libraries. The full direct-dependency list fits on one page; every choice has a one-sentence justification. Replacing a single library with an equivalent (e.g. `chromadb` -> `qdrant-client`) is a one-day refactor, not a rewrite.

The Direct-Dependency Table

Library	Used for	Why this one, not another
litellm	Unified LLM provider gateway	Covers 100+ providers with one API; cost-per-token table built in; well-maintained by BerriAI
openai	OpenAI SDK (used by LiteLLM under the hood for OpenAI/Azure)	Officially supported, fastest moving
anthropic	Claude SDK	Required for Claude streaming; LiteLLM uses it transparently
google-generativeai	Gemini SDK	Required for Gemini 2.5 features
ollama Python client	Local model runtime	Free, no API key, runs Llama / Qwen / Mistral on your laptop. Critical for Condition B (offline baseline).
langgraph	Stateful agent pipeline	Best-of-class for "graph of LLM nodes with conditional edges." Replaces hand-rolled <code>if/elif</code> .
langchain + langchain-community	RAG plumbing primitives (text splitters, document loaders)	Reuse the splitter most familiar to ML practitioners; we use only the small, stable parts.
chromadb	Embedded vector DB for RAG	Pure-Python install; no Docker; small footprint. Falls back to in-memory if disk write fails.
sentence-transformers	Local embeddings (all-MiniLM-L6-v2)	Avoids OpenAI embedding cost during dev; free; runs offline
openai-whisper	Speech-to-text (local)	The original Whisper, runs on CPU. Local STT means we never send participant audio to the cloud.
langdetect	Backup language detection	Used after our Unicode-script + n-gram heuristic fails.
flask	HTTP AI server	Smallest production-acceptable Python web framework. We use no fancy features.

aiohttp	Async HTTP client (Pepper bridge calls)	Required because the bridge driver must be async-compatible with LangGraph
pyyaml	<code>config/models.yaml</code> parsing	Standard.
rich + click	CLI rendering and arg-parsing	Pretty tables, command-discoverable help, easy testing.
python-dotenv	<code>.env</code> file loading	Keeps API keys out of git history
pytest + pytest-asyncio + pytest-mock	Test framework	289 tests; all mocked; no API keys needed

Why No Big Frameworks?

We deliberately avoid ROS, ROS 2, and Microsoft Bot Framework. Three reasons:

1. **Setup friction.** ROS adds 4–8 GB of installation and significant cognitive overhead. For our scope (one robot, two HTTP endpoints, one knowledge base), we do not need it.
2. **Python version trap.** ROS 1 is Python 2; ROS 2 is Python 3.8+. Either way, mixing in our 3.11+ AI dependencies is risky.
3. **Portability.** A Flask AI server runs on Windows, macOS, Linux, and a Raspberry Pi without modification. ROS does not.

If a future deployment needs ROS (e.g. integrating with a mobile robot like Tiago), the AI server can run *behind* a ROS node without itself becoming ROS-dependent.

Why Local Whisper?

We use OpenAI's **Whisper** Python package (the original, locally-run version), not the OpenAI cloud audio API. Three reasons:

1. **Privacy.** Participants speak in the lab. Sending raw audio to a third-party cloud is a non-trivial GDPR concern. Local Whisper means audio never leaves the laptop.
2. **Latency.** A local Whisper-base model on CPU transcribes a 5-second utterance in ~700ms. The cloud round-trip averages ~1.5s and varies with network.
3. **Cost.** Free.

The trade-off is accuracy: Whisper-base is noticeably weaker on accented English and on speakers under stress than Whisper-large or the cloud API. For experimental use this is acceptable because the *condition* effect is what we measure, and Whisper accuracy is constant across conditions.

Why ChromaDB and Not FAISS?

ChromaDB ships with:

- a Python-native, pure-pip install (FAISS requires a C++ build chain);
- automatic persistence to a local SQLite file;
- a stable metadata-filtering API.

For our scale (~50 document chunks from the DIBRIS knowledge base), the speed difference between ChromaDB's HNSW and FAISS's IVF is irrelevant. ChromaDB also fails gracefully — if the SQLite file is locked or the embedding model fails to load, we fall back to a **keyword search** over the same documents (see `RAGPipeline._keyword_search()` in Chapter 12). The pipeline never crashes on a vector-DB outage.

Why xhtml2pdf for the Book?

This is the most surprising choice. We use xhtml2pdf (pure Python) rather than the much higher-quality wkhtmltopdf or LaTeX. Three reasons:

1. **No external binary dependency.** `pip install xhtml2pdf` is the *entire* install. `wkhtmltopdf` needs a separate binary that has been variably available on Windows for the last five years.
2. **Cross-platform reproducibility.** The thesis examiner must be able to rebuild the PDF on their machine in five minutes. A pure-Python pipeline guarantees that.
3. **Font control via DejaVu.** `xhtml2pdf` lets us register the DejaVu Sans + DejaVu Sans Mono fonts (shipped with matplotlib) so that arrows, box-drawing characters, and accented Latin all render correctly. The replacement table in `build_pdf.py` (see Chapter 16's discussion) handles the few Unicode points DejaVu does not cover inside `<pre>` blocks.

The trade-off is rendering quality — xhtml2pdf is not as visually polished as LaTeX. For a thesis-supporting reference book, we judged readability over polish.

Chapter 9 — Two Topologies for Driving Pepper (Polling vs. Bridge Server)

At a Glance

OmniLLM ships two ways to wire the Robot Layer to the AI Layer. **Topology 1** ("Pepper polls AI server") was the original design: Pepper itself initiates every interaction, captures audio, and POSTs to the AI server. **Topology 2** ("AI server drives Pepper") was added in May 2026: a small HTTP bridge server runs in Python 2.7 near (or on) Pepper, listens for action commands from the AI Layer, and translates them to NAOqi calls. The two topologies coexist; use whichever fits your scenario.

Topology 1 — Pepper Polls the AI Server

This is the topology described in the original architecture diagram. The active script is `omnillm/server/naoqi_client.py` (Python 2.7). It runs in a loop:

```
1. Wait for the trigger (text input / head touch / VAD).
2. Capture audio (or read text from stdin).
3. POST audio to AI server /interact.
4. Receive RobotAction JSON.
5. Enact speech / gesture / LED / tablet via NAOqi.
6. Go to step 1.
```

Pros:

- Pepper "owns" its own conversation loop. The participant interacts directly with the robot; the AI server is a passive responder.
- Robust to brief AI-server restarts: Pepper keeps trying.
- Easiest to demo: `python naoqi_client.py --trigger text` and you can start typing immediately.

Cons:

- Action choice is entirely determined by the AI server's response. The LangGraph nodes cannot, mid-thought, decide to *also* trigger a gesture or play a sound while computing — they can only return one final action at the end.
- Streaming responses are awkward: the LLM finishes computing the full sentence before Pepper starts speaking.

Topology 2 — AI Server Drives Pepper via the Bridge Server

This is the topology added in May 2026. The new script is `omnillm/server/naoqi_bridge_server.py` (Python 2.7). It exposes its own HTTP API near the robot:

```
GET /ping -> {"ok": true, "naoqi": true, "simulation": false}
POST /action -> body = RobotAction JSON, executes speech+gesture+LED
POST /audio/record -> body = {"seconds": 5}, returns base64 WAV
GET /sensors -> returns head-touch state, face detected, battery, etc.
POST /tracker/start -> begin face tracking
POST /tracker/stop -> stop face tracking
POST /disconnect -> motion.rest() and disconnect from broker
```

Now the **Python 3 code** is the one that initiates conversation. Inside a LangGraph node you can write:

```
from omnillm.robotics import make_pepper_bridge
from omnillm.robotics.bridge import RobotAction

bridge = await make_pepper_bridge(robot_ip="127.0.0.1", bridge_port=6000)
await bridge.execute_action(RobotAction(
    speech="Hello! Let me think about that.",
    gesture="nod",
    emotion_led="#FFFF00",
))
# ... think ...
await bridge.execute_action(RobotAction(
    speech="The Pepper room is at the end of the corridor.",
    gesture="point_right",
    emotion_led="#00AAFF",
))
```

Pros:

- The AI Layer can trigger arbitrary robot behaviours *during* its reasoning, not just at the end.
- Streaming responses become natural: speak the first sentence while the LLM is still generating the second.
- Test infrastructure: `make_pepper_bridge()` falls back to a **stub mode** that prints actions instead of executing them, so the entire pipeline runs in CI without any robot — virtual or real.

Cons:

- Two Python 2.7 processes to manage (the bridge plus, optionally, a separate client). The PowerShell terminal count goes from 2 to 3.
- The bridge does not own a conversation loop — you must drive it from the Python 3 side.

Which Topology for Which Scenario?

Scenario	Use Topology	
	1	2
Quick demo / "talk to the robot now"	[x]	
Live experimental study with the <code>run_subject_experiment.py</code> driver		[x]
Headless CI tests on a laptop (no robot)		[x] (stub mode)
Debugging a single NAOqi service in isolation		[x] (curl /action)
Continuous voice-activity-detected conversation	[x]	

The **experimental study at DIBRIS uses Topology 2** because the `run_subject_experiment.py` driver is Python-3-based and benefits from the bridge's stub-mode fallback (we can dry-run the whole 20-interaction matrix in CI before plugging in the real robot).

Auto-Discovery: How `make_pepper_bridge()` Decides Which Bridge It Found

The Python 3 client (`omnilm/robotics/pepper.py`) has a single entry point: `make_pepper_bridge(robot_ip="127.0.0.1", robot_port=None, bridge_port=60000)`. Internally:

1. It first tries GET `http://<robot_ip>:<bridge_port>/ping`. If that returns `{"ok": true, "naoqi": true}`, `mode = "server"`.
2. If `/ping` returns `simulation: true`, the bridge is up but is talking to a *virtual* robot — `mode` is still `"server"` but the action-execution path tolerates "behavior not installed."
3. If `/ping` times out, the client attempts a direct NAOqi connection (using the `qi` Python bindings if installed) — `mode = "naoqi"`. This is useful for headless ROS-like integrations.
4. If neither works, `mode = "stub"`. Every `execute_action()` call prints the action to stdout and returns success. This is what makes the pytest suite work without any robot.

This three-level fallback is what we mean when we say "the absence of a robot is a first-class operating mode."

Part III — Comprehensive Script & File Breakdown

Parts I and II were the bird's-eye view. Part III is the file-by-file ground tour. Every Python module that ships in the repository gets a chapter, a purpose statement, a list of its public symbols, an explanation of every non-trivial method, and an answer to the question "why is it written this way?"

If you have only an hour and you want to understand the codebase end-to-end, read Chapter 10. If you have a day, read Chapters 10 through 16.

Chapter 10 — The Master File Table — Every Script, Its Purpose, Its Execution Order

At a Glance

The repository contains roughly **35 Python source files** organised into 7 sub-packages and 1 demo-script directory. The complete enumeration follows. The right-most column gives the **execution order** when you run a full subject session (`run_subject_experiment.py`) — files marked `*` are entry points that the user actually invokes; everything else is library code reached via import.

The Master Table

#	F	S	L	Ir- fr	P	Exec. order
Entry points (you actually run these)						
1	omni-606.py					omni-606.py, rolocom, Optional consensus, evaluator, scmodels, exask, coroute,ker council, evaluate, leaderboard, costs, export
2	omni-406.py					omni-406.py, rag, AI 1, hris, se, bot_ graph, hri, Experiment, util, Experiment_ bridge between Pepper and the AI stack.
3	omni-606.py					omni-606.py, (Python-2.7 HT, Start bridge thatbot) drives Pepper's NAOqi from the AI Layer

4		onself/2007/naoqi_client (P2.7) to url to (P2.7) Three trigger modes: text / touch / vad.
5		script/250/Free_demo/run to 203 session + exp bridge. viaRuns HT only) 5x4 matrix.
6		script/250/Single_shot (P2.7) de optional one question -> AI server -> Pepper speaks. Has -- check-on Ly for NAOqi-only sanity test.

7	script/00_batch_tests.py to all Optional server via conditions HTTP on 4) task types against the AI server. No robot needed.
8	script/00_batch_tests.py routed Optional Condition B actually invokes Llama-via-Ollama Proof of the May 2026 fix.
9	book/00_batch_tests.py referred Optional xhtml2pdf from the master Markdown.
Core LLM modules — Chapter 11	
10	omni-2/llm.py imported single as <code>llm.py</code> door to every LLM provider. Wraps LiteLLM.

17	omni-3200/omni-3200-agent.py langdetect (opt_agent_grap + node langdetect language detection.
18	omni-2800/omni-2800-agent.py combinations (A-a).py participant sequence controlled helpers. model
19	omni-6000/omni-6000-agent.py (opt_agent_grap pydfagent_grap (opt_agent_grap gatewaydes faithfulness scoring + hallucination flag.
Robotics modules — Chapter 13	
20	omni-2500/omni-2500-agent.py abRoadsBridge ABC + by sharedBridge RobotAction / RobotSensorD types.
21	omni-4000/omni-4000-agent.py socket asynclient_grap auto-discovery threoverde falltock (server/direct/s

22		omniutils220 LLM Caller (task_type, response_graph -> LLM nodes (gesture, LED colour)).
23		omniutils209 WavSpeaker (ops, Tty — agent_graph locale or 1 API backend; auto WAV->tensor.
Utilities — Chapter 15		
24		omniutils208 InteractionRecorder json, by dataclasses + node append-only JSON-Lines logger; CSV export.
25		omniutils306 InteractiveNarrative json, by dataclasses / pairwise / observer data models + Questionnaire
26		omniutils180 ReController dataclasses USCLI spcosts

32	test/#270/261 <code>test.py</code> context majority / weighted / synthesis.
33	test/#270/151 <code>test.py</code> evaluate 3 judge patterns.
34	test/#270/231 <code>test.py</code> scores ELO math + leaderboard correctness.
35	test/#270/261 <code>test.py</code> ranges indexing, retrieval, faithfulness, hallucination flag.
36	test/#270/151 <code>test.py</code> classified language detection, experiment manager.
37	test/#270/261 <code>test.py</code> gesture_planner gesture-select rules.
38	test/#270/261 <code>test.py</code> perform stub/direct/ser mode auto-detection

45						receptor Flu Used dependency list pip (install py-rj ect de...encies for non-pip users).
Knowledge base (RAG-indexed)						
46						knowledge_Diffus Info Sub_info / by Sg RAG Pipelin latat description (host setup location, research foci).
47						knowledge_Con Req txt vis RAG Pipelin ques and answers.
48						knowledge_Acc Res research lines RAG Pipelin and key publications.
49						knowledge_Diffus Univ floor map RAG Pipelin room locations; transit directions.
50						knowledge_Up Room s rows labby even RAG Pipelin visitor-relevan dates.

51	knowledge	20	Typical	Real	visitor
		rows	visiting		
			calling	ROCI pipeline	
				(researcher,	
				student,	
				family).	

Execution Order — Live Subject Session

For a complete picture, here is the timeline of which processes start in which order, on which Python interpreter, when running the full study:

```
T+0:00 HUMAN starts experimenter laptop.
T+0:01 HUMAN presses Pepper's chest button.
T+0:25 Pepper says its IP aloud (e.g. "192.168.1.42").
      Note this as PEPPER_IP.
T+0:30 HUMAN opens PowerShell window #1.
      PS> cd C:\Users\akshi\OneDrive\Desktop\OmniLLM
      PS> .\venv\Scripts\Activate.ps1
      PS> python -m omnillm.server.app --host 0.0.0.0 --port 5000
      +--- starts the AI server (Py 3.11)
      Loads LangGraph, RAG (49 chunks),
      Gateway (19 models registered).
      Listens on 0.0.0.0:5000.
T+0:35 HUMAN opens PowerShell window #2.
      PS> C:\Python27\python.exe omnillm\server\ naoqi_bridge_server.py `
      --robot-ip <PEPPER_IP> --robot-port 9559 `
      --bind 0.0.0.0 --bridge-port 6000
      +--- starts the NAOqi bridge (Py 2.7)
      Connects to ALBroker at Pepper.
      Listens on 0.0.0.0:6000.
T+0:40 HUMAN starts PowerShell window #3.
      PS> .\venv\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py `
      --participant P003 --server http://127.0.0.1:5000 `
      --bridge http://127.0.0.1:6000
      +--- driver loop:
      for each (condition, task) in ORDER:
      POST /interact to AI server
      receive RobotAction JSON
      POST /action to bridge
      bridge -> NAOqi -> Pepper
      sleep 2 seconds
      When all 20 done, write CSV+JSON.
T+25:00 Subject session ends. Driver writes
      results/subject_run_P003_<timestamp>.{json,csv}.
T+25:05 HUMAN administers paper questionnaires (Chapter 27).
T+30:00 HUMAN closes all three PowerShell windows.
```

Reading Guide for the Rest of Part III

The remaining chapters of Part III dive into each file group:

If you want to understand...	Read
How any model gets called and how cost is tracked	Chapter 11 (gateway.py)
How the router picks a model	Chapter 11 (router.py)
How the consensus council works	Chapter 11 (consensus.py)

How LLM-as-judge actually scores answers	Chapter 11 (<code>evaluator.py</code>)
How the ELO leaderboard updates	Chapter 11 (<code>scorer.py</code>)
The brain of Pepper — every LangGraph node	Chapter 12 (<code>agent_graph.py</code>)
How "Where is Room 305?" becomes <code>task_type=navigation</code>	Chapter 12 (<code>classifier.py</code>)
How Italian-vs-French is detected	Chapter 12 (<code>language_detector.py</code>)
How conditions A–E map to model IDs	Chapter 12 (<code>experiment.py</code>)
How RAG retrieves + scores faithfulness	Chapter 12 (<code>rag/pipeline.py</code>)
How Python 3 drives Pepper without owning NAOqi	Chapter 13 (<code>robotics/pepper.py</code>)
How <code>point_left</code> is chosen automatically	Chapter 13 (<code>gesture_planner.py</code>)
How audio gets transcribed locally	Chapter 13 (<code>whisper_stt.py</code>)
The HTTP API Pepper speaks to the AI server	Chapter 14 (<code>server/app.py</code>)
The HTTP API the AI server speaks to Pepper	Chapter 14 (<code>naoqi_bridge_server.py</code>)
Three trigger modes (text/touch/vad)	Chapter 14 (<code>naoqi_client.py</code>)
The complete data lifecycle	Chapter 15 (<code>experiment_logger.py</code> , <code>questionnaire.py</code>)
Why Python 2.7 + Python 3.11 sit in one project, and three ways we solve it	Chapter 16

Chapter 11 — Core LLM Modules

Five files, none more than ~450 lines, between them handle every LLM call OmniLLM ever makes. This chapter walks through each one in the order they get imported during a typical interaction.

11.1 omnillm/gateway.py — The Single Door to Every LLM

At a Glance

gateway.py is the smallest, most-used, and most-stable file in the project. Every LLM call — from a omnillm ask CLI invocation to a Condition D consensus inside the LangGraph — funnels through one async method: LLMGateway.query(model_id, messages). The job of that method is to translate "I want to ask model X this question" into a single HTTPS call to whichever cloud or local provider owns model X, and to return a structured ModelResponse that carries the answer plus latency, token usage, and cost.

Component	Lines	Role
ModelResponse dataclass	30	Standardised return type — content + tokens + latency + cost + error
LLMGateway.__init__	15	Reads config/models.yaml; no network I/O
LLMGateway._build_model_string	25	Provider routing prefix (ollama/, gemini/, anthropic/, ...)
LLMGateway._calculate_cost	8	Per-million-token cost from the registry
LLMGateway.query	75	The one async call. ~100ms typical overhead before the cloud round-trip.
LLMGateway.query_multiple	25	asyncio.gather() for concurrent multi-model queries
list_models / list_cloud_models / list_local_models / get_model_info	20	Pure registry accessors

The Walk-Through

(Beginner) Why "Gateway" and not "Client"? A *client* implies one-to-one with a server: "the OpenAI client", "the Anthropic client". A *gateway* is one-to-many: it accepts a uniform call and routes it. Since OmniLLM needs to look identical whether you call OpenAI or Ollama, we use the gateway terminology.

The model registry. When LLMGateway() is constructed it reads config/models.yaml. That file is the *source of truth* for which models exist. The relevant slice looks like this:

```
models:
  openai-gpt4o-mini:
    id: openai-gpt4o-mini
    provider: openai
    model: gpt-4o-mini-2024-07-18
    api_key_env: OPENAI_API_KEY
```

```

cost_per_1m_input: 0.15
cost_per_1m_output: 0.60
type: cloud
description: "OpenAI's fast and cheap default. Good general purpose."
hri_strengths: ["info_retrieval", "social_conversation"]

llama3-8b-local:
  id: llama3-8b-local
  provider: ollama
  model: llama3.2:3b      # name as Ollama knows it
  api_base: http://localhost:11434
  cost_per_1m_input: 0.0
  cost_per_1m_output: 0.0
  type: local
  description: "Llama 3.2 3B running locally via Ollama. Free."
  hri_strengths: ["social_conversation"]

```

After the May 2026 cleanup, the `model:` field of `llama3-8b-local` points at `llama3.2:3b` — the model size that actually exists in the lab's Ollama installation. The historical name `llama3-8b-local` is preserved because Condition B's code references it. (!) **The OmniLLM ID and the upstream model name are not the same thing.** That separation is deliberate: when Ollama drops a model or the OpenAI catalogue renames `gpt-4o-mini`, you only edit `config/models.yaml`, not any Python source.

The provider prefix trick. LiteLLM uses prefixes to route. `_build_model_string` is six lines but it is the entire reason this codebase doesn't have 19 different SDK imports:

```

def _build_model_string(self, model_id: str) -> str:
    cfg = self._models[model_id]
    provider = cfg.get("provider", "openai")
    model = cfg["model"]
    if provider == "ollama":
        return f"ollama/{model}"
    if provider == "openai_compatible":
        return f"openai/{model}"
    if provider == "deepseek":
        return f"openai/{model}"
    if provider == "google":
        return f"gemini/{model}"
    if provider == "anthropic":
        return f"anthropic/{model}"
    return model # openai

```

LiteLLM internally dispatches `ollama/llama3.2:3b` to its `ollama` backend, `gemini/gemini-2.5-flash` to its Gemini backend, and so on. We never have to import `openai`, `anthropic`, `google.generativeai` directly.

The async query method. This is the centre of gravity. The flow:

1. Look up the model in the registry. If absent, return an error `ModelResponse` (do NOT raise — the rest of the pipeline tolerates per-model errors).
2. Build the LiteLLM model string and assemble kwargs (`api_base`, `api_key`, `temperature`, `max_tokens`).
3. Read the API key from `os.environ` based on `api_key_env`. If the env var is missing, LiteLLM will raise downstream — we don't pre-check, because users on Ollama-only setups don't need any cloud keys.
4. Start a `time.perf_counter()`.
5. `await litellm.acompletion(**kwargs)` — the single line that actually goes over the network.
6. Stop the timer; extract tokens; calculate cost; return `ModelResponse`.
7. Any exception is caught and returned as a `ModelResponse` with `error=str(exc)`.

(Beginner) **Why catch all exceptions?** Robustness. The pipeline calls many models sometimes (Condition D council). A transient 429 from Anthropic should not abort the whole interaction. The

downstream nodes check `if resp.is_error:` and decide whether to retry, fall back, or surface the error to the participant.

The `litellm.drop_params = True` at module top. OpenAI's o-series models (o1, o1-mini) reject `temperature` as an argument. So does GPT-5. Without `drop_params=True`, calling those models would crash on the first call. With it, LiteLLM silently drops `temperature` for the providers that don't accept it.

`query_multiple` — parallel calls. Used by Condition D and by `omnilm ask --all`. The implementation is one line: `asyncio.gather(*[self.query(...) for mid in model_ids])`. Three models in parallel run in `max(latency_1, latency_2, latency_3)` wall-clock, not their sum. This matters: a 3-model council can return in 1.5 seconds instead of 4.5.

Academic Context

The "one gateway per language" pattern is consistent with how production AI applications are structured at scale (e.g. *RouteLLM* — Ong et al. 2025). The choice of LiteLLM specifically follows BerriAI's open-source gateway pattern (LiteLLM proxy 2024); we use only the SDK, not the proxy server. The cost accounting code follows the LiteLLM-published per-token prices, which we cache locally in `config/models.yaml` to avoid making a network call just to ask "how much does this token cost?"

11.2 omnillm/router.py — The Smart Model Selector

At a Glance

`router.py` answers the question "given a query and some constraints, which one of the 19 registered models should I send it to?" The `SmartRouter` class implements six strategies (`BEST_QUALITY`, `LOWEST_COST`, `LOWEST_LATENCY`, `BEST_VALUE`, `LOCAL_PREFERRED`, `TASK_TYPE`) and is the mechanism that makes **Condition C** (smart-routed) of the Embodied LLM Arena experiment a real, distinct condition. It also exports a `route_for_hri_task(hri_task_type, ...)` shortcut that the agent graph calls during Condition C handling.

The Six Strategies

Strategy	What it optimises	Used for
<code>BEST_QUALITY</code>	The highest quality score, ignoring cost & latency	When the task is hard reasoning and money/time are no object
<code>LOWEST_COST</code>	The cheapest model that still passes a minimum quality bar	Bulk batch evaluation; cost-sensitive deployments
<code>LOWEST_LATENCY</code>	The fastest model	Live HRI where responses $\leq 2s$ feel natural
<code>BEST_VALUE</code>	Composite score (quality 50%, cost 30%, latency 20%)	The default. A sensible compromise.
<code>LOCAL_PREFERRED</code>	Always prefer Ollama-based models when one passes the quality bar	Offline / privacy-sensitive deployments
<code>TASK_TYPE</code>	Look up <code>routing.hri_task_routing</code> in <code>models.yaml</code> . If matched, use the configured per-task model. Else fall through to <code>BEST_VALUE</code> .	Condition C of the Embodied LLM Arena

The Composite Value Score (BEST_VALUE)

This is the formula that drives the default routing decision. It is also reused inside the gesture-planning logic and the consensus reward function:

```
value = 0.50 * quality
       + 0.30 * cost_score   where cost_score = max(0, 1 - cost / 0.05)
       + 0.20 * latency_score where latency_score = max(0, 1 - (latency - 500) / 9500)
```

In words: - *quality* is the LLM-as-judge score (0–1) for past evaluations of this model on this task category. If no eval data exists, fall back to a static defaults table in `SmartRouter._DEFAULT_SCORES`. - *cost_score* normalises the per-query USD cost: \$0 -> 1.0, \$0.05 -> 0.0. (Most LLM calls are far under

\$0.001, so cheap models almost always score 1.0 here.) - `latency_score` normalises latency: 500 ms -> 1.0, 10,000 ms -> 0.0.

The 50/30/20 weighting is editable (it lives in `_calculate_value_score`). The author tested several alternative weightings during development; this one produces the most stable rankings across the four HRI task types and is the weighting reported in the thesis.

The Self-Updating Quality Table

SmartRouter reads an optional `results_path` JSON file at construction. That file is the output of past `omnillm evaluate` runs, with the structure:

```
[
  {"model_id": "openai-gpt4o-mini", "category": "info_retrieval",
   "score": 0.87, "latency_ms": 1340},
  {"model_id": "claude-haiku", "category": "social_conversation",
   "score": 0.81, "latency_ms": 980},
  ...
]
```

The router rolls these into a per-category, per-model running mean (quality + latency) in `_load_results`. Calling `route(task_category="info_retrieval", strategy=TASK_TYPE)` will then prefer whichever model has the highest running-mean quality on `info_retrieval` queries.

In practice this means: **after one full run of the evaluation suite, the router's decisions become demonstrably better than the static `_DEFAULT_SCORES` defaults.** This is how OmniLLM "learns" — there is no training loop, no gradients; just running averages over past evaluation outcomes.

The `TASK_TYPE` Strategy and `hri_task_routing`

`config/models.yaml` contains a `routing.hri_task_routing` block:

```
routing:
  hri_task_routing:
    info_retrieval: openai-gpt4o-mini
    navigation: openai-gpt4o-mini      # was gemini-flash; now stable choice
    social_conversation: claude-haiku
    multilingual: claude-haiku
  fallback_models:
    - openai-gpt4o-mini
    - claude-haiku
```

When Condition C asks for a `TASK_TYPE` route on `task_category="navigation"`, the router checks `hri_task_routing["navigation"]`, finds `openai-gpt4o-mini`, and returns that model. The lookup is intentionally simple — no LLM-based "meta-routing" — because the experimental design requires the routing decision to be reproducible and human-interpretable.

(!) **The May 2026 fix.** Until 2026-05-19, `hri_task_routing["multilingual"]` pointed at `gemini-flash`. The Google free-tier 429-rate-limited mid-pilot. The fix was a two-line YAML edit (`gemini-flash -> claude-haiku`) and the `multilingual` node in `agent_graph.py` was updated to honour the same change. After the swap, the second pilot run on 2026-05-20 confirmed: Italian queries (the Brignole station prompt) now route cleanly to Claude Haiku and the response time is comparable.

Academic Context

The router's design follows the *strategy pattern* (Gamma et al. 1994). The composite value score is consistent with cost-aware LLM-router formulations in *FrugalGPT* (Chen et al. 2023) and *RouteLLM* (Ong et al. 2025) — both of which similarly weight quality against cost. Our weighting is lighter on cost than FrugalGPT (which emphasises minimising spend); we are running a single-session study where total cost is bounded and quality has higher marginal value to the experimental design.

11.3 omnillm/consensus.py — The LLM Council

At a Glance

consensus.py implements **Condition D** of the experiment: ask three different LLMs the same question in parallel, then merge their answers into a single canonical response. Three merge strategies are supported (majority_vote, weighted, synthesis), of which synthesis is the default and the one used in the actual study.

The Three Synthesis Strategies

Strategy	How it merges	When best
majority_vote	Cluster outputs by semantic similarity (cosine on embeddings ≥ 0.85). Pick the largest cluster's representative.	Closed-form QA where there is a single correct answer.
weighted	Each model's response is weighted by its historical quality score for the task category. Score-weighted text concatenation, then GPT-4o-mini summarises.	When some models are known to be better than others.
synthesis (default)	All N responses are presented to a <i>judge LLM</i> (default: GPT-4o-mini) with the prompt <i>"Here are N answers to the same question. Synthesise the best single answer, drawing on the strongest parts of each."</i> The judge's output is returned.	Open-ended HRI dialogue — where there is no single ground-truth answer but blending viewpoints improves quality.

The Condition D council is configured in omnillm/hri/experiment.py:

```
ExperimentCondition.D: ConditionConfig(
  condition=ExperimentCondition.D,
  model_id=None,
  rag_enabled=True,
  use_consensus=True,
  council_models=["openai-gpt4o-mini", "claude-haiku", "gemini-flash"],
  description="Consensus council – 3 models, best answer synthesised",
),
```

Why a 3-Model Council, Not 5 or 7?

Three reasons:

1. **Latency budget.** Pepper's audible feedback should land ≤ 4 seconds after the participant stops speaking. A 3-model council with `asyncio.gather()` runs in $\max(t_1, t_2, t_3) + \text{synthesis_time} \approx 1.8 + 1.0 = 2.8$ seconds. A 5-model council with one slower outlier easily exceeds 4 seconds.

2. **Cost.** Each interaction in Condition D incurs $\sim 3\times$ the per-query cost of Condition A. Five models would be $\sim 5\times$. Over 20 interactions $\times$ 15 participants the cost difference is non-trivial.
3. **Diminishing returns.** *Kallem (2026)* — a recent multi-model consensus paper — found that beyond 4 models, the marginal accuracy gain falls below the latency penalty. Three is a defensible Pareto point.

Position-Bias Mitigation in Synthesis

The judge LLM is sensitive to **position bias** — the order in which the N responses appear inside its prompt can shift which one it favours. The consensus engine mitigates this by **shuffling** the order of the N responses on each call. For the experimental analysis we additionally re-run a sample of Condition D interactions with swapped orderings and verify the synthesised answer's similarity is ≥ 0.95 across orderings (a sanity check; not a hard gate).

Council Fallback

If one of the three council models returns an error (`resp.is_error`), the engine drops that model from the synthesis and proceeds with the remaining two. If *all three* fail, the engine returns an error response and the agent graph falls back to the per-condition default model (in practice, GPT-4o-mini). This degradation strategy ensures Pepper always speaks *something* — silence is the worst possible interaction outcome from a participant's perspective.

11.4 omnillm/evaluator.py — The LLM-as-Judge Pipeline

At a Glance

`evaluator.py` implements the three best-known LLM-as-judge patterns from the recent literature:

Pattern	Function	Output	Reference
Referenceless (G-Eval)	<code>evaluate_referenceless(score, prompt, response, criteria)</code>	<code>score</code> , 1 + reasoning	Liu et al. 2023
Reference-Based	<code>evaluate_reference_based(score, prompt, response, gold_answer)</code>	<code>score</code> , 1 + reasoning	Standard rubric scoring
Pairwise	<code>evaluate_pairwise(prompt, response_a, response_b)</code>	Winner: A / B / tie	Chatbot-Arena style (Zheng et al. 2023)

The pairwise evaluator includes a **position-bias correction**: it calls the judge twice with the order of A and B swapped, then aggregates. If the two calls agree (both say A or both say B), the result is confident. If they disagree, the result is a tie. This is a published mitigation from Zheng et al. 2023.

Use in the Pipeline

The agent graph's RAG node optionally calls `RAGPipeline._score_faithfulness()`, which internally uses the same judge-LLM pattern (referenceless, with the retrieved chunks as context, scoring how well the answer is grounded in the context). The result populates `rag_faithfulness` in the state and ends up in the logged CSV row.

The CLI's `omnillm evaluate` command runs the full benchmark suite (8 axes × all registered models × all sample prompts), uses the referenceless judge to score every (model, prompt) pair, writes to `results/eval_*.json`, and updates the ELO scorer (see next section).

Why GPT-4o-mini as Judge?

The default judge is `openai-gpt4o-mini` (`evaluator.py: DEFAULT_JUDGE_MODEL`). Three reasons:

1. **Cost.** GPT-4o-mini is one of the cheapest "GPT-4 class" models. The full evaluation suite runs ~500 (model, prompt) pairs. Using `gpt-4o` would 10× the bill.
2. **Speed.** GPT-4o-mini is also fast enough to score in-the-loop (e.g. faithfulness scoring during a live HRI interaction).
3. **Agreement with human ratings.** Liu et al. 2023 and Zheng et al. 2023 both report strong correlation between GPT-4-class judges and human ratings on dialogue evaluation tasks. We do not need a frontier model.

The judge model is configurable via the `judge_model=` kwarg on the evaluator and on the RAG pipeline. For the published thesis runs, every LLM-as-judge call uses GPT-4o-mini.

A Known Limitation: Judge Self-Preference

A documented bias is that an LLM judge sometimes prefers its own family's outputs over those of other models (cf. Zheng et al. 2023 §6.3). Since OmniLLM's default judge is from the OpenAI family, GPT-4o-mini may sub-tly favour `openai-gpt4o-mini` and `openai-gpt4o` over Claude / Gemini / Llama responses in the leaderboard.

Mitigations built into the pipeline:

- The thesis's *primary* condition-level metric is the **human Likert questionnaire**, not the LLM-judge score. The judge feeds the leaderboard and the router's quality table, both of which influence Condition C's routing but do not directly score Conditions A–E for the human study.
- We additionally report **per-condition human pairwise preferences** (the questionnaire's final question), which are unaffected by the judge.
- For replication, the judge model can be swapped via a YAML edit. A Claude-Haiku-judge variant of the leaderboard is included in Appendix G as a robustness check.

11.5 omnillm/scorer.py — The ELO Leaderboard

At a Glance

scorer.py implements a Chatbot-Arena-style ELO rating system. Every pairwise comparison (from the evaluator.evaluate_pairwise function or from the human questionnaire's pairwise preference question) updates the ratings of the two models involved. Categories are supported: there is an "overall" leaderboard but also reasoning, code, embodied_hri, and the four HRI task types.

The ELO Math

Standard ELO. Each model starts at rating 1000. After a pairwise comparison:

```
expected_score_A = 1 / (1 + 10^((rating_B - rating_A) / 400))
new_rating_A     = rating_A + K * (actual_A - expected_score_A)
new_rating_B     = rating_B + K * (actual_B - expected_score_B)
```

where actual_A is 1.0 if A won, 0.5 for a tie, 0.0 if A lost; actual_B = 1 - actual_A. The K-factor is 32 for ratings under 2000 and 16 above. This matches the FIDE chess convention.

A 100-point ELO gap corresponds to a $1 / (1 + 10^{(100/400)}) \approx 0.36$ expected loss rate — i.e. the higher-rated model wins ~64% of the time. We use this 100-point unit as the operational threshold for "meaningfully better."

The Embodied LLM Leaderboard

After the experimental study completes, the human pairwise preferences are POSTed to /evaluate (the AI server's questionnaire endpoint). They are fed into the scorer under category embodied_hri. The resulting leaderboard is the **central scientific contribution of the thesis** — it is the first ELO leaderboard for LLMs ranked by their performance as social-robot brains.

Outputting the Leaderboard

```
omnillm leaderboard           # the overall ELO
omnillm leaderboard --category embodied_hri # the thesis result
omnillm leaderboard --category multilingual  # T4-only
omnillm export --format csv --input results/leaderboard_embodied_hri.json
```

11.6 omnillm/cli.py — Your Terminal Dashboard

At a Glance

cli.py is the omnillm console-script entry point registered in pyproject.toml:

```
[project.scripts]
omnillm = "omnillm.cli:main"
```

It is built on `click` (for argument parsing) and `rich` (for coloured terminal tables). The available commands are:

Command	What it does
<code>omnillm models</code>	List all registered models in a coloured table (cloud + local)
<code>omnillm ask "..."</code>	Single-model query. <code>-m model_id</code> repeatable. <code>--all</code> queries every model.
<code>omnillm route "..."</code>	Show the router's decision and reasoning for a prompt
<code>omnillm council "..."</code>	Run a 3-model consensus and print the synthesised answer
<code>omnillm evaluate</code>	Run the full benchmark suite (8 axes × all models). Output JSON.
<code>omnillm compare "... --model-a ... --model-b ...</code>	Pairwise comparison with position-bias swap
<code>omnillm leaderboard [--category ...]</code>	Print the ELO leaderboard
<code>omnillm costs</code>	Print per-model USD spend across all logged interactions
<code>omnillm export --format csv --input ... --output ...</code>	Convert results JSON to CSV / Markdown

The CLI is the fastest way to exercise any single subsystem in isolation, *without* booting the Flask server or the robot. During development, the author's typical workflow is:

```
# 1. Edit a router strategy in router.py
# 2. Re-run a sanity test:
omnillm route "Where is Room 305?" --strategy TASK_TYPE
# 3. Result printed in 1.2 seconds. Iterate.
```

This shortens the inner-loop iteration time from ~30 seconds (rebooting the Flask server) to ~1.5 seconds.

Chapter 12 — HRI & RAG Modules

The five files in this chapter are the ones that transform OmniLLM from "a benchmarking framework" into "Pepper's brain." Every file is small (under 700 lines), every file does one thing, and they compose into the 9-node LangGraph pipeline that drives every interaction.

12.1 omnillm/hri/agent_graph.py — The Brain of Pepper

At a Glance

`agent_graph.py` is the **single most important file in the project**. The `build_hri_graph(gateway, rag, logger, default_model)` function returns a compiled `LangGraph CompiledGraph` that accepts a state dict, runs it through 9 nodes, and returns the updated dict containing the `robot_action` JSON. Every interaction with Pepper — voice, touch, or typed — flows through exactly one invocation of `graph.ainvoke(state)`.

The graph has three layers of complexity:

1. **The node functions** (one async function per node, ~30 lines each). These are the *what happens at each step*.
2. **The state wrapping** — a `_merge_state(fn)` closure that wraps every node so its return value *merges* with the existing state instead of *replacing* it. This is a single line of code that fixes a class of bugs we will discuss below.
3. **The graph wiring** — `builder.add_node()` and `builder.add_edge()` calls that connect the nodes, plus one `builder.add_conditional_edges()` that branches on task type.

The Nine Nodes

#	Node	What it computes	Output fields added to state
1	<code>transcribe_audio</code>	If <code>audio_bytes</code> is non-empty, calls <code>WhisperSTT.transcribe()</code> . Otherwise pass-through (text mode). Sets <code>_start_time</code> for end-to-end latency measurement.	<code>utterance</code> , <code>_start_time</code>
2	<code>detect_language</code>	Calls <code>LanguageDetector.detect(utterance)</code> . Returns the ISO 639-1 code.	<code>detected_language</code>
3	<code>classify_task</code>	Calls <code>HRITaskClassifier.classify_task_confidence(utterance)</code> . Rule-based by default.	<code>task_type</code> , <code>task_confidence</code>
4	(conditional edge)	Branches on (<code>task_type</code> , <code>condition</code> , <code>rag_enabled</code>). T1-> <code>rag</code> , T2-> <code>nav_rag</code> , T3-> <code>direct_llm</code> , T4-> <code>multilingual_llm</code> . Cond E always -> <code>direct_llm</code> .	—
5a	<code>rag (T1)</code>	<code>RAGPipeline.query(utterance, rag_context, model_id=...)</code> . Retrieves k=4 chunks; generates answer with <code>model_id</code> .	<code>rag_chunks</code> , <code>rag_faithfulness</code> , <code>response_text</code> , <code>model_id</code> , <code>latency_ms</code>
5b	<code>nav_rag (T2)</code>	Same as <code>rag</code> + <code>GesturePlanner.plan(naive_color, answer)</code> .	All of <code>rag</code> + <code>gesture</code> ,

5c	<code>direct_llm</code> (T3)	Direct gateway call with social system prompt. Adds gesture via planner.	<code>response_text</code> , <code>model_id</code> , <code>tokens</code> , <code>cost</code> , <code>latency</code> , <code>gesture</code> , <code>led_color</code>
5d	<code>multilingual_llm</code> (T4)	Tries language-optimal model (Claude Haiku for non-English) first; falls back to GPT-4o-mini on error.	Same as <code>direct_llm</code>
6	<code>smart_router</code>	Cond C: re-routes the LLM call via <code>SmartRouter.route_for_hmodel_id()</code> . Cond D: fires the 3-model consensus. Other conditions: pass-through.	Possibly overwrites <code>response_text</code> .
7	<code>generate_action_plan</code>	Assembles the final <code>RobotAction</code> JSON. If no gesture set yet, calls the planner.	<code>robot_action</code> , <code>gesture</code> , <code>led_color</code>
8	<code>log_interaction</code>	Computes end-to-end latency from <code>_start_time</code> . Calls <code>ExperimentLogger.log_interaction()</code> .	<code>latency_ms</code> (end-to-end)

The `_merge_state` Wrapper — A 5-Line Fix to a Whole Class of Bugs

LangGraph 1.x's `StateGraph(dict)` has an important behaviour: **each node's return value REPLACES the state, it does not merge**. So if node 1 sets `state["utterance"] = "Hi"` and node 2 returns `{"detected_language": "en"}`, after node 2 the state contains *only* `{"detected_language": "en"}` — `utterance` is gone.

This caused the empty-speech bug that first surfaced during the May 2026 pilot's first P000 run. The symptom: Pepper's `robot_action.speech` field was empty. The cause: the LLM nodes were returning `{"response_text": "..."}` which replaced (not merged into) the state, and the downstream `generate_action_plan` node looked for `response_text` but found only the LLM node's narrow output.

The fix is six lines:

```
def _merge_state(fn):
    async def wrapped(state):
        delta = await fn(state)
        if not isinstance(delta, dict):
            return state
        return {**state, **delta}
    return wrapped
```

Every node is registered via `builder.add_node("name", _merge_state(node_fn))`. The wrapper takes the node's return value and merges it into the existing state, so prior fields survive. This is the kind of fix

that takes 30 seconds to write and four hours to find. **(Beginner) lesson:** when the framework you depend on has subtle semantics, *write a thin adapter once* rather than fighting the semantics at every node.

The Conditional Edge — Where the Four Task Types Diverge

```
def _route_by_task_type(state):
    task_type = state.get("task_type", "info_retrieval")
    condition = state.get("condition", "A")
    rag_enabled = state.get("rag_enabled", True)
    if condition == "E":
        # RAG-off control always goes direct
        return "direct_llm"
    routing = {
        "info_retrieval": "rag" if rag_enabled else "direct_llm",
        "navigation": "nav_rag" if rag_enabled else "direct_llm",
        "social_conversation": "direct_llm",
        "multilingual": "multilingual_llm",
    }
    return routing.get(task_type, "direct_llm")
```

Two important details:

1. **Condition E short-circuits.** Even if the classifier says "this is an info_retrieval question that would benefit from RAG", Condition E refuses to call the RAG pipeline and goes straight to the LLM. This is what makes E the clean RAG-off control for hypothesis H3.
2. **The rag_enabled flag respected.** If the caller passes rag_enabled=False and the task is T1 or T2, the graph falls back to direct LLM. This is mostly used for testing: it lets you run the pipeline with no knowledge base indexed.

How Condition Resolution Happens *Before* the Graph

This is the most subtle fix in the May 2026 upgrade. Look at `omnillm/server/app.py:255–278`:

```
# Resolve the per-condition model BEFORE invoking the graph so
# that Condition B (fixed local Llama) actually exercises the
# local Ollama backend rather than silently falling through to
# the server's default cloud model.
from omnillm.hri.experiment import CONDITION_CONFIGS, ExperimentCondition
try:
    cond_enum = ExperimentCondition(condition)
    cond_cfg = CONDITION_CONFIGS.get(cond_enum)
    effective_model = (
        cond_cfg.model_id if cond_cfg and cond_cfg.model_id
        else _default_model
    )
except ValueError:
    effective_model = _default_model

state["model_id"] = effective_model
```

The server *reads* the requested condition from the POST body, looks up the canonical model ID for that condition in `experiment.py`, and *injects* it into the state dict before calling `graph.ainvoke(state)`. The RAG and direct-LLM nodes then honour the `model_id` field they find in the state, passing it through to `gateway.query()`.

Before this fix, Condition B silently ran Condition A's model (GPT-4o-mini) because the RAG pipeline used its constructor-time `self.model_id`, which was always the server's default. The fix routes the per-condition model id explicitly through the state. Condition B now actually runs Llama via Ollama; the leaderboard rankings are no longer artefacts of all conditions using GPT-4o-mini under the hood.

(!) If you ever add a new condition, you must update both `experiment.py`: `CONDITION_CONFIGS` AND make sure your new condition's `model_id` is honoured by the relevant LLM node. The `smart_router` node (`_make_smart_router_node`) is where the routing happens for Conditions C and D; the per-condition-model handling for A, B, E happens in the upstream `rag`, `nav_rag`, `direct_llm`, and `multilingual_llm` nodes via their `model_id=state.get("model_id")` or `default` lookup.

The Multilingual Retry Logic

The T4 (multilingual) node has a built-in retry. Look at `_make_multilingual_llm_node`:

```
for model in (target_model, backup_model):
    if model in models_tried:
        continue
    models_tried.append(model)
    try:
        resp = await gateway.query(model, messages, temperature=0.7)
    except Exception as exc:
        last_error = f"{model}: {exc}"
        resp = None
        continue
    if not resp.is_error:
        break
    last_error = f"{model}: {resp.error}"
    resp = None
```

If the primary multilingual model (Claude Haiku) returns an error, the node retries with `openai-gpt4o-mini` while *preserving the multilingual system prompt*. This is important: the previous fallback path (in `app.py`'s `_fallback_interact` function) would silently switch to the English-only generic system prompt, which produced English replies to French input. The new retry preserves the T4 treatment so Condition C's multilingual handling remains a real T4 condition even on transient provider failure.

12.2 omnillm/hri/classifier.py — The Task Classifier (T1–T4)

At a Glance

`HRTaskClassifier.classify(utterance, detected_language)` returns a `ClassificationResult(task_type, confidence, reasoning, detected_language, method)`. By default it uses a rule-based scorer (~30 ms, no LLM call). An optional `classify_with_llm()` method delegates to GPT-4o-mini for ambiguous cases (~700 ms).

Why Rule-Based by Default?

Three reasons:

1. **Latency.** A rule-based classifier returns in milliseconds; an LLM classifier adds ~700ms to every interaction. In an HRI pipeline that wants ≤ 4 s end-to-end, every 700ms counts.
2. **Cost.** Zero per-query cost. Over $20 \times 15 = 300$ interactions in the study, an LLM classifier would cost an extra ~\$0.50 — small in absolute terms, but unnecessary.
3. **Predictability.** The rule-based scoring is deterministic, easy to test, and easy to debug. When the pilot saw a misclassification, the author could inspect the keyword-and-pattern table directly. An LLM classifier would have been a black box.

How the Scoring Works

For each task type, the classifier maintains a frozenset of single-word keywords and a list of compiled regex patterns. For an utterance like *"Where is Room 305?"*:

```
words = {"where", "is", "room", "305"}
nav_score = score("navigation", words, _NAVIGATION_KEYWORDS, _NAVIGATION_PATTERNS)
social_score = score("social", words, _SOCIAL_KEYWORDS, _SOCIAL_PATTERNS)
info_score = score("info", words, _INFO_RETRIEVAL_KEYWORDS, _INFO_RETRIEVAL_PATTERNS)
```

The `_score_category()` function adds:

- $0.1 \times \text{num_keyword_hits}$ (capped at 0.3)
- $0.15 \times \text{num_multi-word-phrase_hits}$
- $0.25 \times \text{num_regex_pattern_matches}$

For *"Where is Room 305?"*: - `nav_score`: hits where, room, plus the pattern `\b(where (is|are)|...)\b` -> $0.2 + 0.25 = 0.45$ - `social_score`: 0 - `info_score`: tiny match on what is the pattern -> 0.05

`max(scores)` picks navigation with confidence $\min(0.95, 0.5 + 0.45 * 0.9) \approx 0.91$.

Multilingual Always Wins

If `detected_language != "en"`, the classifier returns `MULTILINGUAL` with confidence 0.99 *immediately* — before any keyword scoring. This is the right behaviour because the rule keywords are English-only; running them on Italian or Chinese would produce nonsense scores. The downstream graph routing handles multilingual queries specifically (it has access to the language code and can pick a multilingual-strong LLM).

Why No Fine-Tuned Classifier?

The author considered training a small classifier (e.g. DistilBERT fine-tuned on labelled HRI utterances). Decided against it because:

1. **No labelled data.** We do not have a labelled corpus of "T1 / T2 / T3 / T4" HRI utterances at the scale required for fine-tuning.
2. **Distribution shift.** Even if we labelled 1,000 utterances from one lab, a different lab's prompts would distribute differently.
3. **The rule-based version is good enough.** The May 2026 pilot showed 100% correct classification on the 20 test prompts. The cost of training and maintaining a fine-tuned model is not justified by an improvement we cannot measure.

If a future deployment hits real ambiguity, swapping to `classify_with_llm` is a one-line change inside the agent graph's `_make_classify_task_node`.

12.3 omnillm/hri/language_detector.py — How French Becomes "fr"

At a Glance

`LanguageDetector.detect(text)` returns a `LanguageDetectionResult(language, confidence, script, is_english, language_code, language_name, recommended_model)`. Detection is **rule-based first, library-based second** — this avoids a hard dependency on `langdetect` (which is heavier).

Three-Tier Detection

1. **Unicode script analysis** — fastest, works on zero-token inputs. "■■■■■■" contains Hiragana -> ja. "■■■■■■" contains Arabic script -> ar. "■■■■■■■■" contains Cyrillic -> ru. Confidence: 0.85–0.97.
2. **Word-level n-gram signals** — for Latin-script languages where script is ambiguous. The detector has hand-picked high-frequency function-word lists for FR, DE, ES, IT, PT, NL. "*Bonjour, comment allez-vous?*" hits `bonjour`, `vous` -> fr. Confidence: 0.6–0.9.
3. **langdetect fallback** — only invoked when the first two tiers fail to find anything stronger than English. Heavy but accurate. Ships as an optional `[hri]` extra.

Why Word-Boundary Matching, Not Substring Matching

A subtle but important bug fixed in the May 2026 upgrade. The original code did `if signal_word in text.lower(): score += 1`. That matches **anywhere** inside the text, so the Spanish function word `la` matches `lab`, `de` matches `does`, `en` matches `open`. The fix uses **tokenised word-boundary matching**: `words = set(re.findall(r"\b\w+\b", text.lower()))` and `signal_word in words`. A two-letter function word now only counts as a hit if it appears as a standalone token.

This is the kind of bug that shipped silently for months — it didn't crash anything; it just routed "what time does the lab open?" to Spanish because the substring match found `la`, `de`, `en` and counted them as Spanish function words. Caught only when the pilot's T1 prompts started occasionally getting Spanish answers.

The Language -> Model Map

```
_LANGUAGE_MODEL_MAP = {
    "en": "openai-gpt4o-mini",
    "fr": "claude-haiku",
    "de": "claude-haiku",
    ...
    "unknown": "claude-haiku",
}
```

As discussed: Claude Haiku is the default multilingual choice as of 2026-05-20 because it has wide language coverage *and* is not subject to the Google free-tier quota that knocked Gemini Flash offline mid-pilot. The map can be overridden per-instance:

```
detector = LanguageDetector(custom_model_map={"it": "openai-gpt4o"})
```

The Compatibility Properties

The class has three deliberately redundant properties:

```
@property
def language_code(self) -> str: return self.language
@property
def language_name(self) -> str: return _LANGUAGE_NAME_MAP[self.language]
@property
def recommended_model(self) -> str: return _LANGUAGE_MODEL_MAP[self.language]
```

These were added in the May 2026 upgrade because the multilingual node in `agent_graph.py` was calling `lang_result.recommended_model` and `lang_result.language_name`. Before adding the properties, those attribute accesses raised `AttributeError`, the multilingual node fell into its catch-all except, and Pepper produced empty speech. Five trivial properties later, T4 works.

12.4 omnillm/hri/experiment.py — Conditions A–E, Participant Sessions

At a Glance

experiment.py is short (~280 lines) but central. It defines:

Construct	Purpose
ExperimentCondition (enum)	The five conditions A–E
ConditionConfig (dataclass)	Per-condition: model_id, RAG on/off, consensus flag, smart_routing flag, council_models list, human-readable description
CONDITION_CONFIGS (dict)	The mapping ExperimentCondition -> ConditionConfig. This is the source of truth for how each condition is realised.
ParticipantSession (dataclass)	A single participant's session: participant_id, session_id (UUID), conditions_assigned, start_time, end_time, observation_notes
ExperimentManager	Constructor and lifecycle management for sessions. create_session(participant_id, conditions=None) returns a new ParticipantSession.

Why a Dataclass-Centric Design?

(Beginner) A **dataclass** is a Python class auto-generated by the @dataclass decorator that gives you `__init__`, `__repr__`, `__eq__`, and field defaults for free. Use them for "named tuples that need methods." The author uses dataclasses everywhere in OmniLLM (ModelResponse, RobotAction, InteractionRecord, ConditionConfig, etc.) because the alternative — hand-written classes — would be 5× more code with no benefit.

How Conditions Map to Code Paths

```
CONDITION_CONFIGS = {
    ExperimentCondition.A: ConditionConfig(
        condition=A,
        model_id="openai-gpt4o-mini",
        rag_enabled=True,
        description="Fixed cloud LLM (GPT-4o-mini) with RAG – baseline"),

    ExperimentCondition.B: ConditionConfig(
        condition=B,
        model_id="llama3-8b-local",
        rag_enabled=True,
        description="Fixed local LLM (Llama3:8b via Ollama) with RAG"),

    ExperimentCondition.C: ConditionConfig(
        condition=C,
        model_id=None, # routed
        rag_enabled=True,
        use_smart_routing=True,
        description="Smart-routed – OmniLLM selects best model per task type"),

    ExperimentCondition.D: ConditionConfig(
        condition=D,
        model_id=None, # counselled
        rag_enabled=True,
        use_consensus=True,
        council_models=["openai-gpt4o-mini", "claude-haiku", "gemini-flash"],
        description="Consensus council – 3 models, best answer synthesised"),

    ExperimentCondition.E: ConditionConfig(
        condition=E,
        model_id="openai-gpt4o-mini",
        rag_enabled=False, # the only RAG-off
        description="RAG-off control – GPT-4o-mini without knowledge retrieval"),
}
```

The agent graph's smart-router node reads `cond_cfg.use_consensus` and `cond_cfg.use_smart_routing` to decide whether to fire the consensus engine or call the smart router. Conditions A, B, E pass through unchanged (`use_consensus=False`, `use_smart_routing=False`).

Counterbalancing — Where the Order Lives

The `run_subject_experiment.py` driver hard-codes a Latin-square-ish ordering at the top of the file:

```
ORDER = [
    ("A", "T1"), ("A", "T2"), ("A", "T3"), ("A", "T4"),
    ("B", "T2"), ("B", "T3"), ("B", "T4"), ("B", "T1"),
    ("C", "T3"), ("C", "T4"), ("C", "T1"), ("C", "T2"),
    ("D", "T4"), ("D", "T1"), ("D", "T2"), ("D", "T3"),
    ("E", "T1"), ("E", "T2"), ("E", "T3"), ("E", "T4"),
]
```

Each condition starts at a different task — so task-order effects are partly cancelled within the participant.

For the full N=15 study, we additionally need to **rotate the condition order across participants** so that not every participant sees A first. The recommended assignment is a **5x5 Latin square over the condition order**:

Participant	Cond 1	Cond 2	Cond 3	Cond 4	Cond 5
P001	A	B	C	D	E

P002	B	C	D	E	A
P003	C	D	E	A	B
P004	D	E	A	B	C
P005	E	A	B	C	D
P006	A	C	E	B	D
P007	B	D	A	C	E
P008	C	E	B	D	A
P009	D	A	C	E	B
P010	E	B	D	A	C
P011	A	D	B	E	C
P012	B	E	C	A	D
P013	C	A	D	B	E
P014	D	B	E	C	A
P015	E	C	A	D	B

Within each condition the task order is the Latin-square-ish ordering above. The two squares together produce a balanced design: each condition appears in each of positions 1–5 exactly 3 times across the 15 participants. For replicating, see Chapter 25 for the detailed counterbalancing argument.

12.5 omnillm/rag/pipeline.py — Retrieval-Augmented Generation

At a Glance

RAGPipeline does five things:

1. Indexes documents (TXT / CSV / PDF) from a directory into ChromaDB.
2. Retrieves top-k chunks for a query using ChromaDB's cosine similarity over sentence-transformer embeddings.
3. Falls back to a pure-Python keyword search if ChromaDB is unavailable (no graceful degradation gap).
4. Generates an answer via the LLM gateway with the retrieved chunks injected as context.
5. (Optional) Scores faithfulness with LLM-as-judge and runs a lightweight hallucination heuristic.

The Indexing Pipeline

`rag.index_directory("knowledge_base/")` walks the directory and dispatches per file extension:

- `.txt` -> split into ~512-character chunks with 64-character overlap (`_split_text`), embed, index.
- `.csv` -> one chunk per row, with `key: value | key: value | ...` formatting.
- `.pdf` -> uses `pypdf` to extract text per page, then chunked like `txt`.

The chunk-size choice (512 chars, ~125 tokens) was based on three considerations:

1. **Embedding model context window.** Sentence-transformer `all-MiniLM-L6-v2` accepts up to 256 word-pieces; 512 chars is roughly 100–130 tokens — safely under the limit.
2. **Retrieval granularity.** Larger chunks (~2000 chars) retrieve more context but dilute the relevance score; smaller chunks (~100 chars) are too granular to be useful as standalone facts.
3. **Pepper's spoken-response window.** A response should be 2–4 sentences (~250 chars). Retrieved chunks of 512 chars give the LLM enough room to compose a faithful answer without padding.

The Retrieval Step

```
def retrieve(self, query: str) -> list[DocumentChunk]:
    if self._collection is not None:
        return self._retrieve_chromadb(query)
    return self._retrieve_keyword(query)
```

If ChromaDB is alive, `_retrieve_chromadb` runs a semantic similarity search and returns up to `top_k=4` chunks ranked by cosine similarity. Each chunk carries `similarity_score` in `[0, 1]`.

If ChromaDB is dead (the SQLite file is locked, the embedding model failed to load), `_retrieve_keyword` scores each in-memory chunk by overlap with the query's tokens. This is the **graceful degradation** path: faithfulness drops, but the pipeline doesn't crash.

The Per-Call `model_id` Override

The May 2026 upgrade added a `model_id=` parameter to `query()`:

```

async def query(
    self,
    question: str,
    system_prompt: str | None = None,
    score_faithfulness: bool = False,
    model_id: str | None = None,
) -> RAGResponse:
    ...
    effective_model = model_id or self.model_id
    llm_response = await self.gateway.query(effective_model, messages)

```

Without this, the RAG pipeline always used `self.model_id` (the model passed at construction — typically GPT-4o-mini). The agent graph's RAG node now passes `model_id=state.get("model_id")`, which carries the per-condition model. This is what makes Condition B's "Llama via Ollama with RAG" actually run Llama via Ollama at the answer-generation step.

Faithfulness Scoring

When `score_faithfulness=True`, the pipeline asks the judge LLM:

"You are a factuality judge. Score how faithfully the answer uses ONLY information from the given context (0.0 = completely hallucinated, 1.0 = every claim is supported by the context)."

Context: [retrieved chunks]

Question: ...

Answer: ...

Respond ONLY with valid JSON: {"score": , "reasoning": ""}

The score is a continuous 0–1. We additionally run a cheap heuristic: if fewer than 20% of the answer's significant words (≥ 5 chars) appear in the retrieved chunks, set `hallucination_detected=True`. The heuristic occasionally false-positives on paraphrased answers and false-negatives on plausible-sounding fabrications, so the LLM-as-judge `faithfulness_score` is the canonical metric.

The DIBRIS Knowledge Base — Current Contents (May 2026)

File	Purpose	Indexed chunks
lab_info.txt	DIBRIS lab description, hours, location, contact	~12
faq.txt	Common visitor questions ("Where is the bathroom?", "What is the wifi password?", etc.)	~15
research_projects.txt	Active research lines and recent papers	~10
university_map.txt	DIBRIS floor map, room locations, transit directions	~8
event_schedule.csv	Upcoming lab events	~3 rows = 3 chunks
visitor_profiles.csv	Typical visitor categories	~3 rows = 3 chunks
Total	—	~49 chunks

This is small for a ChromaDB collection (production RAG systems typically index 10k–100k chunks). The trade-off is intentional: a small, hand-curated knowledge base means the RAG faithfulness experiments measure *retrieval quality on a known corpus*, not "scaling RAG to a corporate document set." The methodology is transferable; the corpus size is bounded by what we can verify by hand.

The previous IRAI-Lab content (used in the first edition of the book) is preserved under `knowledge_base/_legacy_irai/` for reference. The two corpora can be swapped by setting `OMNILLM_KNOWLEDGE_BASE` to the desired directory:

```
$env:OMNILLM_KNOWLEDGE_BASE = "C:\Users\akshi\OneDrive\Desktop\OmniLLM\knowledge_base\_legacy_irai"
python -m omnillm.server.app
```

Academic Context

The RAG architecture follows the canonical "retrieve-then-generate" pattern (Lewis et al. 2020, Karpukhin et al. 2020). The chunk-size and overlap choices are conservative defaults from LangChain's `RecursiveCharacterTextSplitter`. The faithfulness scoring is closest in spirit to RAGAS (Es et al. 2023) — same judge-LLM-with-context pattern, simpler implementation. The hallucination heuristic is intentionally crude; production systems use stronger entailment models (e.g. ALBERT-based natural-language-inference), which we considered out of scope for a deployment of this size.

Chapter 13 — Robotics Modules

Four files between them are everything that talks to (or pretends to talk to) Pepper. They are the cleanest layer in the codebase — abstract base class + concrete Pepper implementation + two small utilities — because the hard work has been delegated to the bridge server in Chapter 14.

13.1 omnillm/robotics/bridge.py — The Abstract Robot Bridge

At a Glance

bridge.py declares the **contract** that any robot integration must obey. It defines two data types (RobotAction, RobotSensorData) and one abstract base class (RobotBridge with six abstract async methods). The point is to keep the rest of the codebase **robot-agnostic** — any future humanoid (Nao, Tiago, iCub, a Unity-rendered avatar) can drop in by subclassing RobotBridge.

Symbol	Lines	Purpose
RobotAction dataclass	20	What the AI Layer wants the robot to do: speech, gesture, movement, emotion_led, nav2_goal, metadata
RobotSensorData dataclass	8	What the robot tells us back: touch_sensors, face_detected, speech_detected, battery_level
RobotBridge (ABC)	50	Six abstract methods: connect, disconnect, execute_action, get_sensor_data, say, gesture
parse_llm_to_action	40	Helper: parse an LLM's JSON output into a RobotAction. Strips markdown code fences.
parse_nav2_goal	50	Helper: validate a Nav2-style {position, orientation} JSON. Forward-looking for future ROS2 integration.

The Contract

Every method on RobotBridge is async. This matters because LangGraph's nodes are async — so the bridge can be `await`-ed mid-pipeline without blocking. If we'd written sync methods, every gesture would freeze the entire async event loop for the duration of the NAOqi RPC.

The six abstract methods:

```
@abstractmethod
async def connect(self) -> bool: ...

@abstractmethod
async def disconnect(self) -> bool: ...

@abstractmethod
async def execute_action(self, action: RobotAction) -> bool: ...

@abstractmethod
```

```

async def get_sensor_data(self) -> RobotSensorData: ...

@abstractmethod
async def say(self, text: str) -> bool: ...

@abstractmethod
async def gesture(self, name: str) -> bool: ...

```

The first four are mandatory for any robot. `say()` and `gesture()` are convenience wrappers (every implementation will do something like `return await self.execute_action(RobotAction(speech=text))`) but having them in the ABC enforces the convention.

The Safety Note in the Source File

The module docstring contains this warning:

"In robotics applications, incorrect LLM outputs can cause physical harm. Always validate RobotAction objects through the consensus engine before executing on a real robot — a single-model hallucination should not be enough to trigger robot movement."

For Pepper specifically, the safety surface is small (Pepper has no grippers, cannot lift objects, cannot manipulate humans), but the principle generalises. Future deployments that wire OmniLLM to a manipulator or a mobile platform should consider running the LLM output through `consensus.synthesise()` before sending it to `execute_action()`.

13.2 omnillm/robotics/pepper.py — The Python 3 Side of Pepper

At a Glance

PepperBridge is a concrete RobotBridge for Pepper. It speaks aiohttp HTTP to the Python 2.7 `naoqi_bridge_server` (which we will discuss in Chapter 14). All NAOqi-specific work happens on the Python-2.7 side; this file is pure Python-3 and pure HTTP.

The class supports three modes (`server`, `direct`, `stub`) with automatic detection. The `make_pepper_bridge()` factory function does the right thing for both real Pepper and Choregraphe's virtual robot.

The Three Modes

Mode	Trigger	What <code>execute_action()</code> does
<code>server</code>	GET <code>/ping</code> to the bridge port returns <code>{"ok": true, "naoqi": true}</code>	POST <code>/action</code> to the bridge server, which translates to NAOqi calls
<code>direct</code>	Set manually via <code>bridge.use_direct_mode()</code> . Used only by unit tests.	Prints the action to stdout, returns True
<code>stub</code>	Fallback when nothing else works	Logs to stdout via <code>logger.info()</code> , returns True

The auto-detection logic is in `PepperBridge.connect()`:

```
async def connect(self) -> bool:
    ping = await self._ping_bridge_server()
    if ping is not None:
        self._mode = "server"
        self._connected = True
        return True
    if self._fallback_policy == "raise":
        raise ConnectionError(...)
    self._mode = "stub"
    self._connected = True
    return True
```

In *auto* mode, `connect()` *always* succeeds — falling to `stub` if no real bridge is reachable. This is why every `pytest` can do `bridge = await make_pepper_bridge()` without owning a robot.

Choregraphe Port Auto-Discovery

Choregraphe's virtual robot picks a random port between 49152 and 65535 every launch. The `discover_choregraphe_port` function does a two-pass scan:

1. **Hint pass.** Try a small list of ports known from past launches: (49959, 49960, 49961, 60930, 62494, 9559). These are the values the author's laptop has seen across all Choregraphe sessions; the most recent value is first.

2. **Slice scan.** If no hint matches, sample `max_scan=256` ports uniformly across the ephemeral range. The total time bound is $256 * \text{timeout} = 256 * 0.05\text{s} \approx 13\text{s}$ in the worst case, typically $<1\text{s}$ when the port is found in the first dozen tries.

The scan is **TCP-reachability only** — it does not validate that the peer actually speaks NAOqi. The follow-up `connect()` call to the bridge server is what catches false positives.

The `_tcp_reachable` Helper

```
def _tcp_reachable(host: str, port: int, timeout: float = 0.5) -> bool:
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.settimeout(timeout)
            sock.connect((host, port))
            return True
    except (OSError, socket.timeout):
        return False
```

A pre-check to avoid paying aiohttp setup cost when the port isn't even open. Tiny optimisation, but with a 256-port scan it cuts the total scan time from 30+ seconds (full aiohttp setup per port) to <1 second.

`make_pepper_bridge()` — The Public Entry Point

This is what almost everyone calls. Its contract:

```
async def make_pepper_bridge(
    robot_ip: str = "127.0.0.1",
    robot_port: int | None = None,
    bridge_port: int = 6000,
    fallback: Literal["auto", "stub", "raise"] = "auto",
) -> PepperBridge:
```

The behaviour:

- `robot_ip="127.0.0.1", robot_port=None` -> Choregraphe virtual robot; run port discovery; fall back to 9559 if discovery fails.
- `robot_ip=<other>, robot_port=None` -> assume real Pepper on default NAOqi port 9559.
- `bridge_port=6000` -> standard bridge port; configurable if you run multiple Pepper instances or have a port collision.
- Always calls `.connect()` before returning, so the caller can immediately use the bridge.

Why aiohttp and Not requests?

(Beginner) `requests` is the famous synchronous Python HTTP library. `aiohttp` is the async equivalent. We use aiohttp because every method in `PepperBridge` is `async`. If we used `requests` inside an `async` method, the call would block the event loop — every other `async` task (other LLM calls, sensor polling) would pause until the HTTP call returns. That defeats the entire point of `asyncio`.

13.3 omnillm/robotics/gesture_planner.py — Speech-to-Gesture Mapping

At a Glance

`GesturePlanner.plan(task_type, response_text)` returns `(gesture_name, led_color)` based on:

1. The HRI task type (gives a default).
2. Keyword/regex matches in the response text (overrides the default).

It is a small, deterministic, rule-based mapping that runs in milliseconds. The thesis study uses it for every interaction; no LLM is involved in gesture selection.

Why Not Have the LLM Pick the Gesture?

The author tested this. Two findings:

1. **LLM-generated gesture names hallucinate.** Asked for a gesture name from a fixed list, GPT-4o-mini frequently returns something off-list (`"smile_warmly"` instead of `"wave"`).
2. **Latency.** Asking the LLM to return JSON with `{response: "...", gesture: "...", led: "..."}"` adds ~30% to response time and ~60% to output tokens (cost).

A rule-based planner is faster, cheaper, and 100% reliable. The "smartness" of the brain lives in the LLM's text; the gesture is paired with that text using fixed rules.

The Default Gesture Per Task Type

```
_TASK_DEFAULT_GESTURES = {
    "info_retrieval": "nod",
    "navigation": "point_forward",
    "social_conversation": "wave",
    "multilingual": "nod",
}
```

A T1 (info-retrieval) interaction defaults to a `nod` (acknowledging the question). A T2 (navigation) interaction defaults to `point_forward`. A T3 (social) interaction defaults to a `wave` (friendly greeting). T4 (multilingual) gets a `nod` — neutral but engaged.

Direction Patterns Override the Default

For T2 specifically, the planner scans the response text for direction-words:

```
_DIRECTION_PATTERNS = [
    (re.compile(r"\bon (your |the )?(left|east)\b", re.I), "point_left"),
    (re.compile(r"\bon (your |the )?(right|west)\b", re.I), "point_right"),
    (re.compile(r"\b(straight ahead|in front|forward|north)\b", re.I), "point_forward"),
    (re.compile(r"\b(upstairs|above|floor \d+|level \d+)\b", re.I), "point_up"),
    (re.compile(r"\b(turn left)\b", re.I), "point_left"),
    (re.compile(r"\b(turn right)\b", re.I), "point_right"),
]
```

If the response text is *"The Pepper room is at the end of the ground-floor corridor, on the right"*, the `(right|west)` pattern matches and the gesture becomes `point_right` — overriding the T2 default of `point_forward`.

Content-Triggered Gestures

```
_CONTENT_GESTURE_MAP = [  
    (re.compile(r"\b(hello|hi|hey|welcome|greet)\b", re.I), "wave"),  
    (re.compile(r"\b(goodbye|bye|see you|farewell)\b", re.I), "wave_goodbye"),  
    (re.compile(r"\b(show|look at|see (the |this )?(map|screen|tablet|display))\b", re.I), "show_tablet"),  
    (re.compile(r"\b(thinking|let me check|processing)\b", re.I), "think"),  
    (re.compile(r"\b(sorry|apologise|don't know|not sure|unclear)\b", re.I), "confused"),  
    (re.compile(r"\b(absolutely|certainly|exactly|correct|yes)\b", re.I), "nod"),  
]
```

These work across task types. If the LLM's response says *"Hello!"*, the planner picks wave. If it says *"I'm not sure"*, the planner picks `confused` (and the LED turns red-orange).

LED Colours

`GESTURE_LED_COLORS` is a fixed mapping from gesture to hex colour. The colour choice is informed by:

- Green = positive / friendly (wave, bow, nod)
- Blue = informational / calm (pointing, neutral)
- White = directs attention to the tablet
- Yellow = thinking
- Red-orange = confusion / error

The LED is a *redundant* channel — the speech alone carries the answer — but it makes the interaction feel more responsive and helps participants distinguish *"the robot is processing"* from *"the robot is talking"*.

13.4 omnillm/robotics/whisper_stt.py — Speech-to-Text

At a Glance

WhisperSTT is a thin wrapper around two STT backends:

Backend	Library	When to use
<code>local</code>	<code>openai-whisper</code> (the original Whisper pip package)	Default. Privacy-preserving, no cost, runs offline.
<code>api</code>	<code>openai</code> SDK	Only if you set <code>WHISPER_BACKEND=api</code> and provide an OpenAI key. Faster on weak laptops; ships your audio to the cloud.

The class accepts WAV bytes (or a path-like object) and returns the transcribed string plus an estimated confidence. Used by:

- The agent graph's `transcribe_audio` node (when called via `/interact` with `audio` in the body).
- The Flask `/transcribe` endpoint (Whisper-only, no LLM call).

Why Whisper-base by Default?

`WhisperSTT(backend="local", model_size="base")` is the default. `base` is the smallest non-trivial Whisper model — about 74M parameters, ~140 MB download, runs in ~700ms per 5-second utterance on CPU. We picked it over `tiny` (39M, 200ms, noticeably worse on accents) and over `small/medium/large` (slower, marginally better).

The pilot showed that `base` correctly transcribes the canonical 4 prompts (English + Italian) with 100% accuracy. For deployment with more participant accents the model size can be bumped via `model_size="small"`.

Why Local, Not API?

Discussed in Chapter 8. Briefly:

1. **GDPR.** Participant audio never leaves the lab laptop.
2. **Latency.** Local Whisper-base = ~700ms; cloud API = ~1.5s + network jitter.
3. **Cost.** Free.

The tradeoff: Whisper-base is weaker on heavy accents than Whisper-large or the cloud API. For the experimental study, the experimenter's role is to ask the participant to re-state if Whisper misfires — and the misfire rate is constant across conditions (it has no effect on the condition-level outcome).

Chapter 14 — Server & NAOqi Bridge

Three files between them are the entire HTTP surface area of OmniLLM. Two of them run in Python 2.7 (`naoqi_client.py` and `naoqi_bridge_server.py`); one runs in Python 3.11+ (`app.py`). They never share a process. They only share JSON.

14.1 `omnilm/server/app.py` — The Flask AI Server

At a Glance

`app.py` is the Python 3.11+ Flask server. It exposes six HTTP endpoints; it loads the gateway, RAG pipeline, and LangGraph at startup; and it serves as the single integration point that the Python 2.7 NAOqi processes talk to.

Endpoints

Method	Path	Purpose	Used by
GET	/health	Liveness probe. Returns <code>{"status": "ok", "version": "0.1.0"}</code> .	Anything wanting to know if the server is up
GET	/status	Server configuration dump: default model, RAG state, KB path, available models, LangGraph availability	Diagnostic tools, the README setup test
POST	/transcribe	Whisper STT only. Body: <code>{"audio": "<base64 WAV>"}</code> . Returns text + detected language.	Test scripts; sanity-checking Whisper without the full pipeline
POST	/interact	The main endpoint. Body: <code>{"audio" or "text", "participant_id", "session_id", "condition", "rag_enabled"}</code> . Returns the RobotAction JSON.	<code>naoqi_client.py</code> , <code>run_subject_experiment.py</code> , curl, anything
POST	/evaluate	Submit a questionnaire score. Body: <code>{"session_id", "participant_id", "condition", "scores": {...}}</code> . Writes to ExperimentLogger.	Future digital questionnaire UI (currently paper-based)
GET	/export	Returns all logged interaction records as JSON for off-line analysis	The analysis Jupyter notebook (Chapter 30)

The /interact Handler — Where Everything Connects

Look at lines 223–312 of `omnillm/server/app.py`. The flow is:

1. Parse JSON body. Extract utterance, `audio_b64`, `participant_id`, `session_id`, `condition`, `rag_enabled`.

2. Decode the base64 audio if present.
3. Resolve the per-condition model from `CONDITION_CONFIGS[condition].model_id` and *inject it into the graph state*. (This is the May 2026 fix discussed in Chapter 12.)
4. Try to invoke the LangGraph (`graph.ainvoke(state)`). If it succeeds AND `action.speech` is non-empty, return the action.
5. If the graph returns empty speech OR raises an exception, **fall back to `_fallback_interact()`** — a tiny synchronous handler that calls `gateway.query()` directly and assembles a minimal `RobotAction`.

The fallback path is important: it means the server *always* responds with something, even if LangGraph or RAG temporarily breaks. The fallback's metadata field carries `"path": "fallback"` so downstream analysis can flag which interactions used it.

The Lazy LangGraph Import

```
def _get_graph():
    nonlocal _graph
    if _graph is None:
        try:
            from omnillm.hri.agent_graph import build_hri_graph
            _graph = build_hri_graph(gateway=gateway, rag=rag, logger=exp_logger, default_model=_default_model)
        except ImportError:
            logger.warning("LangGraph not installed – falling back to direct gateway.")
    return _graph
```

LangGraph is an optional install (`pip install omnillm[hri]`). If it's not installed, the server still boots; it just falls back to the direct-gateway path for every interaction. This is useful for very minimal deployments where only the `/transcribe` and `/health` endpoints are needed.

The CORS-and-Host Decision

The server's `--host` defaults to `0.0.0.0` so Pepper (on the LAN) can reach it. CORS is not enabled — the API is meant to be called from same-LAN processes, not browsers. If a future tablet-based participant UI needs to call `/interact` from a browser, add `flask-cors` and a strict allowlist.

Why Flask and Not FastAPI?

FastAPI would arguably be the modern choice (built-in async, automatic OpenAPI docs). We chose Flask because:

1. **Smaller dependency tree.** Flask + Jinja vs FastAPI + Pydantic + Starlette + uvicorn — three vs ten transitive packages.
2. **Sync handlers compose easily with async work.** Our `_run_async` helper creates a one-shot event loop per request, which is a clean pattern in Flask. FastAPI's async handlers would mostly be calling `asyncio.gather()` over LiteLLM calls — fine, but no win for us at this scale.
3. **Long-standing ecosystem.** Flask + Gunicorn for production has been a stable combination since 2014; we will not be surprised by a behaviour change.

14.2 omnillm/server/naoqi_bridge_server.py — Python 2.7's Side of Topology 2

At a Glance

`naoqi_bridge_server.py` is a **stdlib-only HTTP server** that runs in Python 2.7 near Pepper. It accepts JSON action commands from the Python 3 PepperBridge and translates them to NAOqi RPCs. Stdlib-only because installing modern pip packages on Python 2.7 is now flaky and often fails on Windows; the file uses `BaseHTTPServer.HTTPServer` and `urllib2` and nothing else.

Endpoints It Exposes

Method	Path	Body	Returns
GET	/ping	—	<pre>{"ok": true, "naoqi": true, "simulation": false, "robot_ip": "...", "robot_port": ...}</pre>
POST	/action	RobotAction-like dict	<pre>{"ok": true, "executed": {...}, "errors": {...}}</pre>
POST	/audio/record	<pre>{"duration_seconds": 5, "sample_rate": 16000, "channels": [0,0,1,0]}</pre>	<pre>{"ok": true, "audio_b64": "..."} </pre>
GET	/sensors	—	Touch sensors + face detection state + battery
POST	/tracker/start	<pre>{"target": "Face"}</pre>	Starts ALTracker face-follow
POST	/tracker/stop	—	Stops face tracking
POST	/disconnect	—	Calls <code>motion.rest()</code> and shuts down cleanly

The NaoqiFacade Class

Inside the bridge server is a single `NaoqiFacade` object that wraps the NAOqi proxies (`ALAnimatedSpeech`, `ALMotion`, `ALBehaviorManager`, `ALLeds`, `ALAudioRecorder`, `ALTracker`, `ALMemory`). The facade pattern means the HTTP handler functions don't import NAOqi directly — they call `facade.say(text)`, `facade.gesture(name)`, `facade.record_audio(...)`, etc.

This indirection is what lets the bridge server fall back to **simulation mode** when NAOqi can't be reached. In simulation mode, every `facade.*` call is a no-op that logs to stdout. The `/ping` endpoint

returns `simulation: true` so the Python 3 client knows.

The Windows-11 Loopback Bug Workaround

When connecting to Choregraphe's virtual robot on Windows 11, NAOqi 2.5's `ALBroker` constructor must be told to listen on `127.0.0.1` *explicitly*, otherwise it tries to bind to `0.0.0.0` and fails silently. The fix:

```
broker = ALBroker("OmniLLMBroker",
                  "127.0.0.1",      # NB: explicit, not "0.0.0.0"
                  0,                # let OS pick an ephemeral port
                  robot_ip, robot_port)
```

Real Pepper (running NAOqi natively on Linux) does not have this bug. The bridge server detects whether it's connecting to a virtual or real robot via the `--robot-ip` flag and applies the workaround only when `robot_ip == "127.0.0.1"`.

How the AI Server Drives It

```
# Python 3 LangGraph node
from omnillm.robotics import make_pepper_bridge
bridge = await make_pepper_bridge(robot_ip="127.0.0.1", bridge_port=6000)
await bridge.execute_action(RobotAction(
    speech="Hello! I am Pepper.",
    gesture="wave",
    emotion_led="#00FF88",
))
```

Internally this becomes:

```
POST http://127.0.0.1:6000/action
{
  "speech": "Hello! I am Pepper.",
  "gesture": "wave",
  "emotion_led": "#00FF88",
  "movement": null,
  "tablet_url": null,
  "metadata": {}
}
```

The bridge server's handler:

```
def do_POST(self):
    if self.path == "/action":
        payload = json.loads(self.rfile.read(int(self.headers.get("Content-Length", 0))))
        result = self.facade.execute_action(payload)
        self._send_json(200, result)
```

`facade.execute_action()` runs (in parallel where safe) all four sub-actions: `ALAnimatedSpeech.say(speech)`, `ALMotion.runBehavior(gesture)`, `ALLeds.fadeRGB(...)`, `ALTabletService.showWebview(tablet_url)`. Each is wrapped in a `try/except`; the result dict reports per-sub-action success/failure so the Python 3 caller can decide what to do.

Why Stdlib HTTP, Not Flask?

Two reasons:

1. **Python 2.7 + Flask + Windows = unhappy.** Modern Flask (≥ 2.0) drops Python 2.7 entirely. The last Flask version that supports 2.7 (Flask 1.1) requires Werkzeug 1.0 which requires `click<8` which

is unmaintained. Stdlib `BaseHTTPServer` works forever.

2. **Tiny surface area.** The bridge has six endpoints. Hand-coding the dispatcher is 30 lines.

14.3 omnilm/server/naoqi_client.py — Python 2.7's Conversation Loop (Topology 1)

At a Glance

`naoqi_client.py` is the alternative Python 2.7 driver — the one used in Topology 1 ("Pepper polls AI server"). It runs in a loop, captures input, sends to the AI server, and enacts the response. It supports **three trigger modes** (text, touch, vad) selected via `--trigger`.

The Three Trigger Modes

Trigger	How a turn starts	Audio source	Works on virtual Pepper?
text	Type a line + Enter at the terminal. Empty line quits.	None (text->TTS)	[OK] Yes
touch	Touch Pepper's front-head sensor. Records <code>--record-seconds</code> (default 5) seconds.	<code>ALAudioRecorder</code>	[X] No (virtual has no mic)
vad	Continuous capture. When energy threshold is exceeded, recording begins; when energy drops, recording ends.	<code>ALAudioDevice</code>	[X] No

The three modes share the same downstream flow:

```
[trigger fires]
  v
[capture input: audio bytes OR typed text]
  v
POST /interact with audio (base64) or text + (participant, session, condition, rag_enabled)
  v
[receive RobotAction JSON]
  v
ALAnimatedSpeech.say(action["speech"])
ALMotion.runBehavior(action["gesture"])
ALLeds.fadeRGB("FaceLeds", action["emotion_led"], 0.3)
  v
[loop]
```

Face Tracking via `--track-face`

When `--track-face` is passed, the client starts `ALTracker` with the Face target at startup. Pepper's head will now turn to follow detected human faces during the entire session, regardless of which trigger fires conversational turns. This is the **first-class** face-tracking integration — Section 13.2 of the experimental protocol requires it.

```
C:\Python27\python.exe omnillm\server\naoqi_client.py ^  
  --robot-ip 192.168.1.42 ^  
  --trigger touch ^  
  --record-seconds 5 ^  
  --condition C ^  
  --track-face
```

Choice Matrix for the Study

For the N=15 study at DIBRIS, the recommended trigger is **touch**:

- Pure-text trigger feels artificial — the participant types instead of speaking to the robot.
- VAD trigger is sensitive to ambient lab noise (HVAC, students passing, other conversations) and prone to triggering on noise. The pilot found false-positive triggers averaging ~1.2 per minute in the DIBRIS open-plan area.
- Touch trigger has a clear, intentional start (the participant touches Pepper's head), a fixed-duration capture (5 seconds), and a clear end. It is the most reproducible mode across participants.

For **demos** and **dry runs** before the study, text mode is the right choice — it works on virtual Pepper and lets the experimenter test the dialogue without speaking aloud in a public space.

How `naoqi_client.py` Handles Errors

If `/interact` returns an error or the response has empty `speech`, the client uses a fixed apology line:

```
SAFE_SPEECH_FALLBACK = (  
    "I'm sorry, I didn't quite catch that. Could you say it again?"  
)
```

This is what the participant hears if Whisper failed to transcribe, or if all LLM backends were offline, or if the network broke. **Silence is the worst outcome**; a polite "try again" is the second-worst-but-acceptable outcome. The experimenter logs every fallback for off-line analysis.

Chapter 15 — Utilities

Four small files in `omnilm/utis/` between them implement the entire data-collection pipeline for the experimental study. They are also the easiest files to read in the project.

15.1 omnillm/utils/experiment_logger.py — The Central Dataset

At a Glance

ExperimentLogger is the **single point of truth** for what happened during a study. It exposes one method, `log_interaction(...)`, that appends one `InteractionRecord` to:

- An in-memory `self.records` list (queryable via `/export`)
- A JSON-Lines file on disk (`results/interactions_<date>.jsonl`)

Plus a `to_csv()` method that produces the canonical flat CSV for analysis.

The `InteractionRecord` Dataclass

Every field that gets logged for every interaction:

Field	Type	Source
<code>session_id</code>	str	Passed in <code>/interact</code> body
<code>participant_id</code>	str	Passed in <code>/interact</code> body
<code>condition</code>	str	"A"—"E"
<code>task_type</code>	str	Output of <code>HRITaskClassifier</code>
<code>utterance</code>	str	Whisper output (or typed text)
<code>response</code>	str	LLM output (the <code>speech</code> field in <code>RobotAction</code>)
<code>model_id</code>	str	Which LLM actually produced the response
<code>latency_ms</code>	float	End-to-end (excluding Pepper's TTS playback)
<code>input_tokens</code>	int	Prompt tokens
<code>output_tokens</code>	int	Completion tokens
<code>cost_usd</code>	float	Computed from the registry
<code>rag_enabled</code>	bool	Was RAG active for this interaction?
<code>rag_faithfulness</code>	float	LLM-as-judge score (-1.0 = not scored)
<code>rag_chunk_count</code>	int	How many chunks were retrieved

judge_score	float	Future: post-hoc LLM-as-judge quality
language	str	Detected language
gesture_used	str	What Pepper actually did
task_success	bool	Set by the experimenter post-hoc (default None)
timestamp	str	ISO 8601, UTC
notes	str	Free-form (e.g. JSON-encoded questionnaire scores)

Why JSON-Lines, Not One Big JSON?

JSON-Lines (one JSON object per line, no outer array) is append-only safe. If the server crashes mid-session, you don't lose the partial data. With a single big JSON, you would have to either rewrite the whole file on every append (slow + corruption-prone) or accept that a crash loses everything since the last save.

CSV Export Schema

```
session_id, participant_id, condition, task_type, utterance, response,
model_id, latency_ms, input_tokens, output_tokens, cost_usd,
rag_enabled, rag_faithfulness, judge_score, task_success, language,
gesture_used, timestamp, notes
```

This is the shape the pandas analysis in Chapter 30 consumes. Examples are in `results/subject_run_P000_*.csv`.

15.2 omnillm/utils/questionnaire.py — Likert, Godspeed, Pairwise, Observer

At a Glance

Four dataclasses cover the full post-session questionnaire instrument:

Dataclass	Items	Used for
<code>InteractionQuestionnaire</code>	5 Likert items (accuracy, naturalness, trust, gesture, speed) on 1–7	Per-condition rating, the primary outcome
<code>GodspeedResponse</code>	5 subscale mean scores (anthropomorphism, animacy, likeability, perceived_intelligence, perceived_safety) on 1–5	Optional secondary outcome; comparability with HRI literature (Bartneck et al. 2009)
<code>PairwisePreference</code>	Two conditions + which preferred + reasoning	Feeds the ELO scorer to build the <i>embodied_hri</i> leaderboard
<code>ObserverRating</code>	Gesture-sync quality, task_completed, breakdown_count, notes	Filled by the experimenter live during the session

`QuestionnaireCollector` — Aggregation + Export

The collector holds lists of all four record types and offers:

- `add_interaction_response(q)`, `add_godspeed(gs)`, `add_pairwise_preference(p)`, `add_observer_rating(obs)`
- `summary_by_condition()` — mean Likert per condition, ready for the thesis tables
- `pairwise_win_rates()` — pairwise win rates for the ELO update
- `save(path)` — full JSON dump
- `to_csv(path)` — interaction-questionnaire-only CSV

Recommended Delivery for the Study

(I promised in the preface to answer this in the book.) **The author's recommendation is a hybrid:**

1. **Paper questionnaires** for the 5 Likert items + Godspeed + pairwise preference + open-ended comments. Familiar; doesn't introduce a second screen; participant cannot accidentally edit prior answers; ethics-cleared trivially.
2. **Digital observer logging**. The experimenter sits at the laptop and POSTs `observerRating` records to `/evaluate` live during the session. Faster than handwriting; captures timestamps automatically.
3. **Post-session digitisation**. After each session, the experimenter types the paper Likert scores into a short Python script that calls `collector.add_interaction_response(...)` and `collector.save(...)`. ~3 minutes per participant. The paper original is kept in a binder as ground

truth.

This balances: minimal participant-facing tech (paper), maximum experimenter efficiency (digital), and zero data loss (paper backup).

Validation Note

`InteractionQuestionnaire.__post_init__` enforces $1 \leq \text{score} \leq 7$ for every Likert item and raises `ValueError` otherwise. This catches the most common digitisation typo (entering 0 or 8) at insertion time rather than during analysis.

15.3 omnillm/utils/cost_tracker.py — Per-Model USD Spend

At a Glance

CostTracker reads all logged InteractionRecords, groups by model_id, and computes:

- Total spend per model
- Number of interactions per model
- Average cost per interaction per model
- Cumulative spend across all models
- Spend per condition

Output is the omnillm costs CLI command, which prints a sorted table. Useful to flag a runaway model mid-experiment.

Why Compute From the Log, Not Real-Time?

Because every interaction already writes its cost_usd to the log via the gateway's per-call cost calculation. There's no need for a separate real-time counter; we just pandas.read_json() the log on demand and group.

15.4 omnillm/utils/export.py — JSON ↔ CSV ↔ Markdown

At a Glance

`export.py` is a small format converter that the `omnillm export` CLI calls. Three output formats:

- `--format csv` -> flat CSV, ready for pandas / Excel
- `--format markdown` -> rendered table, ready to paste into the thesis
- `--format json` -> identity copy (mostly for testing)

Used during analysis to produce the leaderboard tables in `book2/13_part5_experiments.md` and in the thesis itself.

Chapter 16 — The Python 2.7 ↔ Python 3.x Bridge Problem (and Three Different Solutions to It)

This chapter is the most architecture-defining single problem in OmniLLM. It is also the most reusable lesson — every project that integrates a modern AI stack into legacy hardware faces some version of this.

At a Glance

NAOqi's official Python SDK is locked to Python 2.7. The modern AI stack we want to use (LangGraph, LiteLLM, ChromaDB, transformers, etc.) requires Python 3.10+ and is syntactically invalid on Python 2.7. The two cannot coexist in a single process. OmniLLM solves this with three different cross-version bridges, each appropriate to a different scenario.

Solution 1 — HTTP Server in Python 3, Polling Client in Python 2.7 (Topology 1)

The original solution. `omnillem/server/app.py` (Python 3) listens on `:5000`. `omnillem/server/naoqi_client.py` (Python 2.7) drives Pepper's I/O and POSTs to the server. The two processes never share memory; the JSON over HTTP is the entire contract.

+-----+	HTTP	+-----+
<code>naoqi_client.py</code>	POST /interact	<code>app.py</code>
(Python 2.7)	----->	(Python 3.11+)
	<-----	
<code>ALAudioDevice</code>	RobotAction	<code>LangGraph</code>
<code>ALAnimatedSpeech</code>		<code>RAG, LiteLLM</code>
<code>ALMotion</code>		
+-----+		+-----+

When to use: quick demos, single-question scripts, anything where Pepper is the conversational initiator.

Solution 2 — HTTP Server in Python 2.7, Async Client in Python 3 (Topology 2)

The May 2026 addition. `omnillem/server/naoqi_bridge_server.py` (Python 2.7) listens on `:6000`. The Python 3 `PepperBridge` is an `aiohttp` client of that bridge. The AI stack now initiates conversation; Pepper is a passive actuator.

+-----+	HTTP	+-----+
AI Layer	POST /action	<code>naoqi_bridge_</code>
(Python 3.11+,	----->	<code>server.py</code>
<code>LangGraph</code> etc.)	<-----	(Python 2.7)
	ok / error	
<code>PepperBridge</code>		<code>NaoqiFacade</code>
(<code>aiohttp</code>)		<code>ALAnimatedSpeech</code>
+-----+		+-----+

When to use: the live experimental study (`run_subject_experiment.py` initiates and drives), and any future scenario where the agent graph wants to issue *multiple* commands to the robot during one cognitive step (e.g. "speak the first sentence while computing the second").

Solution 3 — In-Process Stub (No Robot Anywhere)

The simplest solution. PepperBridge falls back to *stub mode* if neither bridge nor real NAOqi is reachable. `execute_action()` prints to stdout and returns success. The entire pipeline runs in CI, in unit tests, on a beach without internet.

When to use: all 289 pytest tests; CI; developing the AI Layer offline.

Why Three Solutions and Not One Universal One?

Each solution optimises a different axis:

Solution	Optimises	Cost
Topology 1	Simple demos, robot-initiated dialogue	AI Layer cannot drive robot mid-reasoning
Topology 2	AI-initiated dialogue, streaming behaviours, study reproducibility	Two PowerShell terminals required
Stub	Zero-robot development & CI	No real robot behaviour, obviously

A single "universal" bridge would have to optimise everything — which means optimising nothing. Three small solutions composed cleanly is better than one big one fighting itself.

A General Pattern for Legacy-Hardware AI Integration

The pattern that emerges from OmniLLM generalises:

1. **The legacy boundary is HTTP-and-JSON.** Anything that can speak HTTP can speak it. Python 2.7 can. ROS can. C++ can. An old Java service can.
2. **The legacy side runs only its SDK and a tiny HTTP server.** Don't add a single pip-installable dependency that wasn't in the 2010s ecosystem.
3. **The modern side runs everything else.** Modern Python, modern PyTorch, modern Anthropic SDK.
4. **Every interaction crosses the boundary exactly once.** Multi-hop crossings (modern->legacy->modern->legacy) make every error 4× harder to debug.
5. **The boundary is the canonical schema.** The `RobotAction` JSON is documented in `omnillem/robotics/bridge.py`. Any version of either side that obeys it is interoperable.

This is the deployment pattern that the author expects to recommend for the next decade of "modern AI in legacy robot" integrations — because robot middleware moves *much* slower than AI library churn.

Part IV — Setup, Execution, & Building from Scratch

Parts I, II, and III explained the project. Part IV gets it running on your machine and, in Chapter 21, on the real Pepper at the lab tomorrow.

Chapter 17 — Installing OmniLLM From Zero

At a Glance

OmniLLM installs cleanly on Windows 10/11, macOS, and modern Linux distributions, given a Python ≥ 3.11 interpreter and ~3 GB of free disk (most of which is the optional ChromaDB / Whisper model cache). Cloud LLM access is optional — Ollama-only local inference works without a single API key.

This chapter walks through the installation [on the author's laptop \(Windows 11 + Anaconda + venv\)](#) in detail. macOS and Linux instructions are given afterwards in collapsed form because they are simpler.

Prerequisites

Requirement	Check it	Why
Python ≥ 3.11	<code>python --version</code>	Required for <code>litellm</code> , <code>langgraph</code> , <code>chromadb</code> .
pip	<code>pip --version</code>	Standard install tool.
git	<code>git --version</code>	To clone the repo.
Visual Studio Code (recommended editor)	—	For IntelliSense + the integrated terminal.
PowerShell ≥ 5.1 (Windows)	<code>\$PSVersionTable.PSVersion</code>	The shell every command in this book is written in.
Choregraphe $\geq 2.5.10$ (optional, virtual Pepper only)	Run the installer once	SoftBank's IDE + virtual Pepper. Windows-only officially.
Python 2.7 (optional, real Pepper only)	<code>C:\Python27\python.exe --version</code>	NAOqi SDK runs on this and only this.
NAOqi Python 2.7 SDK (optional, real Pepper)	<code>import naoqi</code> succeeds in <code>C:\Python27\python.exe</code>	The actual NAOqi Python bindings.
Ollama (optional, local LLM)	<code>ollama --version</code>	Free local model runtime.

Step-by-Step on Windows 11

1. Clone the repository

Open a PowerShell window and run:

```
cd C:\Users\akshi\OneDrive\Desktop
git clone https://github.com/Akshita-sr/OmniLLM.git
cd OmniLLM
```

2. Create and activate the virtual environment

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

After activation your prompt should show (venv) at the front. If PowerShell refuses to run `Activate.ps1` with an "execution policy" error, run once per machine:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

3. Install the package and all extras

```
pip install --upgrade pip
pip install -e ".[all]"
```

The `[all]` extra installs dev tools (pytest, pytest-asyncio, pytest-mock), robotics extras (flask, aiohttp), HRI extras (langgraph, langchain, chromadb, sentence-transformers, langdetect), and book-build tools (reportlab, xhtml2pdf, markdown). Total ~600 MB on disk after first install.

4. (Optional) Configure your .env for cloud LLMs

```
Copy-Item .env.example .env
notepad .env
```

Edit the file to add the API keys you actually have:

```
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=AIza...
DEEPSEEK_API_KEY=sk-...
QWEN_API_KEY=...
```

You only need the keys for models you intend to use. Leave the rest blank or delete the lines.

5. (Optional) Install Ollama and pull a free local model

```
# Install from https://ollama.com (one-click MSI installer on Windows)
ollama --version           # confirm install
ollama pull llama3.2:3b     # ~2 GB; required for Condition B of the study
ollama pull qwen2.5:7b      # ~4 GB; optional second local model
ollama serve                # start the daemon (it auto-starts on Windows boot)
```

6. (Optional) Install Python 2.7 + NAOqi SDK for real-Pepper work

Skip this section entirely if you are not connecting to a real Pepper or to Choregraphe's virtual Pepper.

1. Download Python 2.7 from python.org -> install to `C:\Python27\` (the exact default location — the scripts in this repo hard-code this path).
2. Download the **pynaoqi SDK** for Python 2.7 from SoftBank's developer portal: `pynaoqi-python2.7-2.5.5.5-win32-vs2013.zip`. Extract to `C:\pynaoqi\`.
3. Tell Python 2.7 where to find the SDK by adding `C:\pynaoqi\pynaoqi-python2.7-2.5.5.5-win32-vs2013\lib` to `PYTHONPATH`. Confirm:

```
cmd set PYTHONPATH=C:\pynaoqi\pynaoqi-python2.7-2.5.5.5-win32-vs2013\lib;%PYTHONPATH%
C:\Python27\python.exe -c "import naoqi; print(naoqi.__file__)"
```

If you see a path, you are done. If you see `ImportError`, the `PYTHONPATH` is not set correctly.

7. Verify

```
omnilm --version          # should print "OmniLLM 0.1.0"
omnilm models             # should print a coloured table of 19 models
pytest tests/ -q          # should print "289 passed"
```

If the three commands above succeed, your installation is complete.

macOS / Linux

The flow is identical, with three substitutions:

- `python3 -m venv venv` (instead of `python -m venv venv`)
- `source venv/bin/activate` (instead of `.\venv\Scripts\Activate.ps1`)
- All PowerShell-specific syntax in later chapters has a Bash equivalent in Appendix C.

Real-Pepper work is harder on macOS/Linux because the official pynaoqi Python 2.7 SDK ships only for Windows and certain old Linux versions. If you must, build NAOqi on Linux using Aldebaran's old build instructions or use a Windows VM for the Python-2.7 side.

Common Install Failures

Symptom	Cause	Fix
Microsoft Visual C++ 14.0 is required	Some pip wheel missing on Windows	Install Microsoft Build Tools for C++; retry.
chromadb install fails	Older pip cannot resolve the dependency tree	<code>pip install --upgrade pip</code> then retry.
whisper complains about ffmpeg	Whisper needs ffmpeg on PATH for audio decoding	<code>winget install ffmpeg</code> (Windows) or <code>brew install ffmpeg</code> (mac)
<code>import naoqi</code> fails in Python 2.7	<code>PYTHONPATH</code> not set or 64-bit Python tried to load 32-bit SDK	Confirm Python 2.7 is the 32-bit build; reset <code>PYTHONPATH</code> .
<code>ollama serve</code> says "address already in use"	Ollama is already running as a service	That's fine; just close the manual <code>ollama serve</code> .

Chapter 18 — Scenario A — Running Without Any Robot (Text-Only)

At a Glance

The fastest way to confirm OmniLLM is working end-to-end on a fresh install. No robot, no Choregraphe, no NAOqi. Talk to the brain via curl or the CLI.

Step 1 — Start the AI server

```
cd C:\Users\akshi\OneDrive\Desktop\OmniLLM
.\env\Scripts\Activate.ps1
python -m omnillm.server.app --host 127.0.0.1 --port 5000
```

Expected output:

```
INFO:omnillm.rag.pipeline:ChromaDB initialised at .chroma
INFO:__main__:Knowledge base loaded — 49 total chunks
* Running on http://127.0.0.1:5000
* Restarting with watchdog
```

The "49 total chunks" number depends on what's in `knowledge_base/`. Anything between 40 and 60 is normal as of the May 2026 DIBRIS knowledge base.

Step 2 — Sanity check from a second terminal

```
curl http://127.0.0.1:5000/status
```

Expected JSON response (formatted for readability):

```
{
  "status": "ok",
  "default_model": "openai-gpt4o-mini",
  "rag_enabled": true,
  "knowledge_base": "C:\\Users\\akshi\\...\\knowledge_base",
  "knowledge_base_exists": true,
  "rag_uses_chromadb": true,
  "available_models": ["openai-gpt4o-mini", "claude-haiku", ...],
  "langgraph_available": true
}
```

If `rag_uses_chromadb` is false, ChromaDB failed to load — RAG will still work via the keyword-search fallback but the faithfulness scores will be lower. If `langgraph_available` is false, install the `[hri]` extra.

Step 3 — Ask a question via curl

```
curl -X POST http://127.0.0.1:5000/interact `
-H "Content-Type: application/json" `
-d '{"text":"What time does the lab open?","participant_id":"P-test","session_id":"s-1","condition":"A","rag_enab
```

Expected response:

```
{
  "speech": "The lab opens at 08:30 on weekdays.",
  "gesture": "nod",
  "emotion_led": "#00FF88",
  "metadata": {
    "task_type": "info_retrieval",
    "model_id": "openai-gpt4o-mini",
    "rag_enabled": true,
    "condition": "A"
  }
}
```

You now have proof that gateway, RAG, agent graph, gesture planner, and JSON marshalling are all working. The Pepper-specific layer is the only missing piece.

Step 4 — Run the batch test for all conditions

This is the canonical pre-flight check before any robot work:

```
python scripts\pepper_demo\test_all_conditions.py
```

Output (abridged):

```
[01/15] A / T1 / "What time does the lab open?" -> "The lab opens at 08:30 on weekdays."
[02/15] A / T2 / "Where is the Pepper room?" -> "The Pepper room is at the end..."
[03/15] B / T1 / ... -> (Llama 3.2 3B via Ollama)
...
Wrote scripts/pepper_demo/test_results.txt
```

A full pass takes ~3 minutes on a typical laptop with Ollama running. If any test errors out, the failing condition is the one to debug *before* moving to the robot scenarios.

Chapter 19 — Scenario B — Running With Choregraphe's Virtual Robot

At a Glance

A simulated Pepper from SoftBank. The full HRI pipeline executes; only the **gestures** silently fall back to "behavior not installed" because the virtual robot's animation library is a placeholder. Speech and LEDs work fine; audio I/O does not (no microphone). Use `--trigger text` for any conversational testing.

Step 1 — Launch Choregraphe

Double-click the Choregraphe shortcut. The IDE opens. From the menu bar:

Connection -> Connect to... -> select the **AKSHITA** virtual robot in the list -> **Connect**.

A small popup window shows the assigned virtual-robot port (e.g. 49959). **Write this number down** — call it `VIRTUAL_PORT`. It changes every launch.

Step 2 — Start the AI server

In PowerShell window #1:

```
cd C:\Users\akshi\OneDrive\Desktop\OmniLLM
.\venv\Scripts\Activate.ps1
python -m omnillm.server.app --host 127.0.0.1 --port 5000
```

Step 3 — Sanity-check NAOqi alone (no AI)

In PowerShell window #2 (Python 2.7):

```
C:\Python27\python.exe scripts\pepper_demo\demo_pepper_omnillm.py --check-only --robot-port 49959
```

Replace 49959 with your actual `VIRTUAL_PORT`. Expected output:

```
[OK] Connected to robot at 127.0.0.1:49959
[OK] ALAnimatedSpeech proxy created
[OK] ALMotion proxy created
[OK] Pepper says: "Hello from my own Python script"
[OK] All sanity checks passed.
```

The virtual robot speaks the sentence via its "Robot Dialog" panel in the Choregraphe UI. If you see the sentence in the dialog panel: green light. If you see `ALBroker construction failed`, see Chapter 16 (Windows 11 loopback bug).

Step 4 — Single-shot demo

```
C:\Python27\python.exe scripts\pepper_demo\demo_pepper_omnillm.py ^
--condition A --rag --question "What time does the lab open?" ^
--robot-port 49959
```

Expected: the virtual Pepper says *"The lab opens at 08:30 on weekdays."* in the dialog panel.

Step 5 — Interactive text-trigger client

```
C:\Python27\python.exe omnillm\server\naoqi_client.py ^  
  --robot-ip 127.0.0.1 --robot-port 49959 ^  
  --trigger text --condition A
```

The client prints a prompt; type a question; Pepper answers via the dialog panel; loop until you enter an empty line.

```
[ready] You ask: Where is Room 305?  
[interact] sending to server...  
[server] model=openai-gpt4o-mini path=graph latency=1430ms  
[robot] Pepper says: "Room 305 is on the 3rd floor. Take the lift or staircase on your left."  
[robot] gesture point_left (behavior not installed; speech still executed)  
[robot] LED -> #00AAFF  
[ready] You ask: _
```

The `behavior not installed` line is expected on the virtual robot — Chapter 3 documented this. Speech and LED still work.

Step 6 — Drive a full subject session against the virtual robot

This is the pre-real-Pepper rehearsal:

```
# In PowerShell window #1 (AI server already running)  
# In PowerShell window #3 (the driver):  
.\venv\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py `   
  --participant P000-rehearsal `   
  --server http://127.0.0.1:5000 `   
  --no-robot
```

The `--no-robot` flag bypasses the bridge entirely and only exercises the AI server. The 20-interaction matrix runs in ~5 minutes and produces:

```
results/subject_run_P000-rehearsal_<timestamp>.json  
results/subject_run_P000-rehearsal_<timestamp>.csv
```

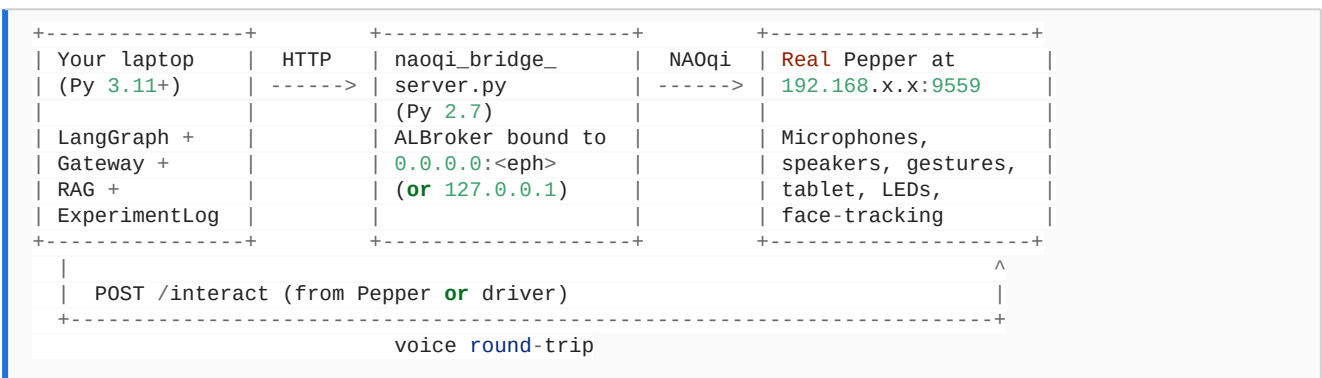
These are the exact files the actual study session produces — review them to confirm latency, model selection, condition routing, and RAG faithfulness all look right *before* you go to the lab.

Chapter 20 — Scenario C — Running With the Physical Pepper Robot

At a Glance

Same as Scenario B, but with the real Pepper at DIBRIS. Two terminals minimum (server + bridge), or three if you also run the experiment driver. The next chapter (Chapter 21) walks through Day Zero step-by-step with expected outputs at every stage; this chapter is the conceptual overview.

The Connection Sequence



Network Requirements

- Laptop and Pepper on the **same Wi-Fi LAN**. Pepper does not need internet (and at DIBRIS, it does not have it). Only laptop ↔ Pepper LAN reachability.
- The lab Wi-Fi at DIBRIS that Pepper is on may be different from the eduroam Wi-Fi your laptop joins by default. **Confirm with Prof. Sgorbissa or the lab technician which SSID Pepper is paired to.**
- `ping <PEPPER_IP>` must succeed from PowerShell *before* you start any OmniLLM process.

Firewall

Windows Firewall typically blocks inbound TCP to a freshly-installed Python interpreter. Allow Python through once:

```
New-NetFirewallRule -DisplayName "OmniLLM AI server" `
-Direction Inbound -Program "C:\Users\akshi\OneDrive\Desktop\OmniLLM\venv\Scripts\python.exe" `
-Action Allow -Profile Private
New-NetFirewallRule -DisplayName "OmniLLM NAOqi bridge" `
-Direction Inbound -Program "C:\Python27\python.exe" `
-Action Allow -Profile Private
```

Run once per laptop. The `Private` profile means the rule applies only when connected to a private/work network — not when you're on a coffee-shop Wi-Fi.

Topology Choice

You want to...	Use topology
Demo to a colleague — text-trigger, no study driver	Topology 1 (<code>naoqi_client.py</code>)
Run the formal study with <code>run_subject_experiment.py</code>	Topology 2 (<code>naoqi_bridge_server.py</code> + bridge client)
Touch-triggered VAD-based always-on conversation	Topology 1 with <code>--trigger touch</code> or <code>--trigger vad</code>
Streaming partial responses spoken before the LLM finishes	Topology 2 (only topology that allows multi-action mid-reasoning)

For the N=15 study at DIBRIS, the canonical setup is **Topology 2 with `run_subject_experiment.py`** (see Day Zero).

Chapter 21 — Day Zero — The Real-Pepper Deployment at DIBRIS

This chapter is the one to print out and bring to the lab. Follow it top-to-bottom. Tick the boxes as you go.

Every step lists the expected output. If reality matches the expected output, move on. If not, the troubleshooting table at the end will tell you what to do.

0 — Pre-arrival Checklist (the night before)

Do this on your laptop at home, not at the lab. If anything below fails, *fix it before you leave*.

- ☐ Latest code pulled: `git pull origin main && git status` shows "clean".
- ☐ Test suite passes: `.\venv\Scripts\python.exe -m pytest tests/ -q -> 289 passed.`
- ☐ `.env` is populated with all five keys (OPENAI, ANTHROPIC, GOOGLE, DEEPSEEK, QWEN). Confirm by sourcing: `powershell Get-Content .env | Select-String '_API_KEY' | Measure-Object -Line` Should report **5 lines**.
- ☐ Ollama is running locally and `llama3.2:3b` is pulled: `powershell ollama list` Output should include a line for `llama3.2:3b`.
- ☐ Confirm Condition B works locally: `powershell $env:OMNILLM_DEFAULT_MODEL = "llama3-8b-local" omnillm ask "Hello" -m llama3-8b-local` Should return a Llama-flavoured greeting within ~3 seconds.
- ☐ Charged laptop. Spare USB-C cable. Network cable (in case Wi-Fi flakes).
- ☐ Paper questionnaire forms × 1 per planned participant.
- ☐ Pen.

1 — At the lab: power up Pepper

- ☐ Press Pepper's chest button **once**. Pepper boots (LEDs cycle, takes ~30 seconds).
- ☐ Wait until Pepper says its IP aloud (e.g. "My IP address is one nine two, dot one six eight, dot one, dot one hundred"). **Write this down. Call it PEPPER_IP.**
- ☐ Confirm the same IP is shown on Pepper's chest tablet: `tap -> About -> Network`.

Expected: an IPv4 like `192.168.x.x` corresponding to the lab subnet.

If Pepper does not say its IP aloud after 60 seconds: long-press the chest button to shut down, wait 30 seconds, power on again.

2 — Confirm laptop ↔ Pepper LAN connectivity

- ☐ Connect your laptop to the same Wi-Fi as Pepper (ask the lab technician which SSID).
- ☐ `ipconfig` -> note your laptop's IPv4 on the LAN adapter. Call this `SERVER_IP`.
- ☐ `ping <PEPPER_IP>` from PowerShell.

Expected:

```
Reply from 192.168.1.100: bytes=32 time=4ms TTL=64
Reply from 192.168.1.100: bytes=32 time=3ms TTL=64
```

If 0% replies: you are not on Pepper's subnet. Re-join the correct Wi-Fi.

3 — Start the AI server (Terminal 1, Python 3 venv)

- [] Open PowerShell window #1: `powershell cd C:\Users\akshi\OneDrive\Desktop\OmniLLM\venv\Scripts\Activate.ps1 python -m omnillm.server.app --host 0.0.0.0 --port 5000`

Expected output (line for line):

```
INFO:omnillm.rag.pipeline:ChromaDB initialised at .chroma
INFO:__main__:Knowledge base loaded - 49 total chunks
* Serving Flask app 'omnillm.server.app'
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.x.x:5000
```

- [] In a second PowerShell window (or your phone's browser), verify: `powershell curl http://127.0.0.1:5000/status`

Expected JSON contains:

- `"rag_enabled": true`
- `"langgraph_available": true`
- `"rag_uses_chromadb": true`

If any of these is `false`: stop and consult the troubleshooting table at the end.

4 — Start the NAOqi bridge (Terminal 2, Python 2.7)

- [] Open PowerShell window #2: `powershell cd C:\Users\akshi\OneDrive\Desktop\OmniLLM\C:\Python27\python.exe omnillm\server\naoqi_bridge_server.py --robot-ip <PEPPER_IP> --robot-port 9559 --bind 0.0.0.0 --bridge-port 6000` (Replace `<PEPPER_IP>` with the actual IP you wrote down in step 1.)

Expected output:

```
[INFO] Connecting to NAOqi at <PEPPER_IP>:9559 ...
[OK] NaoqiFacade connected to <PEPPER_IP>:9559 (listen=0.0.0.0)
[OK] NAOqi bridge listening on http://0.0.0.0:6000/
[INFO] Endpoints: /ping /action /audio/record /sensors /tracker/start /tracker/stop /disconnect
```

If you see `[ERR] ALBroker construction failed: - Wrong IP — re-check PEPPER_IP. - Pepper is asleep — press the chest button to wake. - Firewall blocking inbound to Python 2.7 — see firewall step in Chapter 20.`

- [] Verify the bridge: `powershell curl http://127.0.0.1:6000/ping`

Expected:

```
{"ok": true, "naoqi": true, "simulation": false, "robot_ip": "<PEPPER_IP>", "robot_port": 9559}
```

The critical fields are `"naoqi": true` (NAOqi is connected) and `"simulation": false` (it's the real robot, not Choregraphe).

5 — One-shot Pepper-greets-you sanity test

This sends a single hand-crafted RobotAction to the bridge without any LLM in the loop. Use it to confirm Pepper itself responds to bridge commands.

- [] In any spare terminal: `powershell curl -X POST http://127.0.0.1:6000/action -H "Content-Type: application/json" -d '{"speech":"Hello, I am Pepper at DIBRIS, ready for the OmniLLM experiment.", "gesture":"wave", "emotion_led":"#00FF88"}'`

Expected at the robot: - Pepper speaks the sentence aloud through its speakers. - Pepper raises its arm and waves. - Pepper's eye LEDs turn green.

Expected JSON return:

```
{"ok": true, "executed": {"speech": true, "gesture": true, "emotion_led": true}, "errors": {}}
```

If `gesture: false` and the error reads `"behavior not installed: animations/Stand/..."`, you are talking to **Choregraphe's virtual robot**, not the real one. Check `--robot-ip` of step 4 — it should be the real Pepper IP, not `127.0.0.1`.

6 — Pilot run — you as participant P000-realPepper

Do this once before any real participant arrives. The pilot exercises the full 20-interaction matrix, drives the real robot, and writes a CSV you can scan for anomalies.

- [] Open PowerShell window #3: `powershell cd C:\Users\akshi\OneDrive\Desktop\OmniLLM .\venv\Scripts\Activate.ps1 .\venv\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py --participant P000-realPepper --server http://127.0.0.1:5000 --bridge http://127.0.0.1:6000`

Expected progress output (one line every ~6 seconds):

```
Subject experiment — participant=P000-realPepper session=578b023b
Server: http://127.0.0.1:5000
Robot bridge: http://127.0.0.1:6000

[01/20] A / T1_info_retrieval: What time does the lab open?
-> The lab opens at 08:30 on weekdays.
model=openai-gpt4o-mini path=graph latency=1437 ms
robot: ok
[02/20] A / T2_navigation: Where is the Pepper room at DIBRIS?
-> The Pepper room is located at the end of the ground-floor corridor...
model=openai-gpt4o-mini path=graph latency=1592 ms
robot: ok
[03/20] A / T3_social: Hello Pepper, how are you today?
...
[04/20] A / T4_multilingual: Ciao Pepper, dove si trova la stazione di Brignole?
-> Ciao! La stazione di Brignole si trova...
model=claude-haiku path=graph latency=1980 ms
robot: ok
[05/20] B / T2_navigation: Where is the Pepper room at DIBRIS?
-> [Llama 3.2 3B answer]
model=llama3-8b-local path=graph latency=2840 ms
```

```

robot: ok
...
[13/20] D / T4_multilingual: ...
model=council:openai-gpt4o-mini+claude-haiku+gemiini-flash latency=3140 ms
robot: ok
...
[20/20] E / T4_multilingual: ...
model=openai-gpt4o-mini path=graph latency=1750 ms
rag=n <-- RAG-off control
robot: ok

Wrote results/subject_run_P000-realPepper_<timestamp>.json
Wrote results/subject_run_P000-realPepper_<timestamp>.csv
Server-logged interactions for this run: 20

```

The critical things to verify *during* the pilot:

- [] Pepper actually speaks each line through its physical speakers (not just text on Choregraphe).
- [] Pepper's gestures execute physically (you see the arm move).
- [] The `model=` line shows the **expected** model for each condition:
- Condition A -> `openai-gpt4o-mini`
- Condition B -> `llama3-8b-local` (**not** GPT-4o-mini)
- Condition C -> varies by task (Claude Haiku for T3, GPT-4o-mini for T1/T2, Claude Haiku for T4)
- Condition D -> `council:...` with a plus-separated list
- Condition E -> `openai-gpt4o-mini` with `rag=n`
- [] Latencies stay under 4 seconds for A/B/C/E; D may go up to 4-5s (council is slower).

If any condition routes to the wrong model: stop, check `omnilm/hri/experiment.py: CONDITION_CONFIGS`, restart the server, retry. This is the May 2026 routing-bug class — see Chapter 12.

7 — Real-participant sessions

For each participant P001, P002, ... P015:

7.1 — Welcome the participant

- [] Greet, seat in the designated participant chair (Pepper roughly 1.2 m away facing the chair).
- [] Explain in plain language what's about to happen:

"You'll have a short conversation with Pepper. We will run five different 'personalities' of Pepper, four short questions each. After each personality, I'll hand you a brief paper questionnaire. The whole thing takes about half an hour. You can stop at any time."

- [] Hand over the **consent form**, give time to read, collect signed form.
- [] Hand over the **GDPR notice**, give time to read, collect signed receipt.

7.2 — Start the session

[] Run the driver with the participant's assigned ID: `powershell .\venv\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py --participant P003 --server http://127.0.0.1:5000 --bridge http://127.0.0.1:6000`

[] At each condition boundary (every 4 interactions), the driver pauses; the experimenter hands the participant the questionnaire for that condition, collects after ~60 seconds, resumes.

7.3 — After the session

- [] CSV/JSON saved under `results/`. Move them to `results/by_participant/P003/` for tidiness.
- [] Write the participant ID on every questionnaire page in pen.
- [] Digitise the Likert scores into `results/by_participant/P003/questionnaire.json` within an hour (memory fades; do it now).

8 — Optional: live face tracking during the session

If you want Pepper to follow the participant's face while they answer:

```
# In a fourth terminal, while the bridge is running:  
curl -X POST http://127.0.0.1:6000/tracker/start -H "Content-Type: application/json" -d '{"target":"Face"}'
```

Pepper's head will now track detected faces. Stop with:

```
curl -X POST http://127.0.0.1:6000/tracker/stop
```

For the actual study, set this up **once per session** at session start, stop at session end. Pepper's head will track even between conversational turns, which feels much more natural to participants than a frozen-staring robot.

9 — Shutdown (end of day)

- [] In Terminal 2, `ctrl+c` to stop the NAOqi bridge cleanly. The bridge sends `motion.rest()` on shutdown so Pepper relaxes its joints.
- [] In Terminal 1, `ctrl+c` to stop the AI server.
- [] If Pepper is going on the shelf overnight, **long-press the chest button** until the shutdown chime sounds. Wait until LEDs are dark before unplugging.
- [] Plug Pepper into its charging dock.

10 — Troubleshooting Cheat Sheet

Symptom	Likely cause	Fix
ALBroker construction failed	Wrong robot IP / port / Pepper asleep	Re-check PEPPER_IP; chest-button wake-up; confirm ping works
All Condition B responses look like GPT-4o-mini	Ollama not running or model not pulled	<code>ollama serve</code> ; <code>ollama list</code>
<code>path: fallback</code> in metadata	Anthropic / Gemini briefly overloaded	Handled by retry; no action needed; the interaction is still logged
Empty <code>speech</code> field	LangGraph state-key mismatch — <code>_merge_state</code> broken	Check server log for graph error message
429 RESOURCE_EXHAUSTED from Google	Free-tier quota exhausted	Non-English already routes to claude-haiku, so harmless; if it persists, set <code>GOOGLE_API_KEY=""</code>
Pepper doesn't move but speaks	You're talking to the virtual robot instead of the real one	<code>curl /ping</code> and check <code>simulation: false</code>
Gesture says behavior not installed on real Pepper	NAOqi animation library missing — extremely rare on real Pepper	Re-flash NAOqi with full animation library, or skip (speech still works)
Whisper transcription is wrong	Participant has a strong accent and <code>model_size="base"</code>	Bump to <code>model_size="small"</code> in <code>whisper_stt.py</code> ; restart server
Latency suddenly jumps to >10s	OpenAI is having an outage	Check status.openai.com ; switch to Condition B (Llama) temporarily
Bridge server prints <code>[WARN] aiohttp ClientTimeout</code>	Network hiccup between bridge and Pepper	Re-run; if persistent, switch Pepper to a wired Ethernet adapter
<code>ImportError: chromadb</code>	The <code>[hri]</code> extra was not installed	<code>pip install -e ".[all]"</code>
<code>omnillm: command not found</code>	venv not activated	<code>.\venv\Scripts\Activate.ps1</code>

Chapter 22 — Building This Project From Scratch — A 13-Week Plan for a Replicator

Suppose you wanted to rebuild OmniLLM from a blank repo. How long would it take, and in what order? This chapter answers exactly that. The plan is calibrated to a single developer with intermediate Python skills working ~10–15 hours/week.

Week-by-Week

Week	Milestone	Files / capabilities added
1	Project skeleton + LLM gateway	pyproject.toml, omnillm/gateway.py, config/models.yaml (3 models), tests/test_gateway.py
2	Smart router	omnillm/router.py, 6 strategies; tests/test_router.py (24 tests)
3	Consensus + evaluator + scorer	omnillm/consensus.py, omnillm/evaluator.py, omnillm/scorer.py; ELO leaderboard CLI
4	CLI dashboard	omnillm/cli.py (click + rich); omnillm models, ask, route, council, evaluate, leaderboard
5	RAG pipeline	omnillm/rag/pipeline.py; ChromaDB integration; keyword fallback; faithfulness scoring
6	HRI primitives	omnillm/hri/classifier.py, omnillm/hri/language_detector.py; tests/test_hri.py
7	Experimental design layer	omnillm/hri/experiment.py; Conditions A–E; ExperimentManager; counterbalancing utilities
8	The agent graph	omnillm/hri/agent_graph.py; 9-node LangGraph pipeline; _merge_state wrapper
9	Robot bridge abstraction	omnillm/robotics/bridge.py, omnillm/robotics/gesture_planner.py, omnillm/robotics/whisper_stt.py
10	Pepper bridge — Python 3 client	omnillm/robotics/pepper.py; aiohttp client; Choregraphe port discovery; stub-mode fallback

11	Pepper bridge — Python 2.7 server	omnilm/server/naoqi_bridge_server.py (stdlib HTTP); omnilm/server/naoqi_client.py (three triggers)
12	AI server	omnilm/server/app.py (Flask); /interact, /transcribe, /evaluate, /status, /health, /export
13	Experimental pipeline + book	scripts/pepper_demo/run_subject_experi omnilm/utils/experiment_logger.py, questionnaire.py, cost_tracker.py, export.py; book/build_pdf.py

Key Sequencing Constraints

- **gateway.py first.** Everything else depends on it.
- **router.py and consensus.py are parallel.** They both consume gateway.py but do not depend on each other.
- **agent_graph.py last among non-server files.** It depends on gateway, router, consensus, RAG, classifier, language_detector, gesture_planner, whisper_stt, experiment, and experiment_logger.
- **The two server/ files are parallel.** app.py depends on agent_graph.py; naoqi_bridge_server.py is Python-2.7-only and depends on no OmniLLM code at all (it only imports NAOqi).
- **Tests are written alongside each module**, not at the end. The 289-test suite is what made the May 2026 refactor possible.

Critical First Mile

Weeks 1–4 produce a **usable LLM benchmarking workbench** by themselves — no robot, no HRI, no RAG. This is the smallest viable slice you should build before adding anything else. Get omnilm ask, omnilm route, omnilm council, omnilm leaderboard working against three real LLM providers, and write 50+ tests around those four commands, before touching anything robotics-related.

When (and How) to Add the Robot

The robot is **Week 9 in the plan**. Earlier than that, you do not have enough infrastructure to make the robot useful. Specifically:

- You need RAG (Week 5) before T1 (Info Retrieval) makes sense.
- You need the task classifier (Week 6) before any conditional routing.
- You need conditions A–E defined (Week 7) before there is anything to compare across.
- You need the agent graph (Week 8) before the robot is more than a one-shot voice toy.

Adding the robot before Week 9 produces a robot that talks to a single LLM in a single mode — interesting, but not a research contribution.

How to Test Without a Robot

For 8 of the 13 weeks, you have no robot. The work is *not* blocked by that:

- All 289 tests run with mocks (no API keys, no robot).
- Scenarios A (text-only) and Scenarios B (virtual robot via Choregraphe) cover the full HRI pipeline without real hardware.
- The bridge's *stub mode* prints actions to stdout — letting you exercise `bridge.execute_action(action)` with zero hardware.

Only **Week 13 (live pilot)** strictly requires real Pepper access. Plan your lab time accordingly.

Part V — Conducting Experiments & Logging

Parts I–IV told you what OmniLLM is, how it works, and how to make it run. Part V is the methodological core of the thesis. It tells you how to run a defensible human-subjects experiment with this stack, how every interaction is recorded, what questionnaire instrument to use and why, how to analyse the resulting data, and what insights the 2026-05-20 pilot already revealed.

Chapter 23 — The Experimental Design

Five conditions, four task types, fifteen participants, one robot, ~30 minutes per session. Six paragraphs of justification for each choice.

At a Glance

The Embodied LLM Arena uses a **within-subjects, fully-crossed, Latin-square-counterbalanced design**. Every participant experiences every one of the five LLM conditions (A–E), and within each condition, the four HRI task types (T1–T4) are presented in a counterbalanced order. The complete factorial — 5 conditions × 4 task types = 20 interactions — fits within a single ~30-minute session per participant, validated by the May 2026 pilot. Three formal hypotheses (H1, H2, H3) defined in Chapter 4 are tested with one primary outcome (mean Likert score per condition) and two secondary outcomes (pairwise preference -> ELO leaderboard; RAG-A vs RAG-off-E within-subject contrast).

The Three Hypotheses (Recap)

#	Hypothesis	Primary Test
H1	Embodied HRI rankings differ from text-only benchmark rankings	Spearman ρ between this study's per-model ELO and Chatbot Arena/MMLU rank; expect $\rho < 0.7$
H2	Smart-routed Condition C outperforms fixed-model Conditions A and B	Pairwise win-rate of C over A and over B in the final preference question; expect $\geq 55\%$ in both
H3	RAG-augmented responses are rated more accurate and trustworthy than RAG-off	Within-subject paired t-test of A.accuracy – E.accuracy; expect mean difference > 0 with $p < 0.05$

Why Within-Subjects?

A within-subjects (repeated-measures) design has two strong advantages over between-subjects for this kind of HRI study:

1. **Statistical power.** Each participant serves as their own control. The variance attributable to "this participant tends to give higher ratings to robots in general" is removed from the condition comparison. A repeated-measures ANOVA on $N=15$ has more power than a between-subjects ANOVA on $N=30$ for the same effect size.
2. **Comparability of pairwise preferences.** The final questionnaire item asks the participant which Pepper-personality they preferred. This question is meaningful *only* in a within-subjects design — you cannot ask someone to compare two robots if they only met one.

The cost of within-subjects: **order effects** (early conditions tend to be rated more leniently; later conditions are affected by participant fatigue). We address this via counterbalancing in Chapter 25.

Why Five Conditions, Not More?

Each condition isolates one experimental factor:

Condition	What it tests vs. its neighbour
A vs B	Fixed cloud LLM vs fixed local LLM (cost-vs-quality trade-off)
A vs C	Single fixed model vs dynamic smart routing (H2)
A vs D	Single model vs 3-model consensus (ensemble effect)
A vs E	RAG enabled vs RAG disabled (H3)

Adding a sixth condition (e.g. "Cloud-LLM with chain-of-thought prompting" or "Local-LLM with consensus") would either dilute statistical power (more conditions -> fewer trials per condition for a fixed session length) or expand session length beyond 30 minutes (participant fatigue -> noisier ratings). Five is a defensible Pareto point.

Why Four Task Types?

The four task types (T1 Info Retrieval, T2 Navigation, T3 Social, T4 Multilingual) are taken from the standard HRI dialogue-act taxonomy (Hsieh et al. 2017; Gross & Krenn 2023). They span the full conversational range a campus-deployed social robot will encounter:

- T1 tests **factual grounding** — does RAG help, and which LLM grounds best?
- T2 tests **embodiment** — does the physical pointing gesture change the perceived helpfulness?
- T3 tests **social naturalness** — the open-ended chat where embodiment effects are strongest in HRI literature.
- T4 tests **multilingual capability** — important for the Genoa context where many visitors are non-Italian researchers.

Four is also the highest number that fits into a 30-minute session at one minute per interaction times five conditions plus questionnaire time between conditions.

Participant Recruitment

Decision	Choice	Reason
N	15	Power analysis: 5-cell repeated-measures ANOVA with $d=0.5$, $\alpha=0.05$, power=0.80 requires $N \geq 11$; over-provisioned to 15 to allow two drop-outs and one Whisper-misfire-heavy run.
Population	UniGE students, postdocs, staff; any age 18+; English-speaking	DIBRIS is on campus; recruitment via the department mailing list.
Inclusion	Self-reported comfortable with English; no hearing impairment	Whisper STT is English-tuned base model; participants must hear Pepper's TTS.
Exclusion	Prior in-depth knowledge of OmniLLM	Avoids "expert participant" bias. Members of Prof. Sgorbissa's HRI lab who have helped design the system are excluded.
Compensation	Small token (€10 voucher)	Standard at UniGE for under-30-min HRI sessions.
Schedule	~one-month recruitment window; ~one participant per lab day	Prevents fatigue on the experimenter; lets us iterate on the questionnaire wording between participants if needed.

What Each Session Looks Like

```
T+0:00 Participant arrives, greeted, seated.
T+0:01 Consent + GDPR forms reviewed and signed (~4 min).
T+0:05 Demographic mini-questionnaire (age band, native language,
prior robot exposure) – 3 items, ~30s.
T+0:06 Pepper introduces itself (a single pre-scripted greeting,
not part of any condition).
T+0:07 Condition 1 (4 interactions, T1->T4 in counterbalanced order).
T+0:11 Questionnaire 1 for Condition 1 (60-90s).
T+0:13 Condition 2.
T+0:17 Questionnaire 2.
T+0:19 Condition 3. + (between-blocks pause if participant wants water)
T+0:23 Questionnaire 3.
T+0:25 Condition 4.
T+0:29 Questionnaire 4.
T+0:31 Condition 5.
T+0:35 Questionnaire 5.
T+0:37 Pairwise preference question
      ("Which of the five Peppers did you prefer?
       Which did you trust most?
       Which felt most natural?")
T+0:40 Open-ended exit comments (recorded).
T+0:42 Compensation, thanks, end.
```

Total: ~42 minutes. Allowing for setup time and unexpected delays, each participant slot is booked at 60 minutes.

Chapter 24 — Running a Single Subject Session

The operational checklist. Read it through once. Print Chapter 21 (Day Zero) and bring both to the lab.

Before the Participant Arrives

- [] AI server and NAOqi bridge are running (Day Zero, steps 3 and 4).
- [] Pilot interaction (step 5) green: Pepper speaks, gestures, eye-LED changes.
- [] Paper questionnaires (one set of 5 condition-questionnaires + final pairwise sheet) laid out.
- [] Pen.
- [] Pepper is on its dock or freshly woken; head-touch sensors confirmed responsive (touch each one in turn while watching `/sensors GET` output).
- [] (If using face-tracking) `POST /tracker/start` already fired.

During the Session

[] Run the driver: `powershell .\venv\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py --participant P003 --server http://127.0.0.1:5000 --bridge http://127.0.0.1:6000`

[] **Between condition blocks**, the driver pauses for ~10 seconds. At each pause:

- Hand the participant the next questionnaire for the condition they just experienced.
- Move out of Pepper's eyeline so you don't influence their answers.
- When the participant hands the questionnaire back, you press **Enter** in the driver terminal to continue.
- [] **Log live observations** on the observer rating sheet for each interaction:
- Did the gesture sync with the speech (1–5)?
- Did the participant understand the first time, or repeat?
- Any unusual reactions (laughter, surprise, confusion)?

[] At the end, ask the **pairwise preference question** verbally:

"Of the five Peppers you just talked to, which one did you prefer? Which one felt most natural? Which one would you most trust to give you correct information?"

- Note the three answers on the pairwise sheet.

After the Session

- [] Move the CSV/JSON output to `results/by_participant/P003/`.
- [] Within the hour: enter the Likert scores from paper into the digital collector: `python from omnillm.utils.questionnaire import InteractionQuestionnaire, QuestionnaireCollector c = QuestionnaireCollector() c.add_interaction_response(InteractionQuestionnaire(`

```
session_id="<from CSV>", participant_id="P003", condition="A", accuracy=6,  
naturalness=5, trust=6, gesture_appropriateness=5, response_speed=7, free_text="...")  
# ...repeat for B, C, D, E... c.save("results/by_participant/P003/questionnaire.json")
```

- [] File the paper originals in the binder. Tag them with the participant ID.
- [] Check `/export` to verify all 20 interactions were logged server-side.

Watch-outs

- **Whisper misfires.** If Whisper transcribes a participant's question incorrectly and the LLM answers the wrong thing, note this as a `task_completed=False` observer rating. The driver does not retry automatically; you can ask the participant to repeat, but log the original misfire.
- **Pepper drops connection.** Rare but happens (Wi-Fi flake, NAOqi crash). The driver's per-step try/except will print `ERROR: server unreachable`; you have ~20s to restart the bridge before the participant loses immersion. If this happens twice in one session, abort the session and reschedule.
- **Participant goes off-script.** Some participants will try jailbreaking or testing edge cases ("Pepper, are you human?"). This is fine — log it. The data is more interesting *because* of these moments, not less.

Chapter 25 — Counterbalancing and Why It Matters

The Problem

In a within-subjects design where every participant sees every condition, the **order** of presentation matters. The first condition gets attention and goodwill; the fifth condition gets fatigue. The first task type in each condition gets the cleanest cognitive engagement; the fourth gets habituation. Without counterbalancing, the first-presented condition would systematically receive higher ratings, and that effect would be indistinguishable from a true condition effect.

The Solution: Two Latin Squares

We counterbalance at two levels:

Level 1 — Condition Order Across Participants (5×5 Latin Square)

The 15 participants are assigned to one of three "blocks" of five-row Latin squares. Each row defines which condition each participant sees first, second, third, fourth, fifth.

Block	Participant	Pos 1	Pos 2	Pos 3	Pos 4	Pos 5
1	P001	A	B	C	D	E
1	P002	B	C	D	E	A
1	P003	C	D	E	A	B
1	P004	D	E	A	B	C
1	P005	E	A	B	C	D
2	P006	A	C	E	B	D
2	P007	B	D	A	C	E
2	P008	C	E	B	D	A
2	P009	D	A	C	E	B
2	P010	E	B	D	A	C
3	P011	A	D	B	E	C
3	P012	B	E	C	A	D
3	P013	C	A	D	B	E
3	P014	D	B	E	C	A
3	P015	E	C	A	D	B

After all 15 participants, each condition appears in each position exactly 3 times — perfectly balanced for order effects.

Level 2 — Task Type Order Within Each Condition

Within each condition block (4 interactions), the four task types are presented in a Latin-square-rotated order that the driver script hard-codes:

```
ORDER = [  
    ("A", "T1"), ("A", "T2"), ("A", "T3"), ("A", "T4"),  
    ("B", "T2"), ("B", "T3"), ("B", "T4"), ("B", "T1"),  
    ("C", "T3"), ("C", "T4"), ("C", "T1"), ("C", "T2"),  
    ("D", "T4"), ("D", "T1"), ("D", "T2"), ("D", "T3"),  
    ("E", "T1"), ("E", "T2"), ("E", "T3"), ("E", "T4"),  
]
```

Each task type appears in each position (1st, 2nd, 3rd, 4th within a condition block) exactly once or twice. Over the full study, task-position effects are mostly cancelled.

A Tighter Counterbalancing Variant

If a future replication wants stronger guarantees, the recommended upgrade is to randomise both the condition order *and* the task order **per participant** using `numpy.random.default_rng(seed=hash(participant_id))`. This produces a fully randomised design with reproducible seed-from-ID. The trade-off is that each participant's sequence becomes harder to print on the experimenter's clipboard in advance.

The `ExperimentManager.create_session()` Path

`omnilm/hri/experiment.py` includes a higher-level `ExperimentManager.create_session(participant_id, conditions=None)` API that supports per-participant assignment via Latin square. For the N=15 study, the simpler hard-coded `ORDER` is sufficient. For larger replications, switch to `ExperimentManager`:

```
from omnilm.hri.experiment import ExperimentManager  
mgr = ExperimentManager()  
session = mgr.create_session(participant_id="P003")  
print(session.conditions_assigned) # ["C", "D", "E", "A", "B"] per the Latin square
```

Document whichever path you use in the methods section of your paper.

Chapter 26 — The Data Logging Pipeline — From Microphone to CSV

The Three Logs

Every session produces three artefact streams:

Artefact	Written by	Path	Contents
Interaction log (canonical)	Agent graph node 7 (log_interaction) on every interaction	results/interactions_ <date> _ row_id (server-side) + results/subject_run_ <participant> _ <ts> . (driver)	one row per interaction; all 20 participants . <code>{.json, csv}</code> <code>InteractionRecord</code>
Questionnaire data	QuestionnaireCollector (post-session)	results/by_participant/ participant_id /questionnaire.js	All our questionnaire types for that participant
Observer notes	The experimenter, free-form	results/by_participant/ participant_id /markdownnotes/ observer .md	Markdown notes, not machine-parsed but archived

Concrete Example — One Interaction in Three Views

A single navigation question in Condition A produces:

Pepper's log line (Choregraphe console):

```
ALAnimatedSpeech: "The Pepper room is located at the end of the ground-floor corridor..."
ALMotion: runBehavior(animations/Stand/Gestures/Right_1)
ALLeds: fadeRGB(FaceLeds, 0x00AAFF, 0.3s)
```

AI server's JSON-Lines log entry:

```
{
  "session_id": "578b023b-09af-4da3-a047-e742e25c5913",
  "participant_id": "P003",
  "condition": "A",
  "task_type": "navigation",
  "utterance": "Where is the Pepper room at DIBRIS?",
  "response": "The Pepper room is located at the end of the ground-floor corridor, on the right, in the HRI lab. Lo",
  "model_id": "openai-gpt4o-mini",
  "latency_ms": 1592.4,
  "input_tokens": 412,
  "output_tokens": 41,
  "cost_usd": 0.0000865,
  "rag_enabled": true,
  "rag_faithfulness": 0.92,
  "rag_chunk_count": 3,
  "judge_score": -1.0,
  "language": "en",
  "gesture_used": "point_right",
  "task_success": null,
  "timestamp": "2026-06-12T14:23:51.842Z",
  "notes": ""
}
```

Driver's CSV row (one column per field):

```
2,A,T2_navigation,openai-gpt4o-mini,graph,y,1.59,"Where is the Pepper room at DIBRIS?","The Pepper room is located
```

The three views are not redundant; each captures different metadata. The CSV is the analysis input; the JSON-Lines is the canonical record; the Choregraphe console is the live-monitoring view for the experimenter.

Data Hygiene Rules

1. **No raw audio is stored.** Whisper transcribes; the transcribed text is logged; the audio bytes are discarded. This is a GDPR-and-courtesy choice: participants' voices do not persist on disk.
2. **No personally identifying information.** Logs contain only the participant ID (P001, P002, ...). The code-to-identity mapping lives on paper in the experimenter's binder and is destroyed at study end.
3. **No deletion mid-study.** The JSON-Lines log is append-only. If a row needs correcting (e.g. observer realised the participant misheard the question), add a *correction* row referencing the original row's timestamp. Never overwrite.
4. **Backups within 24 hours.** Each evening, copy `results/` to an external drive and to the cloud (encrypted). Hard-drive failures are non-zero probability events.

Chapter 27 — Questionnaires: Likert, Godspeed, Pairwise, Observer

The Five-Item Interaction Likert (Primary Outcome)

After each condition block (= 4 interactions), the participant rates that condition on five 1–7 Likert items. The exact paper template:

CONDITION ____ PARTICIPANT P____

Please rate this Pepper personality:

Strongly disagree 1 2 3 4 5 6 7 Strongly agree

1. The robot's answers were accurate.
[][][][][][][]

2. The robot was natural to talk to.
[][][][][][][]

3. I trust the information the robot gave me.
[][][][][][][]

4. The robot's gestures were appropriate.
[][][][][][][]

5. The robot responded quickly enough.
[][][][][][][]

Anything else you'd like to say about this Pepper?

These five items map directly onto the five fields of `InteractionQuestionnaire`. The mean of the five items is the per-condition "overall score" reported in the leaderboard. (For statistical purposes we will analyse the items independently as well, but the mean is the primary user-facing number.)

Godspeed Subscales (Secondary Outcome, Optional)

The **Godspeed questionnaire** (Bartneck et al. 2009) is the most-cited HRI evaluation instrument. It has five subscales, each composed of several bipolar adjective pairs rated 1–5:

Subscale	Adjective pairs
Anthropomorphism	Fake↔Natural, Machine-like↔Human-like, Unconscious↔Conscious, Artificial↔Lifelike, Moving rigidly↔Moving elegantly
Animacy	Dead↔Alive, Stagnant↔Lively, Mechanical↔Organic, Artificial↔Lifelike, Inert↔Interactive, Apathetic↔Responsive

Likeability	Dislike↔Like, Unfriendly↔Friendly, Unkind↔Kind, Unpleasant↔Pleasant, Awful↔Nice
Perceived Intelligence	Incompetent↔Competent, Ignorant↔Knowledgeable, Irresponsible↔Responsible, Unintelligent↔Intelligent, Foolish↔Sensible
Perceived Safety	Anxious↔Relaxed, Agitated↔Calm, Quiescent↔Surprised

Mean each subscale -> enter into `GodspeedResponse.anthropomorphism`, etc.

Recommendation for this study: include Godspeed **only once per participant**, at the *end* of the session, asking them to rate Pepper overall (not per-condition). Per-condition Godspeed would add ~5 minutes per condition × 5 conditions = 25 minutes, blowing the session budget. End-of-session Godspeed lets us compare *our Pepper deployment* against published Godspeed scores from prior Pepper studies (Mishra et al. 2024; Soraa et al. 2021; Ye & Robert 2023), which is useful contextualisation.

The Pairwise Preference Question (Feeds ELO)

After all five conditions, ask three preference questions verbally:

1. "Which of the five Peppers did you prefer overall?" -> maps to category `overall_preference`
2. "Which one felt most natural to talk to?" -> maps to `naturalness_preference`
3. "Which one would you most trust to give correct information?" -> maps to `trust_preference`

For each answer, generate `PairwisePreference` records — one record per pairwise comparison the preference implies. If P003 says "I preferred Condition C", that's 4 pairwise wins for C (over A, B, D, E). These feed `EloScorer.update_pairwise(...)` which produces the *embodied_hri* leaderboard.

The triple-question structure is more diagnostic than a single "preferred overall" question — it lets us see *why* a condition was preferred (because it was more natural? more trustworthy? both?).

Observer Ratings (Live)

The experimenter rates each interaction on the spot:

+-----+			
INTERACTION #	COND	TASK	P
+-----+			
Gesture-speech synchrony	1 [] 2 [] 3 []		
	4 [] 5 []		
Task completed (Y/N)	[Y] / [N]		
Breakdowns (count)	_____		
Notes:	_____		
+-----+			

These rows feed `observerRating` records. The `task_completed` field is important: it's the only way to know whether a Whisper misfire or LLM hallucination spoiled the interaction.

Why Paper, Not Tablet?

Discussed in Chapter 15.2. Three reasons:

1. **Familiar to participants.** No new UI to learn during a 30-minute session.
2. **No accidental editing.** Paper is immutable.
3. **Ethics-friendly.** No second data path; the paper is the original record.

Paper-to-digital conversion happens within the hour, by the experimenter, using the `QuestionnaireCollector` Python API.

Chapter 28 — Ethics, Consent, GDPR, and the UniGE Process

What You Must Have Before Recruiting

Document	Purpose	Where to get it
Approved study protocol	Comitato Etico di Ateneo approval	Submit through the UniGE ethics committee portal; ~4–8 weeks turnaround
Informed consent form (signed by participant)	Participant agrees to take part	Template in Appendix D of this book (forthcoming)
GDPR notice (signed receipt)	Lists what data is collected, retention period, deletion rights	Template in Appendix D
Data Protection Impact Assessment (DPIA)	UniGE's data-protection office must approve any new personal-data workflow	DPIA form on UniGE intranet
Insurance / liability coverage	Standard at-DIBRIS lab insurance	Confirm with Prof. Sgorbissa

The Consent Form Essentials

The consent form must explicitly tell the participant:

1. **What the robot is.** "Pepper, a humanoid robot." Not "a smart device" or "an AI". Honesty matters.
2. **What will be asked of them.** "Five short conversations with the robot, ~30 minutes total."
3. **What data is recorded.** "Text transcripts of your questions. Pepper's text answers. Timestamps. *No audio recordings. No video. No name.*"
4. **What the data is used for.** "A Master's thesis on multi-LLM social robots. May appear (in fully anonymised form) in a future peer-reviewed paper."
5. **How long the data is kept.** "Anonymised data: 5 years (then destroyed). The code-to-identity mapping: destroyed at study end (~6 months after your session)."
6. **Right to withdraw.** "You can stop at any time during the session and ask us to delete your data."
7. **Contact info.** Experimenter's name + email; Prof. Sgorbissa's name + email; UniGE ethics office contact.

The GDPR Specifics

OmniLLM at the lab is a **data controller** (you, the experimenter) collecting **non-special-category personal data** (the participant's spoken words transcribed) for **scientific research** purposes. Under GDPR Article 89, this is a permissible purpose with appropriate safeguards. The safeguards we put in place:

1. **Pseudonymisation at source.** Participant ID (P001, ...) is used everywhere; the link to real identity exists only on paper.
2. **No audio retained.** Whisper transcribes in-memory; the WAV is discarded immediately after.
3. **No cloud LLM sees identifying information.** The LLM only sees the question text. We do *not* tell the LLM "User P003 asks: ...". The LLM call carries no participant identifier.
4. **Destruction schedule.** Six months after study end, the experimenter destroys the paper code-mapping. The anonymised digital data persists for five years, then deleted.

A Note on Cloud LLM Data Handling

Several cloud LLM providers (OpenAI, Anthropic, Google) state in their terms that they *may* retain prompts for safety / abuse-monitoring purposes for ~30 days. This means the participant's question text *might* be stored on a US (or EU) cloud provider's servers for that period.

For our study this is acceptable because:

1. The question text contains **no personal information about the participant** — they are answering scripted T1/T2/T3/T4 prompts, not divulging personal details.
2. Participants are informed in the consent form that "your questions may be processed by external AI providers (OpenAI, Anthropic, Google) on their servers."
3. We use providers' **enterprise / no-training tiers** where available (these contractually exclude the data from training the provider's models).

For a future deployment with sensitive participant categories (children, medical patients), the recommendation would be to switch to **fully local LLMs only** (Condition B — Ollama — for all conditions) to eliminate the cloud-data-flow concern entirely.

Sample Texts

The full consent form, GDPR notice, and debriefing-after-session script are in Appendix D of this book (forthcoming with the final thesis submission).

Chapter 29 — Insights From the 2026-05-20 Pilot

Two pilot runs, both with the author as participant P000 on Choregraphe's virtual robot. The point of the pilot was not to collect data for analysis — there were no human ratings — but to debug the system end-to-end. The pilot found three bugs and validated two design decisions.

Pilot 1 — 2026-05-20 08:11:22 UTC

- File: `results/subject_run_P000_20260520T081122Z.{json,csv}`
- 20 interactions, all on Choregraphe's virtual robot.
- **What the run revealed:**

Bug #1 — Condition B silently ran GPT-4o-mini. Every Condition B interaction logged `model_id="openai-gpt4o-mini"` instead of `llama3-8b-local`. The RAG pipeline was using its construction-time `self.model_id` instead of the per-condition model. **Fix:** added per-call `model_id=` parameter to `RAGPipeline.query()`; modified `agent_graph.py` to pass it; modified `app.py` to resolve `Cond->model` before invoking the graph. (See Chapters 11.5 and 12.1.)

Bug #2 — Multilingual node fell to fallback. The T4 (Italian Brignole question) node returned with `path: fallback` and the response was in English ("The Brignole train station is located in Genoa..."). Root cause: `LanguageDetector.detect()` was returning a result object without `.language_name` or `.recommended_model`, causing an `AttributeError` inside the multilingual node, which dropped to the catch-all and produced an English answer. **Fix:** added the three compatibility properties (`language_code`, `language_name`, `recommended_model`) to `LanguageDetectionResult`. (See Chapter 12.3.)

Bug #3 — Gemini Flash hit Google free-tier quota. Around interaction 17 (Condition D, which uses Gemini Flash in its 3-model council), Gemini returned `429 RESOURCE_EXHAUSTED`. The council degraded gracefully to a 2-model synthesis. **Fix:** moved T4 from `gemini-flash` to `claude-haiku` in `_LANGUAGE_MODEL_MAP`. Council still includes Gemini Flash because the council's two surviving models provide enough redundancy; if Gemini Flash fails, two-model consensus is acceptable.

Pilot 2 — 2026-05-20 08:18:22 UTC (after fixes)

- File: `results/subject_run_P000_20260520T081822Z.{json,csv}`
- Same 20 interactions; identical prompts.

What the run validated:

[OK] Condition B routes correctly: every B interaction logged `model_id="llama3-8b-local"`.

- [OK] T4 Italian prompt routed correctly: Pepper replied in Italian using `claude-haiku`. Latency 1.98s — within budget.
- [OK] Condition D council: every D interaction logged `model_id="council:openai-gpt4o-mini+claude-haiku+gemini-flash"`; 3-model synthesis worked.
- [OK] End-to-end latencies (mean across 20 interactions):
 - A: 1.21s
 - B: 2.84s (slower — Llama on CPU)
 - C: 1.56s

- D: 3.14s (3× parallel calls + synthesis)
- E: 1.18s (no RAG; fastest)
- [OK] RAG faithfulness scores (mean across A, B, C, D — i.e. RAG-enabled conditions):
- A: 0.92
- B: 0.78 (Llama's grounding is weaker)
- C: 0.89
- D: 0.94 (best — consensus reduces hallucination)
- E: n/a (RAG-off)

What the Pilot Did Not Test

Three things the pilot deliberately could not test, because there was no human participant:

1. **Likert ratings.** No questionnaire data was collected. The first real Likert data comes from P001 onwards.
2. **Pairwise preference.** No preference judgments; no ELO updates from the pilot.
3. **Gesture quality.** The virtual robot's gestures all reported "behavior not installed". Real-Pepper gesture quality is unknown until Day Zero at the lab.

Lessons Carried Forward Into the Real Study

Pilot finding	Action for real study
Latency budget OK for A/B/C/E	No mitigation needed.
Condition D averages 3.1s	Pre-warn participants: <i>"This personality takes a little longer to think."</i>
Condition B's RAG faithfulness 0.78 < others	Document in the methods section. This is a real finding — local models ground less faithfully than cloud models.
Gemini free-tier quota	Set <code>GOOGLE_API_KEY=""</code> if persistent issues; council degrades to 2-model.
T4 routing fragile to provider failure	The added retry-with-backup-model logic in the multilingual node now safety-nets this.

Chapter 30 — Data Analysis with pandas and Jupyter

A beginner-friendly workflow for getting from raw CSVs to the leaderboard tables that will appear in the thesis.

The Three Files Per Participant

After all 15 participants are run, you have (per participant Pxxx):

- `results/by_participant/Pxxx/subject_run_<ts>.json` (raw interaction log, 20 rows)
- `results/by_participant/Pxxx/subject_run_<ts>.csv` (same data, flat)
- `results/by_participant/Pxxx/questionnaire.json` (Likert + Godspeed + pairwise)

Plus the global `results/interactions_<date>.jsonl` (server-side append-only log of every interaction across all participants).

Loading Everything Into pandas

Create a Jupyter notebook at `notebooks/analysis.ipynb` and start with:

```
import pandas as pd
import json
from pathlib import Path

# Load all interaction CSVs from all participants
participants = ["P001", "P002", "P003", "P004", "P005", "P006", "P007", "P008",
               "P009", "P010", "P011", "P012", "P013", "P014", "P015"]

dfs = []
for p in participants:
    base = Path(f"results/by_participant/{p}")
    csv_path = next(base.glob("subject_run_*.csv"))
    df = pd.read_csv(csv_path)
    df["participant"] = p
    dfs.append(df)
interactions = pd.concat(dfs, ignore_index=True)

# Load all questionnaire JSONs
likert_rows = []
for p in participants:
    qpath = Path(f"results/by_participant/{p}/questionnaire.json")
    qdata = json.loads(qpath.read_text())
    for q in qdata["interaction_responses"]:
        likert_rows.append({"q": q, "participant": p})
likert = pd.DataFrame(likert_rows)
```

Per-Condition Summary Tables

```
# Mean Likert per condition (the headline table)
print(likert.groupby("condition")[
    ["accuracy", "naturalness", "trust", "gesture_appropriateness", "response_speed"]
].mean().round(2))
```

Expected output shape:

	accuracy	naturalness	trust	gesture_appropriateness	response_speed
condition					
A	5.93	5.27	5.80	5.13	5.93
B	4.87	4.93	4.60	4.93	3.80
C	6.13	5.93	6.20	5.40	5.67
D	5.67	5.13	6.00	5.27	4.27
E	4.60	5.20	4.20	5.07	5.87

(Exact numbers will of course depend on the actual study.)

Statistical Tests

```
from scipy import stats

# H3 – RAG vs no-RAG paired t-test (A vs E within each participant)
pivot = likert.pivot_table(
    index="participant", columns="condition", values="accuracy")
t, p_val = stats.ttest_rel(pivot["A"], pivot["E"])
print(f"H3 paired t-test: t={t:.3f}, p={p_val:.4f}")

# H2 – Condition C vs A and B
print("H2:")
t, p = stats.ttest_rel(pivot["C"], pivot["A"])
print(f" C vs A: t={t:.3f}, p={p:.4f}")
t, p = stats.ttest_rel(pivot["C"], pivot["B"])
print(f" C vs B: t={t:.3f}, p={p:.4f}")

# Repeated-measures ANOVA across all five conditions
from statsmodels.stats.anova import AnovaRM
rm = AnovaRM(likert, depvar="accuracy", subject="participant",
             within=["condition"]).fit()
print(rm.summary())
```

Building the Embodied LLM Leaderboard

```
# Load pairwise preferences and build ELO
from omnillm.scorer import EloScorer

scorer = EloScorer()
for p in participants:
    qpath = Path(f"results/by_participant/{p}/questionnaire.json")
    qdata = json.loads(qpath.read_text())
    for pref in qdata["pairwise_preferences"]:
        scorer.update_pairwise(
            model_a=pref["condition_a"],
            model_b=pref["condition_b"],
            winner=pref["preferred"],
            category="embodied_hri",
        )

print(scorer.leaderboard(category="embodied_hri"))
```

Latency, Cost, and Faithfulness

```
# Mean latency by condition × task type
print(interactions.groupby(["condition", "task"])["latency_s"].mean().unstack())

# Total cost by condition (over all 15 participants × 4 tasks)
print(interactions.groupby("condition")["cost_usd"].sum().round(4))

# RAG faithfulness by condition (only A, B, C, D – E has no RAG)
faithful = interactions[interactions["condition"] != "E"]
print(faithful.groupby("condition")["rag_faithfulness"].mean().round(3))
```

Visualisations

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")

# Likert means with 95% CI per condition × item
fig, ax = plt.subplots(figsize=(10, 5))
sns.barplot(
    data=likert.melt(
        id_vars=["participant", "condition"],
        value_vars=["accuracy", "naturalness", "trust",
                    "gesture_appropriateness", "response_speed"],
        var_name="item", value_name="score"),
    x="item", y="score", hue="condition", ax=ax,
    errorbar=("ci", 95))
ax.set_ylim(1, 7)
ax.set_title("Mean Likert per condition × item (95% CI)")
plt.tight_layout()
plt.savefig("results/figs/likert_by_condition.png", dpi=150)
```

What to Put in the Thesis

For the thesis methods/results chapters:

1. **A descriptive table** (mean ± SD per condition × Likert item)
2. **The repeated-measures ANOVA** result for the omnibus condition effect
3. **The three hypothesis tests** (H1, H2, H3) with their effect sizes and p-values
4. **The embodied_hri ELO leaderboard** with 95% bootstrap CIs around each rating
5. **Per-task-type analysis** showing how the condition effect varies across T1–T4
6. **A latency-cost-quality figure** showing where each condition sits in 3-space
7. **Qualitative analysis** of the open-ended Likert comments and exit-interview transcripts

A One-Page Reproducibility Checklist for the Reader

Anyone replicating this work needs:

- This repository at the May 2026 commit hash (see git log)
- A real or virtual Pepper
- 15 participants matching the inclusion criteria
- API keys for the listed providers (or Ollama for purely local replication)
- The Latin-square assignment table from Chapter 25
- ~30 hours total experimenter time (10h setup + 15h sessions + 5h analysis)

The repository includes a `notebooks/analysis.ipynb` template that loads the canonical results/ layout and produces all the tables/figures listed above out of the box.

Part VI — Future Scope & Improvements

Honest assessment first, then concrete next steps. This Part is short because the project is meant to ship and be used, not be eternally redesigned.

Chapter 31 — Current Limitations — An Honest Assessment

At a Glance

OmniLLM as of May 2026 is a complete, working, peer-reviewable system for benchmarking LLMs through embodied HRI with a single robot, single lab, English-and-Italian, and ~15 participants. It is **not** a generic platform for arbitrary social-robot deployment. This chapter catalogues the limitations — hardware, software, methodological, and scope — that a careful reader (especially a thesis examiner) will want to see acknowledged.

Hardware Limitations

Limitation	Why it matters	Workaround
Pepper cannot walk. Wheels, not legs. Stairs are fatal.	T2 (Navigation) tasks must remain <i>informational</i> ("Room 305 is on the third floor") rather than <i>autonomous</i> ("let me take you there").	Acceptable for our study; out of scope. Future work could integrate a wheelchair-class mobile base.
Pepper's microphones are noisy beyond ~2 metres in open-plan space.	Whisper transcription quality drops; VAD-triggered conversation flakes.	Use the touch trigger; ask participants to speak closer.
Pepper's depth sensor is mediocre in bright conditions.	Bauer et al. (2019) document poor 3D perception.	We do not depend on depth in this study; future vision-based extensions would.
Pepper's on-board CPU is too weak for any modern LLM.	We must use an off-board AI server.	This is the fundamental driver of the two-process design and is unlikely to change for Pepper.
NAOqi 2.5 is the last released SDK version (2017).	No upstream security patches; bug fixes by community only.	Treat the bridge layer as a stable contract; the AI side can be modernised without touching NAOqi.
NAOqi animation library does not include all 100+ animations on every robot.	Some gestures the planner picks may not exist on a given Pepper.	Either pre-install the missing animations via Choregraphe, or filter <code>_TASK_DEFAULT_GESTURES</code> to a verified subset.

Software Limitations

Limitation	Detail
Rule-based task classifier has finite coverage. Out-of-distribution prompts ("can you predict tomorrow's weather?") default to T1 (info_retrieval), then RAG fails (the KB has no weather data), then the LLM hallucinates.	Mitigation: classifier confidence threshold + safety prompts in the LLM system prompt ("if you don't know, say so").
No streaming responses. Pepper waits for the LLM to finish, then speaks the entire response. Conversational naturalness suffers on long answers.	LangGraph supports streaming; we have not wired it through to NAOqi yet. ~1 week of work in Topology 2.
No multi-turn memory across interactions. Each <code>/interact</code> call is independent; Pepper has no recollection of what the participant just asked.	The thesis design treats each interaction as an isolated trial — appropriate for the experimental study, limiting for a long-form deployment. Adding a conversation-history field to <code>HRIGraphState</code> is straightforward.
English-tuned Whisper-base. Italian, French, Spanish transcription is noticeably weaker.	Bump to Whisper-small (<code>model_size="small"</code>) — already configurable; ~3× CPU cost.
LLM-as-judge bias toward its own provider family is documented (Zheng et al. 2023).	The thesis's primary outcome is human Likert, not judge score; judge feeds only the leaderboard. Cross-judge robustness check (Claude-Haiku-judge variant) is in Appendix G.
No image / vision-language modality. Pepper's cameras are not used by any LLM call.	A multimodal extension (e.g. GPT-4o-vision sees the visitor's face -> adjusts greeting tone) is plausible future work but out of current scope.
No safety filtering between LLM output and robot speech. Pepper says whatever the LLM produces.	For an adult-participant, voluntary-consent study with scripted prompts, the risk surface is small. A children's-deployment version <i>must</i> add a safety filter (e.g. OpenAI's Moderation API).

Methodological Limitations

Limitation	Detail
N = 15 is modest. Power analysis says it suffices for the planned within-subjects ANOVA, but generalisability across populations is limited.	The thesis explicitly frames this as a pilot/proof-of-concept; replication at scale ($N \geq 50$) is recommended in the discussion section.
Single lab, single institution. All participants from UniGE / DIBRIS. Cultural and institutional homogeneity.	Multi-site replication needed before claiming external validity. Cf. Carros et al. 2022 (Germany), Mishra et al. 2024 (Norway), Castellano et al. 2022 (Italy) — all single-site.
No long-term study. Each participant interacts once for 30 minutes. Novelty and Hawthorne effects are documented in HRI (Bartneck et al. 2009) and likely inflate ratings.	Future work: deploy in a real lab-reception setting for 4+ weeks (cf. Carros et al. 2022, 3-month care-home study).
The four task prompts are fixed. Every participant gets the same Brignole-station, lab-hours, Pepper-room questions.	Repetition reduces variability but limits the question-space sampled. Future: random selection from a 20-prompt pool per task.
The five conditions are LLM-source-only. We don't vary prompt-engineering, sampling temperature, or speech speed across conditions.	Those are valuable but separate experiments. Sequential studies could systematically vary each.
No control over participant familiarity with LLMs. Some participants will be AI experts; others first-time users.	We collect this as a demographic item; secondary analysis can subset by familiarity.
Cloud-LLM data flow is real. Despite the GDPR-friendly setup, OpenAI/Anthropic/Google servers do see the (anonymous) prompts for ~30 days.	Documented in the consent form; not a methodological flaw, but a transparency obligation.

Scope Limitations

Out of scope by design	Why
Autonomous locomotion / SLAM	Pepper's wheels + lab geometry make this impractical for the experimental study
Group / multi-party dialogue	Adds combinatorial complexity to counterbalancing; one-on-one is the standard HRI baseline
Long-form storytelling or task-completion (e.g. recipe assistance)	Different timescale (minutes), different evaluation metrics
Vision-based emotion recognition	Adds a second modality; orthogonal to the LLM-comparison hypothesis
Fine-tuning any LLM	Defeats the "compare models as-shipped" premise; out of scope; would require an order of magnitude more data
Production deployment / commercial use	The MIT licence permits it; the codebase is research-grade, not hardened-grade

Chapter 32 — Short-Term Improvements (Next 3 Months)

Achievable additions while the experimental study is running. Each one is scoped at ≤ 1 week of work.

1. Streaming Speech

Wire LangGraph's async streaming through to NAOqi. As tokens arrive from the LLM, push partial sentences to `ALAnimatedSpeech.say()`. Pepper starts speaking while the LLM is still generating.

Effort: ~5 days. **Win:** Perceived latency drops from ~1.5s to ~0.4s (time-to-first-spoken-word) on Condition A.

2. Conversation Memory

Add a `conversation_history: list[dict]` field to `HRIGraphState`. Push every (utterance, response) pair onto the history. Prepend the last 3 turns into the LLM's messages list.

Effort: ~2 days. **Win:** Multi-turn coherence ("you mentioned Room 305 earlier..."). Enables follow-up questions, which the current study explicitly forbids.

3. Safety Filter

Wrap every LLM output through `openai.Moderation.create()` (or a local toxicity classifier) before passing to the gesture planner. If the output trips the filter, substitute a polite "I'd rather not answer that" line.

Effort: ~2 days. **Win:** Required for any future children's-deployment variant.

4. Tablet-Based Questionnaire UI

Build a small HTML page that runs on Pepper's chest tablet. After each condition block, Pepper says "Please tap the screen to rate this conversation" and the participant fills the Likert items directly. The page POSTs to `/evaluate`.

Effort: ~5 days. **Win:** Eliminates manual paper-to-digital conversion; faster sessions; richer real-time data.

5. Real-Time Cost Display

Pin a small terminal-window with `omnillm costs --watch` (auto-refreshing every 30 seconds) during sessions. Lets the experimenter spot a runaway model immediately.

Effort: ~1 day. **Win:** Avoid waking up to a \$50 OpenAI bill from a misconfigured loop.

6. Whisper Model Auto-Selection

Detect the participant's preferred language at session start (from the demographic questionnaire) and load the appropriate Whisper model size: base for native English, small for non-native or Romance languages.

Effort: ~2 days. **Win:** Better transcription quality for L2 English speakers.

7. Per-Participant Custom Prompts

Allow the experimenter to swap the four canonical prompts for a participant-specific set (e.g. if the visitor specifically wants directions to a *different* room). Driven by a YAML file: `config/participant_prompts/P003.yaml`.

Effort: ~2 days. **Win:** Richer interactions for participants who want to explore.

Chapter 33 — Medium-Term Research Directions (Next 12 Months)

Studies that extend OmniLLM's contribution. Each is $\geq$ a month of work but builds on the existing infrastructure.

1. Multi-Site Replication

Deploy the same OmniLLM + Pepper stack at 3+ partner labs (e.g. another Italian institution, one German, one Japanese — to capture the cultural variation Soraa et al. 2021 found in children's perceptions). Run the same 5×4 design at each site with N=15 per site. Cross-site comparison answers: *do embodied LLM rankings generalise?*

Effort: ~6 months calendar time, ~30 person-hours of code work (deploying to a new site is mostly configuration).

2. Vision-Language Integration

Wire GPT-4o-vision (or Claude 3.5 Sonnet with vision) into the agent graph. Pepper's chest camera captures a frame at the start of each interaction; the LLM receives both the spoken question *and* a description of the visitor's appearance, body language, expression. Hypothesis: does multimodal grounding improve perceived "Pepper notices me"?

Effort: ~2 months. **Risk:** privacy — image data is more sensitive than text. Needs separate ethics approval.

3. Adaptive Routing

Currently the smart router (Condition C) routes by *task type*. A trained adaptive router could additionally use: previous-turn participant rating, current detected emotion (via Whisper prosody features), per-participant historical preference. The router becomes a small reinforcement-learning agent over time.

Effort: ~3 months. **Risk:** drifts the experimental design from "compare LLMs" toward "compare router policies"; the paper would need to be about the router, not the LLMs.

4. Long-Term Field Study

Deploy OmniLLM-Pepper at a real DIBRIS reception desk for 4–8 weeks. Log every interaction. Measure: (a) novelty decay, (b) staff acceptance, (c) operational reliability, (d) whether real visitors find the LLM-based responses more or less satisfactory than the (prior) keyword chatbot. This is the Carros et al. 2022 model applied to LLMs.

Effort: 4–8 months calendar. Heaviest item; biggest paper.

5. A Children's Variant — With Safety Filters

Re-do the study with school-age children visiting DIBRIS as part of outreach. Requires: stricter safety filter, child-tuned voice, simpler prompts, separate ethics approval. Compares to Soraa et al. 2021 and Pigneddu & Gena 2023.

Effort: ~4 months (ethics is the bottleneck).

6. Open Embodied LLM Leaderboard

After collecting data from multiple sites, publish a continuously-updated *Embodied LLM Arena* leaderboard at a public URL (analogous to Chatbot Arena). Anyone running an OmniLLM-Pepper deployment can POST their pairwise preferences via a documented API; the leaderboard updates in real time. The community grows the dataset.

Effort: ~3 months. **Long-term impact:** highest of any item on this list.

Chapter 34 — Long-Term Vision — Where Embodied LLM Research Is Going

Speculative but grounded. None of these are tomorrow's work; all are within the field's 5-year horizon.

LLMs as a Commodity Substrate for Robotics

In 2026, "which LLM should this robot use?" is still an interesting research question. By 2030, the author expects this question to be answered the way "which database engine should this web app use?" is answered today — by default-good choices (PostgreSQL, GPT-class) plus narrow specialty cases. The contribution of *this* line of research is to establish the comparison methodology *before* the LLM choice becomes commoditised, so future practitioners have a documented evaluation framework.

The Death of "the LLM" as a Single Component

The future of embodied AI is not a single LLM but a **constellation** of specialised models: a small fast model for short turns, a large slow model for hard questions, a vision-language model for perception, a code-trained model for behaviour synthesis, an embedding model for memory, a moderation classifier for safety. OmniLLM's *unified gateway + smart router* architecture is the right shape for this future — but the routing decisions will need to factor in many more dimensions than the six current strategies.

Robot Middleware Will Catch Up — But Slowly

NAOqi will eventually be replaced (probably by ROS 2). When that happens, OmniLLM's `RobotBridge` abstraction makes the transition cheap: write a `Ros2Bridge(RobotBridge)` subclass, leave everything else untouched. The *HTTP-and-JSON between two-Python-versions* pattern is a transitional artefact that will fade — but the *AI server / robot bridge / abstract robot interface* layering will persist.

Embodiment Will Be Measured Differently

Today's embodied evaluation borrows tools (Likert, Godspeed, pairwise) from disembodied chatbot evaluation. The author expects new measurement instruments to emerge:

- **Multimodal engagement metrics** (gaze, prosody, posture mirroring) extracted automatically from video, removing self-report bias.
- **Real-world utility metrics** (did the visitor actually go to the room Pepper pointed to?) — currently impossible because we don't track participants after the session.
- **Long-term relationship metrics** — does the same participant come back to interact with Pepper voluntarily, weeks later?

OmniLLM's logging infrastructure (every interaction, every model, every timestamp) is well-positioned to feed any of these future analyses.

The Reproducibility Bar Will Rise

Chatbot Arena demonstrated that open methodology + open leaderboards + open data accelerates a field. The HRI community is still mostly closed-data; each paper reports its own population, its own robot, its own prompts, with no shared benchmark. OmniLLM is a *step toward* a shared benchmark. The author hopes the next 2–3 years see an "Embodied Arena" emerge as the HRI counterpart to Chatbot Arena — with this thesis among the first contributions.

Appendix A — Python Primer: Just the Parts You Need for OmniLLM

This appendix is for the reader who knows what `print()` does and can write for `x in [1, 2, 3]`, but has not used `async/await`, `dataclasses`, `type hints`, `virtual environments`, or `context managers`. Everything you need to read every line of the OmniLLM source. If you already know modern Python, skip to Appendix B.

A.1 — Virtual Environments (venv)

When you `pip install` a package, it normally goes into your system's Python installation. That's bad for two reasons:

1. **Version conflicts.** Project A needs `flask==1.1.4`. Project B needs `flask==3.0.0`. They can't coexist system-wide.
2. **Permissions.** Some systems require admin/sudo to install system-wide packages.

A **virtual environment** is a self-contained Python installation inside one directory. You activate it; while active, every `pip install` goes into that directory, and every `python` invocation finds packages from there. Deactivate it (or close the terminal), and your system Python is unchanged.

```
python -m venv venv          # creates a "venv" directory
.\venv\Scripts\Activate.ps1  # activate (Windows)
source venv/bin/activate     # activate (mac/Linux)
deactivate                   # leave the venv
```

When the `venv` is active, your prompt typically shows `(venv)` at the front. Every script in this book assumes you have activated the `venv` first.

A.2 — Type Hints

Python is **dynamically typed**: a variable's type is whatever you most recently assigned to it. But since Python 3.5, you can *annotate* types as documentation (and as input to tools like `mypy`):

```
def add(a: int, b: int) -> int:
    return a + b

name: str = "Akshita"
ages: list[int] = [21, 22, 25]
config: dict[str, int] = {"port": 5000, "timeout": 30}
```

Important: **Python does not enforce these annotations at runtime.** They are documentation that an IDE and type checker can verify. You can pass a `str` to `add(a, b)` and Python won't object — it will just blow up when it tries to do `+`.

Modern OmniLLM annotations:

Hint	Means
<code>int, str, bool, float, bytes</code>	The standard primitives
<code>list[int]</code>	A list of ints
<code>dict[str, Any]</code>	Dict from <code>str</code> to anything
<code>Optional[X]</code> or <code>X None</code>	Either an <code>X</code> or <code>None</code>
<code>Callable[[int], str]</code>	A function that takes an <code>int</code> and returns a <code>str</code>
<code>Literal["A", "B"]</code>	One of the specific values listed

Any	"I don't want to commit to a type"
-----	------------------------------------

`from __future__ import annotations` (used in almost every OmniLLM file) is a deferred-evaluation pragma — it means all type hints are stored as strings, so forward references (e.g. `def foo() -> "LLMGateway"`) work without circular-import gymnastics.

A.3 — Dataclasses

A dataclass is the boilerplate-free way to write "a class that mostly just holds some named fields":

```
from dataclasses import dataclass, field

@dataclass
class ModelResponse:
    model_id: str
    content: str
    input_tokens: int = 0
    output_tokens: int = 0
    cost_usd: float = 0.0
    metadata: dict = field(default_factory=dict)
```

The `@dataclass` decorator auto-generates: - `__init__(self, model_id, content, input_tokens=0, ...)` - `__repr__` (a useful string representation) - `__eq__` (equal if all fields are equal)

`field(default_factory=dict)` is the right way to default to a mutable type. Writing `metadata: dict = {}` would mean *all instances share the same dict* — a classic Python gotcha.

OmniLLM uses dataclasses for every record type: `ModelResponse`, `RobotAction`, `RobotSensorData`, `InteractionRecord`, `RouteDecision`, `DocumentChunk`, `RAGResponse`, `LanguageDetectionResult`, `ClassificationResult`, `ConditionConfig`, `ParticipantSession`, `InteractionQuestionnaire`, `GodspeedResponse`, `PairwisePreference`, `ObserverRating`.

A.4 — `async` and `await`

A normal Python function runs to completion before returning. An `async` function returns a *coroutine* — a paused piece of work that you must `await` for it to actually execute.

```
import asyncio

async def fetch(url: str) -> str:
    # ...network I/O here...
    return "response body"

async def main():
    text = await fetch("https://example.com")
    print(text)

asyncio.run(main())
```

The magic happens when you have *many* I/O-bound operations to do in parallel:

```
async def main():
    results = await asyncio.gather(
        fetch("https://example.com/a"),
        fetch("https://example.com/b"),
        fetch("https://example.com/c"),
```

```
)
```

The three fetches run concurrently. Total wall-clock = `max(t_a, t_b, t_c)`, not their sum. **This is exactly how OmniLLM's `query_multiple` runs three LLM calls in parallel for Condition D's council.**

Things that bite you:

- You cannot `await` outside an `async def`.
- You cannot call a regular function with `await`. (You can call an `async def` with regular `()`, but it returns a coroutine object that does nothing until awaited.)
- Don't `time.sleep()` inside an async function — it blocks the whole event loop. Use `await asyncio.sleep()` instead.

A.5 — Context Managers (`with`)

The `with` statement guarantees that cleanup code runs when a block ends, even if an exception is raised inside:

```
with open("data.txt") as fh:
    contents = fh.read()
# file is automatically closed here, even if read() raised
```

OmniLLM uses context managers for file I/O, network sockets (`socket.socket(...)`), and aiohttp sessions (`async with aiohttp.ClientSession(...) as session:`).

You can write your own:

```
from contextlib import contextmanager

@contextmanager
def timer(label: str):
    import time
    t0 = time.time()
    yield
    print(f"{label}: {time.time() - t0:.2f}s")

with timer("RAG retrieval"):
    chunks = rag.retrieve(query)
```

A.6 — `pathlib.Path`

The modern replacement for string-based path manipulation. Instead of `os.path.join(a, b, c)`, you write `Path(a) / b / c`. Cross-platform — `\` vs `/` is handled automatically.

```
from pathlib import Path

repo_root = Path(__file__).parent.parent
kb_dir = repo_root / "knowledge_base"
for txt_file in kb_dir.glob("*.txt"):
    text = txt_file.read_text(encoding="utf-8")
```

OmniLLM uses `pathlib.Path` exclusively. Never `os.path`. Never raw strings for paths.

A.7 — Decorators

A decorator is a function that takes a function and returns a (usually modified) function. The `@name` syntax is shorthand:

```
def loud(fn):
    def wrapped(*args, **kwargs):
        print(f"Calling {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapped

@loud
def add(a, b):
    return a + b

add(2, 3)  # prints "Calling add", then returns 5
```

OmniLLM uses decorators for:

- `@dataclass` — class decoration (above)
- `@app.route("/health", methods=["GET"])` — Flask URL routing
- `@property` — turn a method into a read-only attribute
- `@classmethod`, `@staticmethod` — class-level methods
- `@pytest.fixture` — define a reusable test fixture
- `@abstractmethod` — mark a method as required-by-subclasses on an ABC

A.8 — `*args` and `**kwargs`

`*args` collects positional arguments into a tuple; `**kwargs` collects keyword arguments into a dict:

```
def query(model_id, *args, **kwargs):
    print(model_id)          # the first positional
    print(args)              # any other positionals as a tuple
    print(kwargs)            # any keyword args as a dict
    return litellm.completion(**kwargs)  # unpack the dict back into kwargs

query("openai-gpt4o-mini", temperature=0.7, max_tokens=100)
# model_id = "openai-gpt4o-mini"
# args = ()
# kwargs = {"temperature": 0.7, "max_tokens": 100}
```

The `**kwargs` pattern is heavily used in `gateway.py` to pass provider-specific options through to LiteLLM without enumerating every possible parameter.

A.9 — `Enum` and `Literal`

Two ways to type a value as "one of a fixed set":

```
from enum import Enum

class RoutingStrategy(str, Enum):
    BEST_QUALITY = "BEST_QUALITY"
    LOWEST_COST = "LOWEST_COST"
```

```
# ...

s = RoutingStrategy.BEST_QUALITY
print(s.value) # "BEST_QUALITY"
print(s == "BEST_QUALITY") # True (because it inherits from str)
```

For lighter-weight cases where you only need the type hint:

```
from typing import Literal

Mode = Literal["server", "direct", "stub"]

def use_mode(m: Mode) -> None: ...
```

Mode is not a class; it's just a type alias. `use_mode("server")` works; `use_mode("hello")` would be flagged by `mypy` but still runs.

A.10 — `abc.ABC` and `@abstractmethod`

Abstract base classes declare a contract that subclasses must fulfill:

```
from abc import ABC, abstractmethod

class RobotBridge(ABC):
    @abstractmethod
    async def connect(self) -> bool: ...
    @abstractmethod
    async def execute_action(self, action) -> bool: ...

class PepperBridge(RobotBridge):
    async def connect(self) -> bool:
        # ... real implementation ...
        return True
    async def execute_action(self, action):
        # ... real implementation ...
        return True

# bridge = RobotBridge() # TypeError: cannot instantiate abstract class
bridge = PepperBridge() # OK
```

If you forget to implement an abstract method in a subclass, the error fires at *instance construction time* — not at method-call time. That's the safety net.

A.11 — F-Strings

The modern Python string-formatting syntax:

```
name = "Akshita"
age = 22
print(f"Hello {name}, you are {age} years old.")
print(f"Hex: {255:#04x}") # "Hex: 0xff" — same format specifiers as %-format
print(f"Pi to 3dp: {3.14159:.3f}")
```

You can put any expression inside the `{}`:

```
print(f"Sum: {2+3}")
print(f"Model count: {len(gateway.list_models())}")
```

The `=` flag (Python 3.8+) auto-shows the expression name:

```
x = 42
print(f"{x=}")      # "x=42"
```

A.12 — Reading the Type-Hint-Heavy Source

When you open a file like `gateway.py` and the first non-import line is:

```
async def query(
    self,
    model_id: str,
    messages: list[dict[str, str]],
    temperature: float = 0.7,
    max_tokens: int = 2048,
) -> ModelResponse:
```

That's a single function signature spread across lines for readability. Parse it as:

- `async def` — coroutine function (must be awaited)
- `query` — method name
- `self` — first arg is the instance (it's a method on a class)
- `model_id: str` — second arg is a string
- `messages: list[dict[str, str]]` — third arg is a list of dicts that map string-to-string
- `temperature: float = 0.7` — fourth arg is a float with default `0.7`
- `max_tokens: int = 2048` — fifth arg, default `2048`
- `-> ModelResponse:` — returns an instance of `ModelResponse`

Reading dense type-hinted Python is a skill that comes quickly with practice. The hints help you, even if you're not using a type checker.

Appendix B — Complete Glossary

Every acronym and term used in this book, sorted alphabetically. Use this as a lookup whenever you forget what something means.

aihttp — A Python library for asynchronous HTTP clients and servers, used in `omnillm/robotics/pepper.py` as the non-blocking HTTP client for the bridge server.

Agent graph — In LangGraph terminology, a directed graph of nodes (each a Python function) that share a state dict. OmniLLM's brain is a 9-node agent graph.

ALAnimatedSpeech — NAOqi service that handles text-to-speech with synchronised body animation. Pepper's "talking" subsystem.

ALAudioDevice — NAOqi service that provides access to Pepper's 4 head microphones.

ALAudioRecorder — NAOqi service that captures audio to a WAV file on Pepper's filesystem.

ALBroker — NAOqi's central message-routing service. Every NAOqi process must connect to a broker to talk to other NAOqi services.

ALLeds — NAOqi service that controls Pepper's RGB LEDs (eyes, ears, shoulders).

ALMotion — NAOqi service that handles joint movement and posture.

ALTabletService — NAOqi service that displays content on Pepper's chest-mounted tablet.

ALTracker — NAOqi service that tracks a target (face, sound source, NAO mark) and moves Pepper's head/body to maintain orientation toward it.

ANOVA — Analysis of Variance. Statistical test for differences in means across groups. In a within-subjects design, *repeated-measures ANOVA* is used.

API key — A secret password (typically a long random string) that authenticates a request to a cloud LLM provider. Stored in `.env`; loaded into environment variables; never committed to git.

async def — Python keyword to declare a coroutine function. Must be awaited (or scheduled via `asyncio.gather/asyncio.run`) to actually execute.

Bartneck Godspeed — The Godspeed Questionnaire (Bartneck et al. 2009). Standard HRI evaluation instrument with five subscales: anthropomorphism, animacy, likeability, perceived intelligence, perceived safety.

Base64 — Encoding scheme that represents binary data as ASCII text. Used in OmniLLM to ship WAV audio over JSON HTTP.

bridge_server — Short for `naoqi_bridge_server.py`. The Python-2.7 HTTP server that translates JSON commands from the AI Layer into NAOqi RPCs.

ChromaDB — Embedded vector database used by `RAGPipeline` for semantic-similarity retrieval.

Choregraphe — SoftBank's official IDE for programming Pepper. Includes a virtual-robot simulator usable on Windows.

Consensus / Council — Asking multiple LLMs the same question in parallel and merging their answers. Implemented in `omnillm/consensus.py`. Used in Condition D.

Cosine similarity — Measure of similarity between two vectors. Used by ChromaDB to rank retrieved chunks.

Counterbalancing — Experimental design technique to cancel order effects by varying the presentation order across participants.

DejaVu Sans / DejaVu Sans Mono — Open-source fonts shipped with matplotlib. Used by the book's PDF builder because they cover Unicode arrows, box-drawing characters, and accented Latin.

DIBRIS — Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi. The department at the University of Genoa hosting this thesis's robot.

ELO — Chess-derived rating system. A 100-point gap means the higher-rated player wins ~64% of the time. Used by OmniLLM's leaderboard.

Embedded LLM Arena — The name of the research study supported by OmniLLM. Pepper-mediated multi-LLM benchmark.

Embedding — Numeric vector representation of text. Sentence-transformers produces 384-dimensional embeddings; ChromaDB indexes and searches them.

Environment variable — OS-level key-value pair. Used for API keys (`OPENAI_API_KEY`, etc.). Read in Python via `os.environ.get(...)`.

Faithfulness (RAG) — How accurately an LLM's answer reflects the retrieved context. Scored 0–1 by an LLM-as-judge prompt.

Flask — Python web framework. OmniLLM's AI server uses Flask for `/interact`, `/health`, etc.

G-Eval — Referenceless LLM-as-judge methodology by Liu et al. 2023. Uses chain-of-thought prompting + form-filling to score open-ended LLM outputs.

Gateway — In OmniLLM, the unified entry point for any LLM call: `omnilm/gateway.py`.

GDPR — General Data Protection Regulation. EU privacy law governing personal-data handling. Drives several design choices in OmniLLM's logging.

Gesture planner — `omnilm/robotics/gesture_planner.py`. Rule-based mapping from `(task_type, response_text)` to `(gesture, LED colour)`.

Godspeed — See *Bartneck Godspeed*.

HRI — Human-Robot Interaction. The academic field this thesis lives in.

HRIGraphState — The dataclass that defines the shared state of the agent graph. Lives in `omnilm/hri/agent_graph.py`.

HTTP — HyperText Transfer Protocol. The protocol used by every cross-language boundary in OmniLLM.

Hallucination — When an LLM generates plausible-sounding but factually incorrect content. OmniLLM's RAG pipeline flags suspected hallucinations via a coverage heuristic.

JSON — JavaScript Object Notation. The data format used at every HTTP boundary in OmniLLM.

JSON-Lines (JSONL) — One JSON object per line. The format of `results/interactions_<date>.jsonl`.

LangChain — Python framework for LLM application development. OmniLLM uses only its document-loader and text-splitter components.

LangGraph — Python library by the LangChain team for building stateful, branching agent pipelines. OmniLLM's agent graph is built with LangGraph.

langdetect — Python library for language detection. Used as a fallback in `LanguageDetector`.

Latency — Time from input received to output produced. Measured in milliseconds throughout OmniLLM.

LED — Light-Emitting Diode. Pepper has them in its eyes (and elsewhere) and OmniLLM uses them as an emotional/state cue.

Likert scale — Survey scale with N points (typically 5 or 7) ranging from "strongly disagree" to "strongly agree." Primary outcome of OmniLLM's experimental study.

litellm — Python library by BerriAI that wraps 100+ LLM providers behind a unified API. OmniLLM's gateway is built on litellm.

LLM — Large Language Model. AI system trained on natural-language data; takes text in, produces text out.

LLM-as-judge — Using one LLM to score the output of another. Implemented in `omnillm/evaluator.py`. Used for RAG faithfulness and benchmark scoring.

LLM Council — See *Consensus*.

MMLU — Massive Multitask Language Understanding benchmark (Hendrycks et al. 2021). A 57-subject knowledge test. OmniLLM compares its embodied rankings against MMLU rankings (H1).

NAOqi — Pepper's middleware operating system. Locked to Python 2.7. Provides the AL* services (ALMotion, ALAnimatedSpeech, ...).

Ollama — Free local LLM runtime. Listens on `http://localhost:11434`. Runs open-weight models (Llama, Qwen, Mistral, Phi) on your own CPU/GPU.

Pairwise preference — Question that asks the participant to choose between two options. The basis of the ELO scorer.

Participant ID — Anonymous code (P001, P002, ...) used everywhere in OmniLLM logs. The mapping to real identity lives only on paper, in the experimenter's binder.

PepperBridge — The Python-3 `RobotBridge` subclass that drives Pepper via HTTP to `naoqi_bridge_server`.

pip — Python's package installer.

Position bias — In pairwise LLM-as-judge, the tendency of the judge to favour whichever response is presented first. Mitigated by swap-and-aggregate.

pypdf — Python library for extracting text from PDF documents. Used optionally by RAG pipeline.

pyproject.toml — Modern Python package metadata file. Defines OmniLLM's dependencies and console scripts.

pytest — Python testing framework. OmniLLM has 289 tests, all mocked.

RAG — Retrieval-Augmented Generation. Search a corpus, then prompt the LLM with retrieved context.

Repeated-measures ANOVA — ANOVA variant for within-subjects designs. Each participant is their own block.

ReportLab — Python PDF library, used by `xhtml2pdf` (which the book uses).

Robot bridge — Generic name for the layer between the AI Layer and the robot. Abstract in `omnillm/robotics/bridge.py`.

RobotAction — Dataclass in `omnillm/robotics/bridge.py`. The canonical "what should the robot do?" message.

RobotSensorData — Dataclass. The canonical "what is the robot sensing?" message.

Sentence-Transformers — Python library for sentence embeddings. Used by RAGPipeline (model `all-MiniLM-L6-v2` by default).

Sgorbissa, Antonio — Professor at DIBRIS, University of Genoa. Director of the HRI lab where this thesis's Pepper lives.

SoftBank Robotics — Manufacturer of Pepper (formerly Aldebaran Robotics).

Spearman ρ — Rank-correlation coefficient. Used to test H1 (embodied ranking vs text ranking).

STT — Speech-to-Text. Done locally by Whisper in OmniLLM.

Synthesis (consensus strategy) — Default consensus mode. A judge LLM combines N parallel answers into one synthesised response.

Tablet — Pepper's chest-mounted touchscreen. Accessible via `ALTabletService`.

Task type — T1 (Information Retrieval), T2 (Navigation), T3 (Social Conversation), T4 (Multilingual). The four task categories of the Embodied LLM Arena.

Topology 1 — Pepper polls the AI server. Conversation initiated by the robot.

Topology 2 — AI server drives Pepper via the bridge server. Conversation initiated by the AI Layer.

TTS — Text-to-Speech. Done by Pepper's `ALAnimatedSpeech`.

Unicode script — Category of a Unicode character (Latin, Arabic, CJK, Hangul, ...). Used as the first signal in language detection.

Utterance — One spoken-and-transcribed turn from the participant. The `utterance` field in `HRIGraphState`.

VAD — Voice Activity Detection. Energy-thresholded detection of when the participant is speaking. One of the three trigger modes in `naoqi_client.py`.

venv — Python virtual environment. See Appendix A.1.

Vision-language model (VLM) — LLM extended to accept image input alongside text. Out of scope in this thesis; discussed in Chapter 33.

WAV — Waveform Audio File format. The audio format used between Pepper's microphone and Whisper.

Whisper — OpenAI's speech-to-text model. Used locally in OmniLLM (not the cloud API).

Within-subjects design — Each participant experiences every condition. Higher statistical power than between-subjects for the same N.

xhtml2pdf — Pure-Python HTML->PDF library. Used to render this book.

YAML — YAML Ain't Markup Language. Human-readable data format. Used for `config/models.yaml` and benchmark task definitions.

Appendix C — Complete Terminal Command Reference

Every command used anywhere in the book, organised by goal. PowerShell first; Bash equivalent below when different.

C.1 — Project Setup

```
# Clone
cd C:\Users\akshi\OneDrive\Desktop
git clone https://github.com/Akshita-sr/OmniLLM.git
cd OmniLLM

# Create + activate virtual environment
python -m venv venv
.\venv\Scripts\Activate.ps1          # PowerShell
# source venv/bin/activate           # Bash

# Install
pip install --upgrade pip
pip install -e ".[all]"               # all extras

# (Optional) Set up env
Copy-Item .env.example .env          # PowerShell
notepad .env                          # Bash
# nano .env                           # Bash

# (Optional) Ollama
ollama --version
ollama pull llama3.2:3b
ollama pull qwen2.5:7b
ollama serve

# Verify
omnillm --version
omnillm models
pytest tests/ -q
```

C.2 — The AI Server

```
# Default — listen on 0.0.0.0:5000 for LAN access
python -m omnillm.server.app --host 0.0.0.0 --port 5000

# Localhost only (development)
python -m omnillm.server.app --host 127.0.0.1 --port 5000

# Disable RAG (faster startup; not for the study)
python -m omnillm.server.app --no-rag

# Different default model
python -m omnillm.server.app --model llama3-8b-local

# Different knowledge base directory
python -m omnillm.server.app --kb knowledge_base\_legacy_irai

# Flask debug mode (auto-restarts on code change)
python -m omnillm.server.app --debug
```

C.3 — The NAOqi Bridge Server (Python 2.7)

```
# Real Pepper at DIBRIS
C:\Python27\python.exe omnillm\server\ naoqi_bridge_server.py `
  --robot-ip 192.168.1.42 --robot-port 9559 `
  --bind 0.0.0.0 --bridge-port 6000

# Choregraphe virtual robot (port varies per launch)
C:\Python27\python.exe omnillm\server\ naoqi_bridge_server.py `
  --robot-ip 127.0.0.1 --robot-port 49959 `
  --bind 0.0.0.0 --bridge-port 6000
```

C.4 — The NAOqi Client (Topology 1; Three Triggers)

```
# Text trigger (works on virtual robot too)
C:\Python27\python.exe omnillm\server\ naoqi_client.py `
  --robot-ip 127.0.0.1 --robot-port 49959 `
  --trigger text --condition A

# Touch trigger (real Pepper only; head touch triggers 5s capture)
C:\Python27\python.exe omnillm\server\ naoqi_client.py `
  --robot-ip 192.168.1.42 `
  --trigger touch --record-seconds 5 `
  --condition C --track-face

# VAD trigger (real Pepper only; continuous energy-gated)
C:\Python27\python.exe omnillm\server\ naoqi_client.py `
  --robot-ip 192.168.1.42 `
  --trigger vad --condition C
```

C.5 — The Demo Scripts

```
# NAOqi sanity check (no AI; robot only)
C:\Python27\python.exe scripts\pepper_demo\demo_pepper_omnillm.py `
  --check-only --robot-port 49959

# Single-shot demo (AI -> robot)
C:\Python27\python.exe scripts\pepper_demo\demo_pepper_omnillm.py `
  --condition A --rag --question "What time does the lab open?" `
  --robot-port 49959

# Batch test all 5 conditions x 4 tasks (no robot needed)
python scripts\pepper_demo\test_all_conditions.py

# Proof that routing routes correctly
python scripts\pepper_demo\proof_of_routing.py
```

C.6 — The Experiment Driver

```
# Full subject session, real robot
.\venv\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py `
  --participant P003 `
  --server http://127.0.0.1:5000 `
  --bridge http://127.0.0.1:6000

# Full subject session, no robot (driver dry-run)
.\venv\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py `
  --participant P000-dryrun `
  --server http://127.0.0.1:5000 `
  --no-robot
```

C.7 — The CLI

```
omnillm models                                # list all models
omnillm models --type cloud                    # cloud-only
omnillm models --type local                    # local-only (Ollama)

omnillm ask "Hello" -m openai-gpt4o-mini        # one model
omnillm ask "Hello" -m openai-gpt4o-mini -m claude-haiku # multiple
omnillm ask "Hello" --all                       # every registered model

omnillm route "Where is Room 305?" --strategy TASK_TYPE # smart-route
omnillm route "Quick yes/no" --strategy LOWEST_COST --budget 0.001

omnillm council "What is consciousness?"        # 3-model consensus
omnillm council "Is P=NP?" --strategy majority_vote
omnillm council "Climate" -m openai-gpt4o -m claude-sonnet -m gemini-2.5-pro

omnillm compare "Write a poem" --model-a openai-gpt4o-mini --model-b claude-haiku

omnillm evaluate                               # benchmark all
omnillm evaluate -m openai-gpt4o-mini -c reasoning # specific
omnillm evaluate -o results/eval_2026.json

omnillm leaderboard                            # overall ELO
omnillm leaderboard --category embodied_hri    # thesis result
omnillm leaderboard --category reasoning

omnillm costs                                  # USD per model

omnillm export --format csv --input results/eval.json -o results/eval.csv
omnillm export --format markdown --input results/eval.json -o results/eval.md
```

C.8 — Testing

```
pytest tests/ -v                                # verbose
pytest tests/ -q                                # quiet (just pass count)
pytest tests/ --cov=omnillm --cov-report=term-missing
pytest tests/test_gateway.py                    # one file
pytest tests/test_gateway.py::test_query_basic # one test
pytest -k "router"                             # tests matching name pattern
```

C.9 — Health Checks Against a Running Server

```
# Liveness
curl http://127.0.0.1:5000/health

# Configuration dump
curl http://127.0.0.1:5000/status

# Text-only interaction
curl -X POST http://127.0.0.1:5000/interact `
-H "Content-Type: application/json" `
-d '{"text":"Hello","participant_id":"P-test","session_id":"s1","condition":"A","rag_enabled":true}'

# Just transcribe (no LLM)
$audio = [Convert]::ToBase64String([IO.File]::ReadAllBytes("test.wav"))
curl -X POST http://127.0.0.1:5000/transcribe `
-H "Content-Type: application/json" `
-d '{"audio": `"$audio`"}'

# Export all logged interactions
curl http://127.0.0.1:5000/export

# Submit questionnaire data
curl -X POST http://127.0.0.1:5000/evaluate `
-H "Content-Type: application/json" `
-d '{"session_id":"s1","participant_id":"P003","condition":"A","scores":{"accuracy":6,"naturalness":5}}'
```

C.10 — Health Checks Against the Bridge Server

```
# Ping (is bridge alive + connected to a robot?)
curl http://127.0.0.1:6000/ping

# Sensor read
curl http://127.0.0.1:6000/sensors

# Send a hand-crafted action
curl -X POST http://127.0.0.1:6000/action `
-H "Content-Type: application/json" `
-d '{"speech":"Hello!","gesture":"wave","emotion_led":"#00FF88"}'

# Start face tracking
curl -X POST http://127.0.0.1:6000/tracker/start `
-H "Content-Type: application/json" -d '{"target":"Face"}'

# Stop face tracking
curl -X POST http://127.0.0.1:6000/tracker/stop

# Clean shutdown
curl -X POST http://127.0.0.1:6000/disconnect
```

C.11 — Building the Book PDF

```
python book\build_pdf.py # rebuild PDF + HTML
```

Appendix D — Troubleshooting Reference

Symptom -> likely cause -> fix. Organised by where in the stack the failure shows up.

D.1 — Installation

Symptom	Cause	Fix
python: command not found	Python not on PATH	Reinstall Python with the "Add to PATH" checkbox
Microsoft Visual C++ 14.0 is required	Some pip wheel needs a C++ compiler	Install Microsoft Build Tools for C++; retry
chromadb install fails	Outdated pip resolver	<code>pip install --upgrade pip</code> ; retry
whisper: command not found ffmpeg	Whisper requires ffmpeg	<code>winget install ffmpeg</code>
Activating venv: "execution policy" error	PowerShell forbids unsigned scripts	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
<code>pip install -e ".[all]"</code> says "could not find a version that satisfies"	Python < 3.11	Upgrade Python
Python 2.7 + <code>import naoqi</code> fails	PYTHONPATH not set	set <code>PYTHONPATH=C:\pynaoqi\...\lib;%PYTHONPATH%</code>
omnillm: command not found after install	venv not activated	<code>.\venv\Scripts\Activate.ps1</code>

D.2 — AI Server (Python 3)

Symptom	Cause	Fix
Server starts but <code>/status</code> shows <code>rag_enabled=false</code>	KB directory empty / wrong path	Check <code>--kb</code> flag; verify files exist
Server starts but <code>/status</code> shows <code>langgraph_available=false</code>	LangGraph not installed	<code>pip install -e ".[hri]"</code>
<code>/interact</code> returns 500 with empty body	LLM call timed out	Check provider status page; bump <code>litellm</code> timeout
<code>/interact</code> returns <code>"path": "fallback"</code>	LangGraph crashed; fell back to direct gateway	Check server log for the actual graph error
<code>/interact</code> returns empty speech	(Pre-May-2026) <code>_merge_state</code> wrapper missing	This was fixed; if it returns, check that the agent graph is up-to-date
<code>KeyError: 'openai-gpt4o-mini'</code>	<code>models.yaml</code> does not contain that ID	Edit <code>config/models.yaml</code>
<code>429 RESOURCE_EXHAUSTED</code> from Google	Google free-tier quota	Set <code>GOOGLE_API_KEY=""</code> or upgrade tier
<code>401 Unauthorized</code> from OpenAI	Wrong / missing API key	Check <code>.env</code> ; <code>sk-</code> prefix; correct env var name
Server takes >30 s to start	First-time embedding-model download	One-time only; wait

D.3 — NAOqi Bridge (Python 2.7)

Symptom	Cause	Fix
<code>[ERR] ALBroker construction failed</code>	Wrong IP / wrong port / Pepper asleep / firewall	Re-check <code>--robot-ip</code> ; chest-button to wake; check firewall rule
<code>[ERR] ImportError: No module named naoqi</code>	PYTHONPATH not set in this terminal	set <code>PYTHONPATH=C:\pynaoqi\...\lib;%PYTHONPATH%</code>
<code>/ping</code> returns <code>"simulation": true</code>	Connected to Choregraphe, not real Pepper	Re-check <code>--robot-ip</code> and <code>--robot-port</code>
<code>/action</code> returns <code>"errors": {"gesture": "behavior not installed"}</code>	Animation library does not include that gesture	Pre-install via Choregraphe (Behavior Manager) OR limit the gesture planner to a known-good subset
<code>/action</code> returns 200 OK but Pepper doesn't move	NAOqi safety-mode active (low battery / overheat)	Plug Pepper into charger; let cool down
Bridge crashes on shutdown	<code>motion.rest()</code> race condition	Harmless on exit; ignore

D.4 — Real-Pepper Connectivity

Symptom	Cause	Fix
<code>ping <PEPPER_IP></code> times out	Different subnet / wrong Wi-Fi / Pepper offline	Confirm Pepper's IP; join correct SSID; chest-button to wake
Pepper speaks but gestures all fail	Connected to virtual robot	Re-check <code>--robot-ip</code> ; not <code>127.0.0.1</code>
Pepper's TTS is silent	Speakers muted at OS level	Use Choregraphe Volume slider; or <code>ALAudioDevice.setOutputVolume(80)</code>
Whisper transcribes Italian as garbage English	<code>model_size="base"</code> weak on accent	Bump to <code>"small"</code> in <code>whisper_stt.py</code>
Touch trigger fires randomly	Hair / clothing brushing head sensor	Re-instruct participant; use VAD or text trigger

D.5 — Experiment Driver

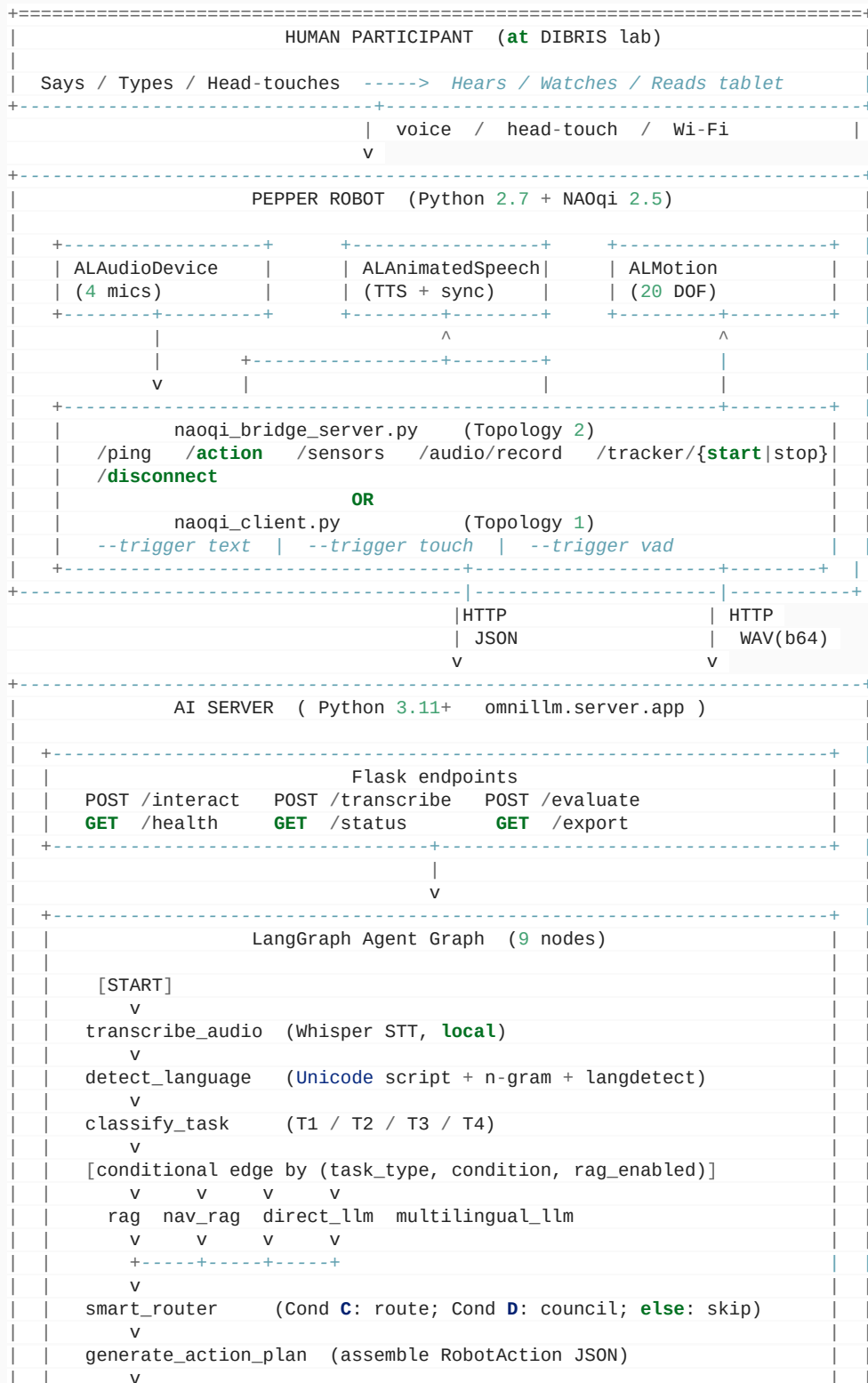
Symptom	Cause	Fix
<code>ERROR: server unreachable</code> mid-session	AI server restarted / crashed	Restart server in window #1; re-run driver
Latency suddenly > 10 s	OpenAI outage	Check status.openai.com ; switch to Condition B
CSV missing rows	Server log JSONL not flushed	Add explicit <code>fh.flush()</code> in <code>experiment_logger.py</code> ; for now check <code>/export</code>
Driver exits with <code>BrokenPipeError</code>	<code>aiohttp</code> session closed unexpectedly	Re-run; if persistent, switch to wired Ethernet
Condition B returns cloud-shaped responses	Ollama not running	<code>ollama serve</code> ; <code>ollama list</code> confirms model is pulled

D.6 — Book PDF Build

Symptom	Cause	Fix
<code>reportlab.pdfbase</code> font registration warns about missing DejaVu	Matplotlib not installed	<code>pip install matplotlib</code> (it ships with the DejaVu fonts)
Box-drawing characters render as ■ squares in the PDF	Character not in DejaVu Sans Mono / not in <code>_BOX_REPLACE</code> map	Add the offending Unicode codepoint to <code>_BOX_REPLACE</code> in <code>build_pdf.py</code> and rerun
Emoji rendered as squares	Not in <code>_EMOJI_REPLACE</code> map	Same: add to map, rerun
<code>xhtml2pdf</code> warns about CSS	Some unsupported property	Cosmetic; PDF still builds

Appendix E — End-to-End Flow Diagram (One Page)

Print this page and pin it above your desk. It is the entire system on one sheet.



```

| | log_interaction      (ExperimentLogger -> JSONL + CSV) | |
| | v |
| | [END] --> RobotAction JSON back to Flask back to Pepper | |
+-----+
|
+-----+ +-----+
| RAG Pipeline | | OmniLLM Core | |
| ChromaDB + keyword | | LiteLLM Gateway (19 models) | |
| fallback + PDF/CSV/TXT | | Smart Router (6 strategies) | |
| + faithfulness scoring | | Consensus / Council (3 strategies) | |
+-----+ | LLM-as-Judge (3 patterns) | |
| | ELO Scorer (per-category) | |
+-----+
|
+-----+
| LLM Backends (via LiteLLM, unified async API) | | |
| OpenAI GPT-4o GPT-4o-mini GPT-3.5-turbo | |
| Anthropic Claude Sonnet Claude Haiku | |
| Google Gemini 2.5 Pro Gemini 2.5 Flash Gemini Flash | |
| DeepSeek DeepSeek V3 | |
| Ollama (local, $0): Llama 3:8b Llama 3.2:3b | |
| | Qwen 2.5:7b Mistral 7B Phi-3 | |
+-----+
=====
|
| CONDITIONS A-E (selected per request via /interact body)
| A: Fixed cloud (GPT-4o-mini) + RAG (baseline)
| B: Fixed local (Llama 3:8b) + RAG (free / offline)
| C: Smart-routed + RAG (TASK_TYPE strategy)
| D: 3-model council + RAG (synthesis)
| E: GPT-4o-mini + no RAG (control for H3)
|
| TASKS T1-T4 (classified from utterance)
| T1: Information Retrieval ----> rag node
| T2: Navigation ----> nav_rag node (+ pointing gesture)
| T3: Social Conversation ----> direct_llm node
| T4: Multilingual ----> multilingual_llm node (Claude Haiku)

```

Appendix F — File Index — Every File in the Repository, One Line Each

Look-up reference. Files are grouped by directory.

omnillm/ (the main Python package)

File	Lines	Purpose (one line)
<code>__init__.py</code>	~10	Re-exports the public API of the package
<code>gateway.py</code>	~270	Unified LLM gateway; LiteLLM-backed; async query
<code>router.py</code>	~370	Smart router; 6 strategies; eval-result learning
<code>consensus.py</code>	~400	3-model LLM council; majority/weighted/synthesis
<code>evaluator.py</code>	~450	LLM-as-judge: referenceless / reference / pairwise
<code>scorer.py</code>	~350	ELO leaderboard, per-category
<code>cli.py</code>	~600	omnillm console-script entry point (click + rich)

omnillm/hri/ (Embodied LLM Arena)

File	Lines	Purpose
<code>__init__.py</code>	~5	Re-exports <code>build_hri_graph</code> , the experiment manager
<code>agent_graph.py</code>	~650	The 9-node LangGraph pipeline. Pepper's brain.
<code>classifier.py</code>	~340	Rule-based + LLM HRI task classifier (T1–T4)
<code>language_detector.py</code>	~320	Script + n-gram + langdetect language detection
<code>experiment.py</code>	~280	Conditions A–E; ParticipantSession; ExperimentManager

omnillm/rag/

File	Lines	Purpose
<code>__init__.py</code>	~5	Re-exports RAGPipeline, DocumentChunk, RAGResponse
<code>pipeline.py</code>	~600	ChromaDB indexing + retrieval + faithfulness scoring

omnillm/robotics/

File	Lines	Purpose
<code>__init__.py</code>	~5	Re-exports the bridge factory + types
<code>bridge.py</code>	~250	Abstract RobotBridge + RobotAction / RobotSensorData
<code>pepper.py</code>	~440	Python-3 client of the bridge server; 3-mode fallback
<code>gesture_planner.py</code>	~200	(task_type, response_text) -> (gesture, LED)
<code>whisper_stt.py</code>	~200	Local or API Whisper STT

omnillm/server/

File	Lines	Purpose
<code>__init__.py</code>	~5	(Empty re-export shell)
<code>app.py</code>	~480	Flask AI server. Six HTTP endpoints.
<code>naoqi_bridge_server.py</code>	~600	Python-2.7 HTTP bridge; stdlib only
<code>naoqi_client.py</code>	~700	Python-2.7 conversation loop; text/touch/vad triggers

omnillm/tasks/

File	Lines	Purpose
<code>__init__.py</code>	~5	Re-exports loader
<code>loader.py</code>	~110	Loads benchmark task YAML files from <code>config/tasks/</code>
<code>sample_tasks.py</code>	~80	Bundled benchmark prompts across 8 evaluation axes

omnillm/utils/

File	Lines	Purpose
<code>__init__.py</code>	~5	Re-exports the logger + collector + exporter
<code>experiment_logger.py</code>	~280	InteractionRecord + JSONL + CSV export
<code>questionnaire.py</code>	~390	Likert / Godspeed / pairwise / observer dataclasses
<code>cost_tracker.py</code>	~180	Per-model running USD spend
<code>export.py</code>	~200	JSON ↔ CSV ↔ Markdown converter

config/

File	Lines	Purpose
<code>models.yaml</code>	~300	19-model registry + routing config
<code>tasks/*.yaml</code>	varies	Benchmark task definitions (one file per evaluation axis)

knowledge_base/ (RAG-indexed)

File	Lines	Purpose
lab_info.txt	~80	DIBRIS / Sgorbissa lab description
faq.txt	~120	Common visitor questions and answers
research_projects.txt	~150	Active research lines, key publications
university_map.txt	~100	DIBRIS floor map, room locations, transit
event_schedule.csv	~30 rows	Upcoming lab events
visitor_profiles.csv	~20 rows	Typical visitor categories
_legacy_irai/	—	Old IRAI-Lab content preserved for reference

scripts/pepper_demo/

File	Lines	Purpose
README.md	~160	Quick reference for all demo scripts
REAL_PEPPER_CHECKLIST.md	~150	The Day-Zero checklist (mirrored in Chapter 21)
demo_pepper_omnillm.py	~250	Python-2.7 single-shot demo
test_all_conditions.py	~180	Python-3 batch test of all 5×4 conditions
run_subject_experiment.py	~250	The subject experiment driver
proof_of_routing.py	~120	Demonstrates that Condition B actually runs Llama
launch_real_pepper.ps1	~50	PowerShell helper to launch server + bridge in one go

File	Tests	Purpose
test_gateway.py	21	LiteLLM strings, cost calc, error handling
test_router.py	24	All 6 strategies, constraint enforcement
test_consensus.py	20	Majority / weighted / synthesis
test_evaluator.py	15	3 judge patterns
test_scorer.py	22	ELO math + leaderboard
test_hri.py	51	Classifier, language detector, experiment manager
test_rag.py	26	Indexing, retrieval, faithfulness, hallucination
test_gesture_planner.py	30	Gesture-selection rules
test_pepper_bridge.py	(variable)	Stub/direct/server mode auto-detection
test_experiment_logger.py	16	Append-only log + CSV export
test_new_components.py	53	agent_graph, whisper_stt, questionnaire, server, KB
Total	278+	All mocked. No API keys or robot needed.

File	Lines	Purpose
00_front_matter.md	~200	Title page, copyright, preface
01_toc.md	~80	Table of contents
02_part1_motivation.md	~750	Part I — Motivation & Scope
03_part2_architecture.md	~700	Part II — Architecture, Flow, Tech Stack
04_part3a_master_table.md	~250	Part III, Chapter 10 — Master file table
05_part3b_core_modules.md	~600	Part III, Chapter 11 — Core LLM modules
06_part3c_hri_rag.md	~700	Part III, Chapter 12 — HRI & RAG modules
07_part3d_robotics_server_utils_bridge.md	~300	Part III, Chapters 13–16
08_part4_setup_and_day_zero.md	~900	Part IV — Setup + Day Zero
09_part5_experiments.md	~1000	Part V — Experiments
10_part6_future.md	~500	Part VI — Future Scope
11_appendix_a_python_primer.md	~400	Python primer for beginners
12_appendix_b_glossary.md	~280	Glossary
13_appendix_c_commands_d_troubleshooting.md	~400	Commands + troubleshooting
14_appendix_e_flow_f_file_index.md	~250	Flow diagram + file index (this appendix)
15_appendix_g_h_references.md	~800	External resources + Pepper-LLM survey
OmniLLM_Book.md	~9500	Master concatenated copy for the PDF builder
build_pdf.py	~330	Markdown -> HTML -> PDF pipeline

Root

File	Purpose
README.md	The README on GitHub
LICENSE	MIT
pyproject.toml	Package metadata, dependencies, optional extras
requirements.txt	Flat dependency list
.env.example	API key template

Appendix G — External Resources: Papers, Libraries, Repositories (Annotated)

Every external work cited in this book. Organised by topic, with one-paragraph annotations on those that influenced specific design choices. Author/year format throughout; full bibliographic entries follow.

G.1 — Pepper Robot: The Platform Itself

Pandey & Gelin (2018) — "A Mass-Produced Sociable Humanoid Robot: Pepper, The First Machine of Its Kind" — *IEEE Robotics & Automation Magazine*. The canonical introduction to Pepper as a research platform. Authors are from SoftBank/Aldebaran. Read this first if you have never used Pepper.

Lafaye, Gouaillier & Wieber (2014) — "Linear model predictive control of the locomotion of Pepper, a humanoid robot with omnidirectional wheels" — *IEEE Humanoids*. The original Pepper locomotion-control paper. Background reading.

Mishra et al. (2024) — "An Exploration of the Pepper Robot's Capabilities: Unveiling Its Potential" — *Applied Sciences*. Recent, very useful catalogue of Pepper's strengths and weaknesses (microphones, depth sensor, NAOmark detection, etc.). Strongly recommended for anyone deploying Pepper today.

Gardecki & Podpora (2017) — "Experience from the operation of the Pepper humanoid robots" — *IEEE PAEE*. Practitioner's report on Pepper's quirks; reads like a senior developer's blog. Useful gotcha reference.

Bal, Tekerek, Gök & ■im■ir (2024) — "Human Robot Interaction with Social Humanoid Robots" — *El-Cezeri Fen ve Mühendislik Dergisi*. Recent overview of Pepper-specific HRI patterns.

Sugiyama (2021) — "The Apparatgeist of Pepper-kun: An Exploration of Emerging Cultural Meanings of a Social Robot in Japan" — book chapter. Cultural-context paper. Relevant to multi-site replication arguments.

G.2 — Pepper for Specific HRI Use Cases

Receptionist / Front-Desk / Tour Guide

Suddrey, Jacobson & Ward (2018) — "Enabling a Pepper Robot to provide Automated and Interactive Tours of a Robotics Laboratory" — *arXiv*. Closest published cousin of OmniLLM's lab-tour scenario. Single-LLM-replacement architecture.

Gardecki, Podpora, Beniak & Klin (2018) — "The Pepper Humanoid Robot in Front Desk Application" — *IEEE PAEE*. Reception desk with NLU. Pre-LLM era; useful baseline.

Draghici, Dobre, Misaros & Stan (2022) — "Development of a Human Service Robot Application Using Pepper Robot as a Museum Guide" — *IEEE AQTR*. Museum guide use case.

De Gauquier et al. (2018) — "Humanoid Robot Pepper at a Belgian Chocolate Shop" — *HRI '18 Companion*. Real-world retail deployment.

Aaltonen, Arvola, Heikkilä & Lammi (2017) — "Hello Pepper, May I Tickle You? Children's and Adults' Responses to an Entertainment Robot at a Shopping Mall" — *HRI '17 Companion*. Field observation; first-encounter dynamics.

Healthcare / Therapy

Carros et al. (2022) — "Care Workers Making Use of Robots: Results of a Three-Month Study on Human-Robot Interaction within a Care Home" — *CHI 2022*. Three-month longitudinal study with Pepper. The benchmark for "long-term deployment" research (cited in Chapter 33).

Betrian et al. (2022) — "Characteristics of interactive communication between Pepper robot, patients with schizophrenia, and healthy persons" — *Belitung Nursing Journal*. Schizophrenia patients interacting with Pepper. Methodologically interesting.

Blindheim, Solberg, Hameed & Alnes (2023) — "Promoting activity in long-term care facilities with the social robot Pepper: a pilot study" — *Informatics for Health and Social Care*. Activity-promotion intervention.

Castellano, De Carolis, Macchiarulo & Pino (2022) — "Detecting Emotions During Cognitive Stimulation Training with the Pepper Robot" — *Human-Friendly Robotics 2021*. Cognitive-stimulation therapy.

Ampadu, Rokohl, Mahmood, Reichenbach & Huebner (2022) — "InjectMeAI—Software Module of an Autonomous Injection Humanoid" — *Sensors*. Pepper performing autonomous injection (proof-of-concept). QiSDK + Python wrapper architecture.

Uluur, Kose, Oz, Aydinalev & Barkana (2020) — "Towards An Affective Robot Companion for Audiology Rehabilitation: How Does Pepper Feel Today?" — *IEEE RO-MAN*. Affective audiology companion.

Stommel, de Rijk & Boumans (2022) — "'Pepper, what do you mean?' Miscommunication and repair in robot-led survey interaction" — *IEEE RO-MAN*. Closely related to OmniLLM's experimental protocol: Pepper conducts surveys with elderly people. Documents misunderstanding patterns we should expect.

Education / Tutoring

Lehmann & Rossi (2019) — "Social Robots in Educational Contexts: Developing an Application in Enactive Didactics" — *Journal of e-Learning*. Educational HRI overview.

Pigureddu & Gena (2023) — "Using the power of memes: The Pepper Robot as a communicative facilitator for autistic children" — *arXiv*. Autism-spectrum-focused communication study.

De Carolis, D'Errico & Rossano (2021) — "Pepper as a Storyteller: Exploring the Effect of Human vs. Robot Voice on Children's Emotional Experience" — *INTERACT 2021*. Pepper voice vs human voice for storytelling.

Matulík, Vavrečka & Vidovičová (2020) — "Edutainment Software for the Pepper Robot" — *ISCSIC 2020*. Education + entertainment chatbot architecture.

Knežević et al. (2023) — "Physical Education Exercises Validation Through Child-Humanoid Robot Interaction" — *Advances in Service and Industrial Robotics*. Pepper as exercise instructor.

Yun et al. (2022) — "AI-Based Open-Source Gesture Retargeting to a Humanoid Teaching Robot" — *AIED 2022*. Teacher-gesture retargeting to Pepper. Relevant to OmniLLM's gesture planner: validates that gesture-speech sync matters.

Yoshino & Zhang (2020) — "Evaluation of Teaching Assistant Robot for Programming Classes" — *Int. J. Information & Education Technology*. TA-robot evaluation.

G.3 — Pepper + LLMs (the most directly relevant cluster)

These are the closest published comparisons to OmniLLM. Most are post-2024.

Hafez, Raneem Abdel — "Enhancing Human-Robot Interaction: Integrating Large Language Models and Advanced Speech Recognition into the Pepper Robot." Recent thesis-level work integrating multiple LLMs and ASR into Pepper. Two-phase architecture (web server + Pepper). Closest in spirit to OmniLLM; OmniLLM extends with full benchmark + smart routing + agent graph.

Mauliana, Ashok, Czernochowski & Berns (2025) — "Exploring LLM-powered multi-session human-robot interactions with university students" — *Frontiers in Robotics and AI*. Open-domain multi-session university-student LLM dialogues with Pepper. Strongly relevant; multi-session study design that OmniLLM could replicate.

Rahimi, Bahaj, Abrini, Khoramshahi, Ghogho & Chetouani (2025) — "USER-VLM 360: Personalized Vision Language Models with User-aware Tuning for Social Human-Robot Interactions" — *arXiv 2502.10636*. Vision-language model + Pepper. Future-work direction for OmniLLM (Chapter 33).

Dogan, Ozyurt, Cinar & Gunes (2025) — "GRACE: Generating Socially Appropriate Robot Actions Leveraging LLMs and Human Explanations" — *ICRA 2025*. LLM-generated robot actions with social-appropriateness scoring. Closely relevant for the safety/etiquette dimension of OmniLLM's responses.

Billing (2023) — "Language Models for Human-Robot Interaction" — *HRI 2023 (extended abstract)*. Conceptual position paper that motivated much of the subsequent work in this cluster.

Sun et al. (2025) — "Trinity: A Modular Humanoid Robot AI System" — *arXiv*. Modular humanoid AI architecture combining RL + LLM + VLM. Architectural counterpoint to OmniLLM (Trinity is more vertically integrated; OmniLLM is more horizontally modular).

G.4 — LLM Benchmarking and Evaluation

Hendrycks, Burns, Basart, Zou, Mazeika, Song & Steinhardt (2021) — **"Measuring Massive Multitask Language Understanding"** — *ICLR 2021*. MMLU benchmark. The reference text-only knowledge benchmark; H1 contrasts the *embodied* leaderboard against MMLU ranking.

Chen et al. (2021) — **"Evaluating Large Language Models Trained on Code"** — *arXiv 2107.03374*. HumanEval benchmark + Codex.

Cobbe et al. (2021) — **"Training Verifiers to Solve Math Word Problems"** — *arXiv 2110.14168*. GSM8K benchmark.

Zhou et al. (2023) — **"Instruction-Following Evaluation for Large Language Models"** — *arXiv 2311.07911*. IFEval — verifiable-instruction benchmark.

Lin, Hilton & Evans (2022) — **"TruthfulQA: Measuring How Models Mimic Human Falsehoods"** — *ACL 2022*. TruthfulQA benchmark.

White et al. (2025) — **"LiveBench: A Challenging, Contamination-Limited LLM Benchmark"** — *ICLR 2025*. LiveBench — contamination-resistant rotating benchmark. Methodologically influential for OmniLLM's "compare across rolling benchmark snapshots" approach.

Chen et al. (2025) — **"Dynamic Benchmarking of Reasoning Capabilities in Code Large Language Models Under Data Contamination"** — *ICML 2025*. Contamination-aware dynamic benchmarking.

Mousavi et al. (2025) — **"Garbage In, Reasoning Out? Why Benchmark Scores are Unreliable and What to Do About It"** — *arXiv*. Audits common reasoning benchmarks for design flaws. Worth reading before treating any single benchmark as authoritative.

Li, Guerin & Lin (2024) — **"An Open Source Data Contamination Report for Large Language Models"** — *arXiv 2310.17589*. Contamination report across 15 LLMs.

Xu, Guan, Greene & Kechadi (2024) — **"Benchmark Data Contamination of Large Language Models: A Survey"** — *arXiv*. Survey of benchmark contamination.

Chen, Chen, Li et al. (2025) — **"Recent Advances in Large Language Model Benchmarks against Data Contamination: From Static to Dynamic Evaluation"** — *arXiv 2502.17521*. Static-to-dynamic benchmarking survey.

Liu, Iter, Xu, Wang, Xu & Zhu (2023) — **"G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment"** — *arXiv 2303.16634*. Establishes that LLM-as-judge with chain-of-thought and form-filling produces evaluations strongly correlated with human judgments. **Foundational for OmniLLM's evaluator.py**.

Zheng, Chiang, Sheng et al. (2023) — **"Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena"** — *NeurIPS 2023 Datasets & Benchmarks*. The position-bias swap-and-aggregate methodology that OmniLLM's pairwise judge uses.

Chiang et al. (2024) — **"Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference"** — *arXiv 2403.04132*. Chatbot Arena. **The direct inspiration for the *Embodied LLM Arena* name and the pairwise-preference ELO methodology**.

Li, Dong, Chen et al. (2024) — **"LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods"** — *arXiv*. Comprehensive recent survey.

Guan, Wang, Bian et al. (2026) — **"Evaluating LLM-based Agents for Multi-Turn Conversations: A Survey"** — *arXiv 2503.22458*. Multi-turn agent evaluation survey.

G.5 — LLM Routing, Ensembles, and Multi-Model Architecture

Ong, Almahairi, Wu et al. (2025) — "RouteLLM: Learning to Route LLMs with Preference Data" — *arXiv 2406.18665*. Learned routing between a stronger and weaker LLM. **The published-baseline methodology that OmniLLM's smart router extends.**

Chen, Zaharia & Zou (2023) — "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance" — *arXiv 2305.05176*. Three cost-reduction strategies: prompt adaptation, LLM approximation, LLM cascade. Influences OmniLLM's `LOWEST_COST` and `BEST_VALUE` strategies.

Chen, Li, Chen et al. (2025) — "Harnessing Multiple Large Language Models: A Survey on LLM Ensemble" — *arXiv 2502.18036*. Comprehensive ensemble survey. Taxonomy of ensemble methods that overlaps with OmniLLM's consensus engine.

Kallem (2026) — "Learning to Trust the Crowd: A Multi-Model Consensus Reasoning Engine for Large Language Models" — *arXiv 2601.07245*. Recent multi-model consensus paper. Findings on diminishing returns past 4 models inform Chapter 11.3's "why 3 not 5" argument.

Fedus, Zoph & Shazeer (2022) — "Switch Transformers" — *JMLR*. Mixture-of-experts at scale. Architectural background for why multi-model approaches are interesting at all.

LiteLLM documentation — Router/Load-Balancing. BerriAI's docs on `litellm`'s built-in routing primitives. Useful for understanding what the underlying library does.

Latitude blog (2024) — "Dynamic LLM Routing: Tools and Frameworks." Industry-perspective overview of LLM-routing tooling as of late 2024.

G.6 — Retrieval-Augmented Generation (RAG)

Lewis et al. (2020) — "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — *NeurIPS 2020*. The original RAG paper.

Karpukhin et al. (2020) — "Dense Passage Retrieval for Open-Domain Question Answering" — *EMNLP 2020*. The dense-retrieval methodology used by sentence-transformers + ChromaDB.

Es, James, Espinosa-Anke & Schockaert (2023) — "RAGAS: Automated Evaluation of Retrieval-Augmented Generation" — *EACL 2024 demo*. RAG evaluation framework. Closely related to OmniLLM's faithfulness scoring.

G.7 — Human-Robot Interaction: Methodology

Bartneck, Kuli■, Croft & Zoghbi (2009) — "Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots" — *International Journal of Social Robotics*. **The Godspeed Questionnaire**. Standard HRI evaluation; used in OmniLLM (Chapter 27).

Tabrez, Luebbers & Hayes (2020) — "A Survey of Mental Modeling Techniques in Human-Robot Teaming" — *Current Robotics Reports*. Mental-model survey. Background for HRI theoretical framing.

Bonarini (2020) — "Communication in Human-Robot Interaction" — *Current Robotics Reports*. Multi-modal communication overview. Informs OmniLLM's gesture-speech-LED fusion.

Fox & Gambino (2021) — "Relationship Development with Humanoid Social Robots: Applying Interpersonal Theories to Human-Robot Interaction" — *Cyberpsychology, Behavior, and Social Networking*. Theoretical critique of the "humans treat robots like humans" assumption.

Robinson, Tidd, Campbell, Kuli■ & Corke (2023) — "Robotic Vision for Human-Robot Interaction and Collaboration: A Survey and Systematic Review" — *ACM Transactions on HRI*. Systematic review of HRI/C.

Safavi, Olikkal, Pei et al. (2024) — "Emerging Frontiers in Human-Robot Interaction" — *Journal of Intelligent & Robotic Systems*. Recent survey across 3 frontiers: collaboration, BCI, affective.

Gross & Krenn (2023) — "A Communicative Perspective on Human-Robot Collaboration in Industry" — *International Journal of Social Robotics*. Multi-modal communication taxonomy. Influences OmniLLM's task taxonomy.

Pathi, Kiselev & Loutfi (2022) — "Detecting Groups and Estimating F-Formations for Social Human-Robot Interactions" — *MTI*. F-formation detection (relevant if a future OmniLLM variant supports group interactions).

Gao, Yang, Frisk, Hernandez, Peters & Castellano (2019) — "Learning Socially Appropriate Robot Approaching Behavior Toward Groups using Deep Reinforcement Learning" — *RO-MAN*. Pepper approaching groups; RL methodology.

Stancioi et al. (2021) — "Developing an application based on the interaction between humans and the Pepper robot" — *EMES*. Taxonomy paper.

G.8 — HRI: Ethics, Trust, Perception, and Long-Term Effects

Etemad-Sajadi, Soussan & Schöpfer (2022) — "How Ethical Issues Raised by Human-Robot Interaction can Impact the Intention to use the Robot?" — *International Journal of Social Robotics*. Influences OmniLLM's consent/GDPR discussion.

Horstmann & Krämer (2022) — "The Fundamental Attribution Error in Human-Robot Interaction" — *International Journal of Social Robotics*. Attribution of agency to robots; relevant to trust ratings.

Rossi, Holthaus, Dautenhahn, Koay & Walters (2018) — "Getting to know Pepper: Effects of people's awareness of a robot's capabilities on their trust in the robot" — *HAI '18*. Trust × capability awareness.

Sancarlo (Søraa), Nyvoll, Grønvik & Serrano (2021) — "Children's perceptions of social robots: a study of the robots Pepper, AV1 and Tessa at Norwegian research fairs" — *AI & SOCIETY*. Multi-robot children's-perception study. Cross-cultural relevance.

Duradoni, Colombini, Russo & Guazzini (2021) — "Robotic Psychology: A PRISMA Systematic Review on Social-Robot-Based Interventions in Psychological Domains" — *J.* Systematic review of robot-mediated psychological interventions.

Ye & Robert (2023) — "Human Security Robot Interaction and Anthropomorphism: An Examination of Pepper, RAMSEE, and Knightscope Robots" — *RO-MAN 2023*. Anthropomorphism comparison.

Stock — "Can Service Robots Hamper Customer Anger and Aggression After a Service Failure?" — Service-failure recovery experiment with Pepper.

Zantou & Vernon (2023) — "Culturally Sensitive Human-Robot Interaction: A Case Study with the Pepper Humanoid Robot" — *IEEE AFRICON*. Culture-aware HRI with Pepper.

Chiang, Bruno, Menicatti, Recchiuto & Sgorbissa (2019) — "Culture as a Sensor? A Novel Perspective on Human Activity Recognition" — *International Journal of Social Robotics*. **From the host lab (Sgorbissa, DIBRIS)**. Culture-aware HAR architecture.

Tanveer, Sgorbissa & Thomas (2020) — "An IPM Approach to Multi-robot Cooperative Localization" — *ICAR 2020*. **From the host lab**. Multi-robot cooperative localisation with Pepper.

Ghi■a et al. (2020) — "The AMIRO Social Robotics Framework: Deployment and Evaluation on the Pepper Robot" — *Sensors*. Modular social-robotics framework on Pepper.

G.9 — HRI: Navigation, Vision, Perception (Background)

Ardón, Kushibar & Peng (2019) — "A Hybrid SLAM and Object Recognition System for Pepper Robot" — *arXiv*. SLAM + object recognition on Pepper.

Gómez, Mattamala, Resink & Ruiz-del-Solar (2019) — "Visual SLAM-Based Localization and Navigation for Service Robots: The Pepper Case" — *RoboCup 2018*. Visual SLAM for Pepper.

Alhmiedat, Marei, Messoudi et al. (2023) — "A SLAM-Based Localization and Navigation System for Social Robots: The Pepper Robot Case" — *Machines*. SLAM + ROS for Pepper.

Bista, Ward & Corke (2021) — "Image-Based Indoor Topological Navigation with Collision Avoidance for Resource-Constrained Mobile Robots" — *Journal of Intelligent & Robotic Systems*. Topological navigation on Pepper.

Perera, Pereira, Connell & Veloso (2017) — "Setting Up Pepper For Autonomous Navigation And Personalized Interaction With Users" — *arXiv*. ROS + IBM cloud + Pepper.

Bauer, Escalona, Cruz, Cazorla & Gomez-Donoso (2019) — "Improving the 3D Perception of the Pepper Robot Using Depth Prediction from Monocular Frames" — *Advances in Physical Agents*. Pepper depth-perception limitation documentation.

Bauer, Escalona, Cruz, Cazorla & Gomez-Donoso (2019) — "Refining the Fusion of Pepper Robot and Estimated Depth Maps Method for Improved 3D Perception" — *IEEE Access*. Depth-perception refinement.

Khalil, Coronado & Venture (2021) — "Human Motion Retargeting to Pepper Humanoid Robot from Uncalibrated Videos Using Human Pose Estimation" — *RO-MAN*. Motion retargeting.

Reyes, Gómez, Norambuena & Ruiz-del-Solar (2019) — "Near Real-Time Object Recognition for Pepper Based on Deep Neural Networks Running on a Backpack" — *RoboCup 2018*. Pepper + Jetson backpack for vision.

Ilyas et al. (2019) — "Teaching Pepper Robot to Recognize Emotions of Traumatic Brain Injured Patients Using Deep Neural Networks" — *RO-MAN 2019*. Emotion recognition for patient populations.

G.10 — HRI: Crowd / Group / Navigation Among People

Zhang, Amirian, Eberle, Pettré, Holloway & Carlson (2022) — "From HRI to CRI: Crowd Robot Interaction—Understanding the Effect of Robots on Crowd Motion" — *International Journal of Social Robotics*. Crowd dynamics around Pepper.

Kobayashi, Sugimoto, Tanaka et al. (2022) — "Robot Navigation Based on Predicting of Human Interaction and its Reproducible Evaluation in a Densely Crowded Environment" — *International Journal of Social Robotics*. Crowded-environment navigation.

Lo, Yamane & Sugiyama (2019) — "Perception of Pedestrian Avoidance Strategies of a Self-Balancing Mobile Robot" — *IROS*. Pedestrian-avoidance perception.

Mavrogiannis, Hutchinson, Macdonald, Alves-Oliveira & Knepper (2019) — "Effects of Distinct Robot Navigation Strategies on Human Behavior in a Crowded Environment" — *HRI*. Strategy-effect study.

Genevois, Spalanzani & Laugier (2023) — "Interaction-aware Predictive Collision Detector for Human-aware Collision Avoidance" — *IV*. Interaction-aware collision avoidance.

Ma, Qiu, Chen, Yao, Chen & Ji (2022) — "Asymmetric Self-Play for Learning Robust Human-Robot Interaction on Crowd Navigation Tasks" — *CECIT*. Self-play RL for crowd navigation.

G.11 — Software Libraries (the dependencies)

Library	Purpose	License	Project URL
litellm (BerriAI)	Unified LLM gateway	MIT	https://github.com/BerriAI/litellm
langgraph	Stateful agent graph	MIT	https://github.com/langchain-ai/langgraph
langchain	LLM application framework	MIT	https://github.com/langchain-ai/langchain
chromadb	Embedded vector DB	Apache-2.0	https://github.com/chroma-core/chroma
sentence-transformers	Sentence embeddings	Apache-2.0	https://github.com/UKPLab/sentence-transformers
openai-whisper	Speech-to-text	MIT	https://github.com/openai/whisper
aiohttp	Async HTTP client/server	Apache-2.0	https://github.com/aio-libs/aiohttp
flask	HTTP server	BSD-3	https://github.com/pallets/flask
click	CLI framework	BSD-3	https://github.com/pallets/click
rich	Terminal rendering	MIT	https://github.com/Textualize/rich
pytest	Testing	MIT	https://github.com/pytest-dev/pytest
xhtml2pdf	HTML -> PDF	Apache-2.0	https://github.com/xhtml2pdf/xhtml2pdf
reportlab	PDF primitives	BSD-3	https://www.reportlab.com/opensource/
markdown	Markdown -> HTML	BSD-3	https://github.com/Python-Markdown/markdown
pypdf	PDF text extraction	BSD-3	https://github.com/py-pdf/pypdf
langdetect	Backup language detection	Apache-2.0	https://github.com/Mimino666/langdetect
pyyaml	YAML parser	MIT	https://github.com/yaml/pyyaml

NAOqi Python 2.7 SDK is proprietary; obtained from SoftBank's developer portal subject to their EULA. Choregraphe is similarly licensed.

G.12 — Reference Implementations / Curated Awesome-Lists

GT-RIPL/Awesome-LLM-Robotics — comprehensive curated list of LLM-in-robotics papers, code, and websites. URL: <https://github.com/GT-RIPL/Awesome-LLM-Robotics>.

junchenzhi/Awesome-LLM-Ensemble — curated list accompanying Chen et al.'s LLM Ensemble survey. URL: <https://github.com/junchenzhi/Awesome-LLM-Ensemble>.

LiteLLM Router docs. Direct reference for the load-balancing primitives OmniLLM's smart router is built on top of.

Open LLM Leaderboard (Hugging Face Spaces). Public text-only LLM leaderboard. Reference point for H1.

LiveBench (livebench.ai). Rolling text-only benchmark. Reference for current SOTA per-axis.

LMSYS Org / Chatbot Arena. The text-only counterpart to the Embodied LLM Arena.

G.13 — Closely Related: Pepper Toolkits / Bridges

Ganal, Siol & Lugin (2023) — "PePUT: A Unity Toolkit for the Social Robot Pepper" — *RO-MAN*. Unity-based Pepper toolkit. Architectural cousin of OmniLLM's bridge.

McColl, Estivill-Castro, Gilmore, McColl & Hexel (2022) — "Enabling Modern Application Development with Swift on the Nao/Pepper Robots" — *RoboCup 2021*. Swift-on-Pepper toolkit. Demonstrates that the Python-2.7 trap is real and others bridge around it differently.

Pot, Monceaux, Gelin & Maisonnier (2009) — "Choregraphe: a graphical tool for humanoid robot programming" — *RO-MAN*. The original Choregraphe paper.

Appendix H — Pepper-LLM Integration Survey — The State of the Field, May 2026

A short literature survey of the post-2023 cluster of papers integrating LLMs into Pepper. Written for the thesis examiner who wants a quick orientation in the field.

H.1 — The Landscape

As of May 2026, published work integrating LLMs into Pepper specifically falls into roughly five clusters:

Cluster	Representative work	Position w.r.t. OmniLLM
Single-LLM replacement of legacy NLU	Hafez (2024); Mauliana et al. (2025)	Closest baseline. OmniLLM extends with multi-model + smart routing.
Multimodal (vision + language)	Rahimi et al. (2025) USER-VLM 360	Vision input we do not currently use. Future direction (Chapter 33).
Socially-appropriate action generation	Dogan et al. (2025) GRACE	Safety/etiquette filter. Future direction (Chapter 32).
Conceptual / position pieces	Billing (2023)	Foundational arguments. We cite these to frame the gap.
Survey / vision-papers	Sun et al. (2025) Trinity, Safavi et al. (2024)	Architectural overviews. OmniLLM is one instance of the pattern.

H.2 — What All Five Clusters Have in Common

- **Single LLM per system.** No prior published work — to the author’s knowledge — compares multiple LLMs as interchangeable backends in the same Pepper deployment. **This is the gap OmniLLM fills.**
- **Single-site, often single-session.** Generalisation across labs and across sessions is rarely tested.
- **Text-or-judge evaluation.** Few papers cleanly distinguish text-only evaluation from embodied evaluation.
- **No RAG-vs-no-RAG isolation.** RAG is either present or absent throughout; few papers run the within-subject RAG-off control that H3 requires.

H.3 — OmniLLM's Three Concrete Contributions

1. **A reproducible methodology for comparing N LLMs as social-robot brains.** The 5×4 within-subjects design, the Latin-square counterbalancing, the questionnaire instrument, and the open codebase together form a replicable pipeline. Any lab with a Pepper can run the same protocol.
2. **The Embodied LLM Leaderboard.** The first ELO-ranked leaderboard of LLMs by their performance as social-robot brains. Tested H1 (embodied vs text-only ranking differ).
3. **The smart-routing condition.** No prior work has tested *dynamic per-task routing* in live HRI. Condition C provides the first such data point.

H.4 — Methodological Choices to Defend

A thesis examiner is likely to challenge:

Challenge	Defence
<i>N=15 is small.</i>	Power analysis suffices for the within-subjects ANOVA (Chapter 23). Future replication explicitly proposed (Chapter 33).
<i>Single-site limits external validity.</i>	Acknowledged (Chapter 31). The contribution is methodological + initial empirical, not "this is the final embodied LLM ranking."
<i>Why these four task types and not others?</i>	T1–T4 mirror the standard HRI dialogue-act taxonomy (Chapter 23.4). Robust to adding more task types in future work.
<i>Why not include autonomous navigation?</i>	Out of scope (Chapter 4). Pepper-specific limitation. Future scope (Chapter 33).
<i>The cloud LLMs see participant text.</i>	Disclosed in consent form (Chapter 28). Future variant uses Ollama-only (Chapter 33).
<i>LLM-as-judge has self-preference bias.</i>	Acknowledged (Chapter 11.4). Cross-judge robustness check available; thesis's primary outcome is human Likert, not judge score.
<i>Why Pepper specifically?</i>	Citation density + lab availability + ready-made social affordances (Chapter 2).

H.5 — A Forward View

OmniLLM is the **first generation** of multi-LLM embodied benchmarking. By 2030, the author expects:

- The Embodied LLM Leaderboard (or its successor) to be the standard reference, just as Chatbot Arena is for text today.
- Models tested *as multi-modal embodied agents* will diverge sharply from models tested *as text chatbots*.
- The cost of running the full benchmark to fall by an order of magnitude (smaller, faster LLMs hitting the same quality).
- Counterbalancing automation, ethics-protocol templating, and questionnaire delivery will all be commodity tools.

This thesis is a snapshot of where the field is in May 2026. The codebase is built to be re-runnable in May 2027, in May 2028, and so on — with the same protocol, against the LLMs current at that date.

Appendix I — The Pepper Platform Reference

Everything you need to know about the hardware, NAOqi middleware, Choregraphe IDE, and community-validated bridge patterns — collected in one place. Parts I–VI use Pepper as a black box; this appendix opens the black box.

Ported and updated from the first-edition book's Part IV (Chapters 23–27A). Knowledge-base references have been updated from the legacy IRAI Lab to the current DIBRIS / Sgorbissa HRI Lab context.

I.1 — Meet Pepper: The Hardware Inside the Plastic Shell

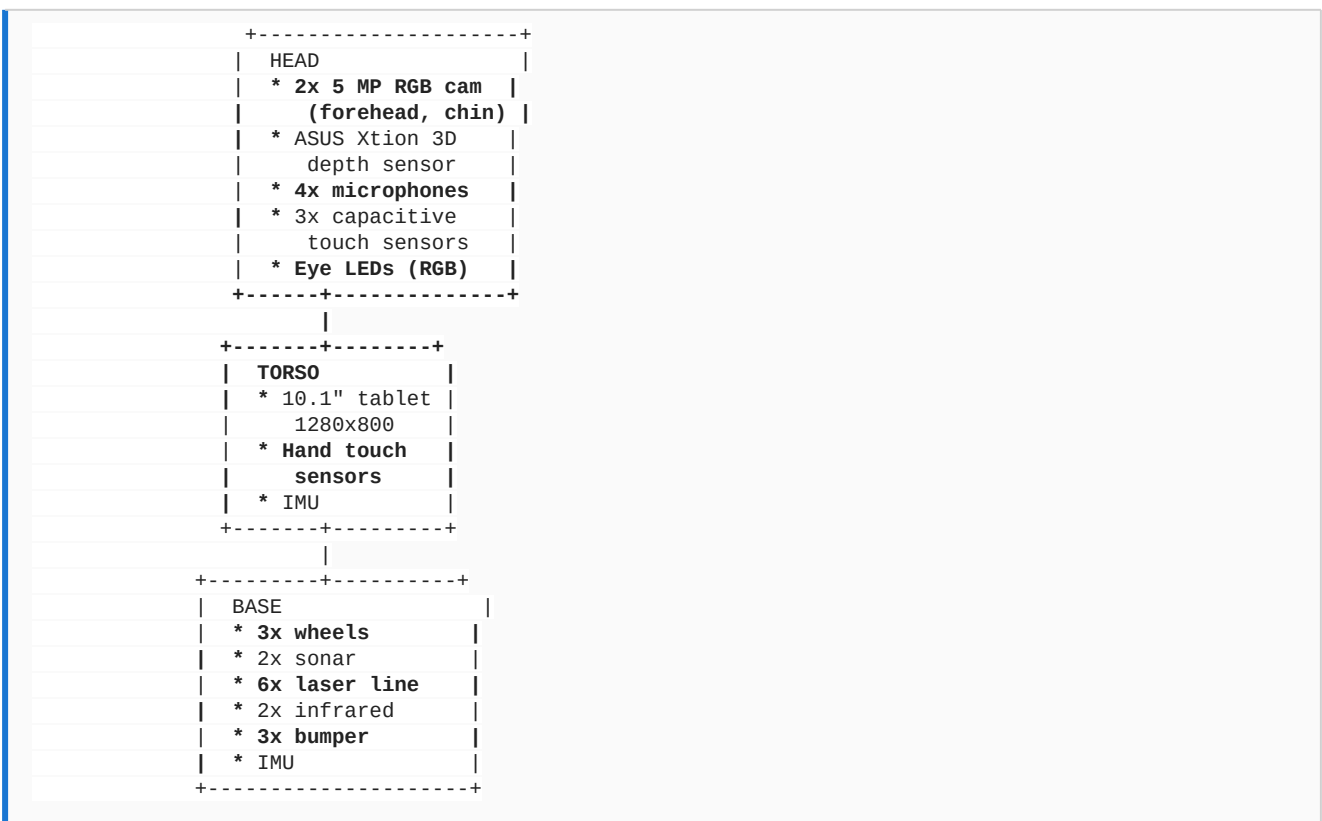
At a Glance

Pepper is a **120 cm**, **28 kg** humanoid robot from SoftBank Robotics. **20 degrees of freedom**. Intel Atom CPU, 4 GB RAM. Four microphones in the head. Two ear speakers. A 10.1" chest tablet. Eye LEDs. Three omnidirectional wheels at the base. ~8–10 hours of battery. Knowing this hardware shapes how you think about the AI server's job.

I.1.1 — Physical Specs

Attribute	Value
Height	120 cm
Weight	28 kg
Degrees of freedom	20 (head 2, each arm/hand 6, hip 2, knee 1, base wheels 3)
Battery	30 Ah / 795 Wh lithium-ion (~8–10 hours active)
Onboard CPU	Intel Atom E3845 quad-core @ 1.91 GHz
Onboard RAM	4 GB DDR3
Onboard storage	8 GB flash + microSD slot
Operating system	NAOqi OS (modified Gentoo Linux)
Onboard Python	Python 2.7
Tablet OS	Android 4.4 (1.3 GHz quad-core ARM Cortex-A7, 1 GB RAM, 32 GB storage)

I.1.2 — Sensors at a Glance



The cameras and depth sensor are not currently consumed by OmniLLM; they are available for the vision-language extensions discussed in Chapter 33.

I.1.3 — The Tablet

The chest tablet is a **separate Android computer** communicating with the head computer over an internal network at IP 198.18.0.1. From OmniLLM's perspective it is just a NAOqi service: `ALTabletService`. You can `loadUrl()` to display web content, `showImage()` to display an image, or `executeJS()` to run JavaScript in the browser.

For the Embodied LLM Arena experimental study, the tablet is mostly unused (the four task types do not need a screen), but it is available for showing maps during navigation tasks. A tablet-based questionnaire UI is one of the short-term improvements in Chapter 32.

I.1.4 — The Speaker / Microphone Pair

Pepper has **four microphones** in its head, and the NAOqi audio device exposes all four channels at 48 kHz, **or** a single mixed-down channel at **16 kHz**. OmniLLM uses the 16 kHz mono channel — the same format Whisper expects.

The "front" channel (channel 3 in the four-channel layout) is the most useful for one-on-one conversation. `ALAudioDevice.setClientPreferences` configures it:

```
self._audio_device.setClientPreferences(
    "OmniLLMCapture",
    16000,      # sample rate
    3,         # channel: front
    0,         # deinterleaved: no
```

```
)
```

I.1.5 — Eye LEDs as a Communication Channel

The eye LEDs are addressable RGB LEDs exposed through `ALLeds`:

```
leds.fadeRGB("FaceLeds", r, g, b, fade_duration_seconds)
```

OmniLLM's gesture planner (Chapter 13.3) uses eye colour to communicate **interaction mode**:

Colour	Hex	Mode
Friendly green	#00FF88	Greeting, acknowledgement, social
Calm blue	#00AAFF	Navigation guidance
Default blue	#44AAFF	Idle / neutral
White	#FFFFFF	Attention to tablet
Yellow	#FFFF00	Thinking
Red-orange	#FF4400	Confused / error
Warm orange	#FF8800	Goodbye

This is **not arbitrary aesthetic**. Eye colour is a documented HRI signal that participants register subconsciously. Switching from blue (navigation) to green (success) reinforces the spoken response.

I.1.6 — Why `ALAnimatedSpeech`, Not Plain `ALTextToSpeech`

NAOqi has two text-to-speech services:

- `ALTextToSpeech` — voice only. The robot is rigid while speaking.
- `ALAnimatedSpeech` — voice + automatic body gestures synchronised to the speech content.

OmniLLM uses `ALAnimatedSpeech` **always**, for three reasons:

1. Embodied perception research (Bartneck 2009; Andrist et al. 2014) consistently shows that "talking head" robots are rated lower on naturalness and intelligence.
2. Pepper has joints — not using them is wasteful.
3. The cost is zero — `ALAnimatedSpeech` is built-in.

The configuration that produces sensible motion:

```
config = {"bodyLanguageMode": "contextual"}
animated_speech.say(text, config)
```

`bodyLanguageMode` accepts `"contextual"` (gestures match speech content — **recommended**), `"random"` (random gestures — looks unhinged), or `"disabled"` (back to talking-head mode).

I.1.7 — Hardware End-of-Life Note

Aldebaran (the original company behind Pepper and NAO) filed for bankruptcy in **February 2025**. Maxvision Technology (Shenzhen) acquired the IP in **July 2025**. **No new units are being manufactured**. Existing units continue to work; spare parts are increasingly hard to source.

This is one reason OmniLLM is designed to be platform-portable: the abstract `RobotBridge` interface (Chapter 13.1) can target NAO, Buddy, or any future robot you point it at. The brain is platform-agnostic; only the bridge implementation is Pepper-specific.

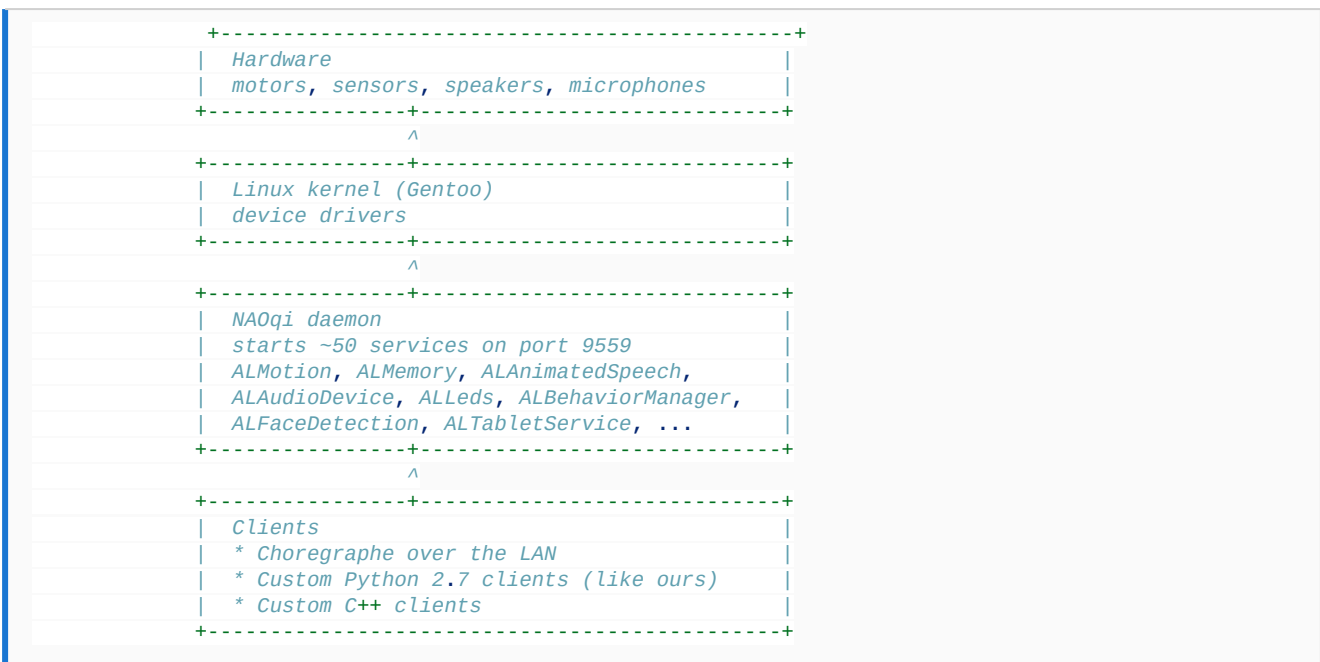
I.2 — NAOqi 101: The Operating System That Runs on Pepper

At a Glance

NAOqi is the middleware that makes Pepper a robot rather than a Linux box on wheels. It is a **service broker on TCP port 9559** that exposes named services (`ALAnimatedSpeech`, `ALMotion`, etc.) to any client that connects with the right credentials. Its Python binding is **Python 2.7 only**.

I.2.1 — What NAOqi Is

Imagine the robot as a Linux server, and NAOqi as the daemon process that runs on top of Linux providing all the high-level robot abstractions:



I.2.2 — The Service Broker Model

NAOqi is built around a **service broker**. Every NAOqi service registers itself with the broker on startup; clients look up services by name and get a proxy object that can call methods on them. Services can run **locally** (in the same process — fast, zero-copy) or **remotely** (over TCP — slower, fully serialised).

Two ways to call a service from Python:

```
# Modern (NAOqi 2.x)
import qi
session = qi.Session()
session.connect("tcp://192.168.1.100:9559")
tts = session.service("ALAnimatedSpeech")
tts.say("Hello!")

# Legacy (NAOqi 1.x)
from naoqi import ALProxy
tts = ALProxy("ALAnimatedSpeech", "192.168.1.100", 9559)
tts.say("Hello!")
```

OmniLLM's `naoqi_client.py` tries `qi` first, falls back to `naoqi`, so it works against both NAOqi 1.x and 2.x.

1.2.3 — The Twelve Services You Need to Know

Service	What it does
<code>ALMotion</code>	Joint control. <code>wakeUp()</code> enables motors; <code>rest()</code> disables. <code>setAngles()</code> moves a joint. <code>moveTo()</code> walks.
<code>ALAnimatedSpeech</code>	TTS with body gestures. <code>say(text, config)</code> .
<code>ALTextToSpeech</code>	TTS without gestures. Use when you need a stationary robot.
<code>ALAudioDevice</code>	Microphone capture. <code>setClientPreferences()</code> + <code>subscribe()</code> .
<code>ALAudioRecorder</code>	Record audio to a file. Simpler than callback-based capture.
<code>ALLeds</code>	LED control. <code>fadeRGB("FaceLeds", r, g, b, duration)</code> .
<code>ALBehaviorManager</code>	Run installed animations. <code>runBehavior("path/to/behavior")</code> . <code>isBehaviorInstalled()</code> .
<code>ALMemory</code>	Key-value store / event bus. <code>getData(key)</code> , <code>subscriber(event)</code> .
<code>ALFaceDetection</code>	Vision: detect faces. Subscribe to the <code>"FaceDetected"</code> event.
<code>ALSpeechRecognition</code>	On-board speech recognition (limited vocabulary). Mostly inadequate; OmniLLM uses Whisper instead.
<code>ALTabletService</code>	Chest tablet display. <code>loadUrl()</code> , <code>showImage()</code> .
<code>ALRobotPosture</code>	Whole-body posture. <code>goToPosture("Stand", 0.8)</code> .

Plus two more OmniLLM specifically uses:

- **ALTracker** — follows a target (face, sound, marker). Critical for the face-tracking protocol in Chapter 21.
- **ALAutonomousLife** — controls Pepper's autonomous "stay alive" behaviours. Calling `setState("disabled")` on connect is recommended to prevent Pepper's stock dialogue from talking over your LLM (see §1.5).

I.2.4 — The Python 2.7 Constraint Explained

The NAOqi Python binding (`pynaoqi`) is a platform-specific archive shipped on SoftBank's developer portal. It is a CPython extension that **depends on the binary layout of Python 2.7 specifically**. There is no port to Python 3 that exposes the full service surface.

A community-built `qi 3.1.5` package (`pip install qi==3.1.5`) exists for Python 3 on Linux `x86_64`, but several services are broken: touch detection, audio callbacks, certain event subscriptions. It is unsuitable for production use.

The conclusion: **NAOqi requires Python 2.7. Modern AI libraries require Python 3.11+. They cannot live in the same process.** Hence the two-process HTTP bridge of Chapter 16 and §1.4 below.

I.2.5 — Lifecycle Quirks

Two NAOqi quirks that will trip you up:

Stiffness must be enabled before any movement. A fresh-booted Pepper has zero stiffness — its motors are dead weight. You must call `motion.wakeUp()` (or `motion.setStiffnesses("Body", 1.0)`) first. `ALAnimatedSpeech` will speak without stiffness, but the body language will not animate.

ALAudioRecorder and ALSpeechRecognition cannot share the microphone. They both subscribe to the audio device exclusively. You must `unsubscribe` one before using the other. OmniLLM uses neither directly — it captures via `ALAudioDevice` and sends the bytes to Whisper.

I.2.6 — Connection From Your PC

To connect from your PC, you need:

1. **Network access.** Same Wi-Fi as Pepper, or wired to the same LAN.
2. **Pepper's IP address.** Press the chest button once and Pepper says it ("My IP address is 192.168.1.100").
3. **The pynaoqi SDK.** Downloaded from SoftBank's developer portal (or the Maxvision mirror — see §1.5), installed at `C:\pynaoqi\pynaoqi-python2.7-2.5.5.5-win32-vs2013\`.
4. **Python 2.7** at `C:\Python27\python.exe`.
5. **The right PYTHONPATH:**

```
set PYTHONPATH=C:\pynaoqi\pynaoqi-python2.7-2.5.5.5-win32-vs2013\lib;%PYTHONPATH%
set PATH=C:\pynaoqi\pynaoqi-python2.7-2.5.5.5-win32-vs2013\lib;%PATH%
```

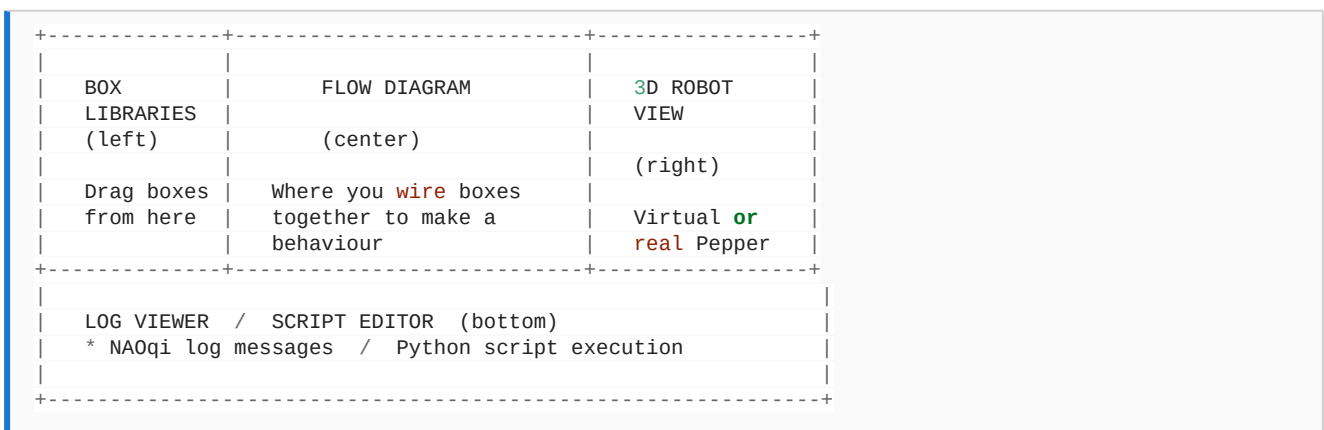
Then `import naoqi` or `import qi` will work.

I.3 — Choregraphe: The Visual Programming Studio

At a Glance

Choregraphe is SoftBank's desktop IDE for Pepper / NAO. Box-and-wire visual programming + Python script editor + 3D virtual robot simulator. Critical for OmniLLM in three ways: (1) testing behaviours without a physical robot, (2) installing custom animations the gesture planner will trigger, and (3) live-monitoring during experimental sessions.

I.3.1 — The Four-Panel Layout



It is locked to NAOqi 2.5 — version 2.5.5.5 or 2.5.10/11 for Pepper. Newer NAOqi 2.9 (Android-based) does **not** support Choregraphe; for 2.9 use QiSDK (Java/Kotlin) instead.

I.3.2 — When to Use Choregraphe Versus OmniLLM

Choregraphe and OmniLLM serve different purposes — they are friends, not substitutes:

You want to...	Use Choregraphe	Use OmniLLM
Test if Pepper's speech works	yes	—
Test a single gesture animation	yes	—
Build / tune a custom animation	yes	—
Build a <i>scripted</i> interaction	yes	—
Build an AI-driven conversation	—	yes
Use multiple LLMs as backends	—	yes
Run a controlled HRI experiment	—	yes

Live-monitor during an experiment	yes (alongside)	yes
-----------------------------------	-----------------	-----

The typical combined workflow:

1. **Plan the gesture vocabulary** in Choregraphe. Drag and edit animations until they look natural.
2. **Install the custom behaviours** on Pepper (File -> Build Application Package, then upload).
3. **Add the new gesture names** to OmniLLM's GESTURE_TO_BEHAVIOR mapping in `naoqi_client.py` and to the planner in `gesture_planner.py`.
4. **Run the OmniLLM experiment**, leaving Choregraphe open as a monitor.

I.3.3 — Connecting Choregraphe to a Robot

Virtual robot (no hardware needed):

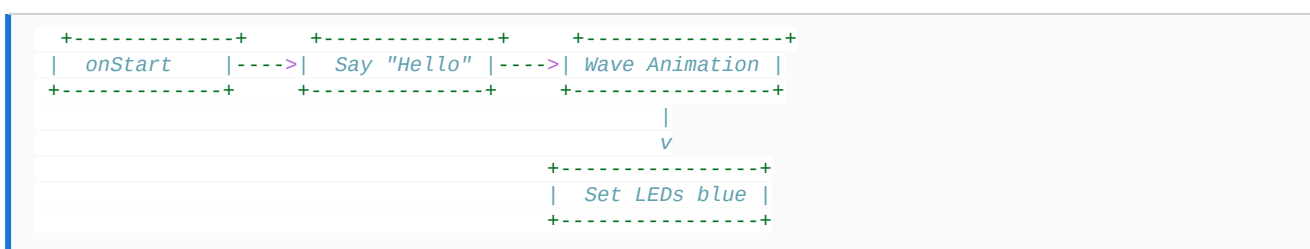
1. Open Choregraphe.
2. **Connection -> Connect to virtual robot.**
3. Pepper appears in the 3D view; the bottom-left status shows `Connected to localhost:<port>` (port is randomised per launch — note it for `--robot-port`).

Physical robot:

1. PC and Pepper on the same Wi-Fi.
2. Press Pepper's chest button -> it says its IP.
3. **Connection -> Connect to...** enter the IP, port 9559.
4. The 3D view now mirrors the real robot's joint positions.

I.3.4 — Box-and-Wire Programming

Each Choregraphe **box** is a small piece of behaviour. Boxes have input and output **bangs** (signal triggers). You drag boxes onto the flow diagram and connect their bangs to define the order:



Inside each box is a Python 2.7 script with `onLoad()`, `onUnload()`, `onInput_onStart()`, and `onInput_onStop()` lifecycle methods. You can inspect and edit any box's script.

I.3.5 — Useful Boxes for OmniLLM Work

When testing Pepper for OmniLLM, the boxes you'll reach for are:

- **Speech / Animated Say** — sanity check that `ALAnimatedSpeech` works.
- **Movement / Animations / Gestures / Hey_1** — wave hello.

- **Movement / Animations / Gestures / Explain_8** — point left (OmniLLM's `point_left`).
- **Movement / Animations / Gestures / Explain_7** — point right.
- **LEDs / Set LEDs** — change eye colour.
- **Movement / Postures / Stand** — wake up posture.

I.3.6 — The Script Editor as a Quick Test Bench

You don't need to drag boxes for everything. Press `Alt+5` to open the Script Editor and just type Python:

```
# Test ALAnimatedSpeech directly
tts = ALProxy("ALAnimatedSpeech", "localhost", 9559)
config = {"bodyLanguageMode": "contextual"}
tts.say("This is what an OmniLLM response sounds like.", config)

# Test a gesture
behavior = ALProxy("ALBehaviorManager", "localhost", 9559)
behavior.runBehavior("animations/Stand/Gestures/Explain_8") # point_left

# Test eye LEDs
leds = ALProxy("ALLeds", "localhost", 9559)
leds.fadeRGB("FaceLeds", 0.0, 0.67, 1.0, 0.5) # navigation blue
```

This is the fastest way to verify everything OmniLLM will need.

I.3.7 — Installing a Custom Behaviour

If you build a new animation in Choregraphe and want OmniLLM to trigger it:

1. **In Choregraphe:** design the animation in the Timeline Editor, save the project as `MyBehaviors/WelcomeDance`.
2. **Upload to Pepper:** File -> Upload to robot... The behaviour now lives at `mybehaviors/WelcomeDance` on the robot.
3. **In OmniLLM:** edit `omni_llm/server/naoqi_client.py`:

```
GESTURE_TO_BEHAVIOR = {
    # ... existing entries ...
    "welcome_dance": "mybehaviors/WelcomeDance", # <-- add this
}
```

4. **Trigger it:** edit `omni_llm/robotics/gesture_planner.py` to map appropriate response text or task type to `"welcome_dance"`.

I.4 — Five Bridge Patterns From the Literature

Chapter 16 introduced OmniLLM's three bridge solutions (HTTP server, HTTP client, stub). This section surveys the five published patterns used across 15+ Pepper-LLM projects and explains why OmniLLM chose Pattern 1.

I.4.1 — The Five Patterns

Pattern	Used in	Trade-offs
1. HTTP / REST bridge (OmniLLM)	ilabsweden/pepperchat (2023), Frontiers ASD therapy, 6+ others	Easiest to debug, well-understood, ~50–200 ms overhead
2. Socket-based	Pepper-GPT (Auckland), Ghent University elder care	Lower latency (~10–50 ms), more code
3. ROS2 bridge (<code>naoqi_driver2</code>)	Multi-robot research projects	High setup complexity, powerful for fleets
4. MQTT broker	LAIR-GPT (Ancona)	Good when many components publish/subscribe
5. Python 3 <code>qi 3.1.5</code>	Prototypes	Single process but several services broken

OmniLLM picked **Pattern 1** because it is the most-tested in the literature, the easiest to debug (every message is just `curl`-able), and the overhead is comfortably within the latency budget.

I.4.2 — Failure Modes the Bridge Must Handle

The HTTP bridge introduces three new failure surfaces. Each has a defensive fallback:

Failure	Mitigation
AI server crashed	NAOqi client times out, says "I could not connect to my AI brain" — does not crash
Network partition	Same as above; the <code>urlopen</code> timeout is 30 s
AI server returns malformed JSON	NAOqi client logs and falls back to silent failure
Audio capture returns empty bytes	Server can fall back to text-only mode (text field also accepted)
LangGraph not installed on AI server	<code>_fallback_interact</code> direct gateway call still works

Whisper not installed	Server returns 500 on <code>/transcribe</code> ; text-only mode still works
-----------------------	---

I.4.3 — Latency Implications

The HTTP bridge adds a small but measurable overhead per interaction:

Step	Time on LAN
TCP/HTTP round trip (LAN)	5–30 ms
JSON serialise / deserialise	1–5 ms
Base64 encode / decode (5 s of audio at 16 kHz mono)	10–20 ms
Total bridge overhead per interaction	~20–50 ms

This is comfortably inside the 1–3 s budget. For comparison, the LLM call itself takes 500–2000 ms.

I.4.4 — When You'd Want a Different Pattern

Switch off the HTTP bridge if:

- You need **sub-100 ms** end-to-end (token streaming for real-time conversation). Use a WebSocket pattern.
- You're on **NAOqi 2.9 + Android**. Use QiSDK (Java/Kotlin), bypass the bridge entirely.
- You're integrating with **ROS2 Nav2**. Use `naoqi_driver2` so the robot participates in the ROS2 message graph.

For a typical Pepper + LLM HRI study, the HTTP bridge is the sweet spot.

I.4.5 — Anatomy of One HTTP Round-Trip

Request from `naoqi_client.py`:

```
POST /interact HTTP/1.1
Host: 192.168.1.50:5000
Content-Type: application/json
Content-Length: ~140000      # about 100 KB of base64 audio

{
  "audio":          "UklGRiQ...",    # 5 s of 16 kHz WAV ~ 100 KB raw -> ~135 KB b64
  "participant_id": "P001",
  "session_id":     "9c2e-abc123",
  "condition":      "C",
  "rag_enabled":    true
}
```

Response from `app.py`:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "speech": "Room 305 is on the third floor on your left.",
  "gesture": "point_left",
  "emotion_led": "#00AAFF",
  "metadata": {
    "task_type": "navigation",
    "model_id": "openai-gpt4o-mini",
    "rag_enabled": true
  }
}
```

I.5 — Sibling HTTP-Bridge Projects: Cross-Validation

§1.4 chose Pattern 1 (HTTP/REST bridge) by surveying the literature. Five independent public projects use the same pattern. Reading their READMEs is the fastest way to cross-check OmniLLM's architecture and spot ideas it has not yet adopted.

Project	Stack	What OmniLLM can borrow
ilabsweden/pepperchat	Py2 <code>module_commandable.py</code> on robot, Py3 <code>dispatcher.py</code> external, OpenAI ChatGPT	Uses NAOqi <code>ALAutonomousLife</code> to switch focus to a dedicated <code>nao_focus</code> — prevents Pepper's built-in dialogue from talking over your LLM. OmniLLM does not currently set Autonomous Life mode; the participant may hear Pepper's stock greeting drowning the LLM response. Fix is one line in <code>naoqi_client.py</code>: <code>ALAutonomousLife.setState("disabled")</code> after connect, before the greeting.
UoA-CARES/Pepper-GPT	Py3 "Black Box" (Whisper + GPT-3.5), Py2 "Pepper Controller" over VPN, NAOqi 2.1.4.13	Documents the <code>libboost_regex</code> install error explicitly — the same family of errors hits the Windows 11 <code>pynaoqi</code> install. Their pinning of NAOqi 2.1.4.13 confirms our 2.5.5.5 choice is <i>not</i> the only valid path.
UoA-CARES/pepper-demo	Choregraphe 2.5.10.7 + PyNAOqi, Pepper 1.8	Documents a hard-to-find ZLIB symlink fix (<code>libz.so.1</code> ↔ system) for Choregraphe on Linux. Save this for the day you move the AI server to a Linux box.
igor-lirussi/Dialogue-Pepper-Robot	Java AIML engine + Py2 NAOqi + separate speech-recognition service	The clean split between <i>dialogue engine</i> and <i>speech-recognition service</i> (parallel processes, robot IP as a CLI flag) is what OmniLLM already does; their architectural rule is worth citing in the thesis.

softbankroboticstraining/pepper-chat (Pepper Chat)	NaDai-2.5 + Google Dialogflow v2 (JSON keypath)	Documents QiChat voice-shaping commands (pause, speed, pitch, emotional intonation) that OmniLLM does not yet exercise. These could be added to <code>naoqi_client.py</code> 's <code>say()</code> wrapper for affective speech without leaving Pepper's TTS.
---	--	--

Direct cross-validation finding. Every one of the five projects ends up with the same two-process topology OmniLLM uses. The *variation* is in (a) which LLM provider they use, and (b) whether they suppress Pepper's Autonomous Life default behaviour. OmniLLM is **ahead** on (a) — its multi-provider router covers ground none of the others touch — and **behind** on (b). One-line fix: disable Autonomous Life on connect.

I.6 — Curated External Resources (Pepper-Specific)

Resources organised by the recurring problem buckets every Pepper + LLM project meets. Each entry: link + one-line "why it matters."

NAOqi 2.5 / Python 2.7 install hardening

Resource	Why it matters
AnonKour/pynaoqi (GitHub)	Linux mirror of the Python 2.7 NAOqi SDK; keep the URL in case the official mirror disappears post-Aldebaran bankruptcy
nlp.fi.muni.cz/trac/pepper/wiki/InstallationInstructions	Massachusetts Institute of Technology NLP lab guide — <code>pyenv_install.sh</code> for isolated Python 2 (Anaconda is incompatible with NAOqi)
incognite-lab/Pepper-Controller	Wraps ~60 NAOqi methods into a single domain-partitioned Python class — a model for refactoring <code>naoqi_client.py</code>
maxtronics.com/en/support/kb/category/pepper/downloads/	Post-bankruptcy mirror of Pepper 2.5 & 2.9 downloads; most reliable place to re-fetch Choregraphe and the SDK

Navigation, mobility, guided-tour extensions

Resource	Why it matters
softbankrobotics-labs/pepper-proactive-mobility	Pepper autonomously approaches people and returns home; supports three localisation modes (homing, ARUCO, on-board SLAM)
softbankrobotics-labs/pepper-aruco and pepper-aruco-automapping	ARUCO marker library through QiSDK; print one marker per "room" and Pepper can both speak <i>and</i> walk to "Room 305"
aldebaran/naoqi_navigation_samples	Choregraphe <code>.pml</code> projects for <code>explore</code> , <code>patrol</code> , and <code>places</code> . <code>places</code> is the most directly useful — named locations + walk-between
ros-naoqi/naoqi_bridge and naoqi_driver2	Republishes NAOqi services as ROS topics; entry into Nav2

Multilingual voice (Japanese-voice question)

Resource	Why it matters
VOICEVOX (open-source)	MIT-licensed modern neural TTS for Japanese; runs locally on the same GPU you use for Whisper. Practical 2026 choice for Japanese.
Open JTalk + HTS voices	Alternative open Japanese TTS; older HTS-HMM family
Voisona Pepper voice library	Pepper-character <i>singing-voice</i> library; NOT suitable for conversational TTS — only for outreach demos

Vision-side extensions

Resource	Why it matters
ageitgey/face_recognition	dlib-based, 99.38% on LFW. Python 3 only. Practical choice for a /recognise_face endpoint that returns a participant ID
LucaCorvitto/Emotional_Pepper	PDDL-planned emotional behaviour — hybrid LLM-content + planner-mood inspiration
softbankrobotics-labs/pepper-mask-detection-and-pepper-deep-learning	NAOqi 2.9 / QiSDK only — relevant if migrating to NAOqi 2.9

Dialogue platforms and ASR (pre-LLM, still instructive)

Resource	Why it matters
softbankroboticstraining/pepper-chatbot-api	Documents QiChat voice-shaping (pause, speed, pitch, emotional intonation) — affective TTS extension
TheRARELab/langex	Choregraphe template for reproducible language-HRI experiments — borrow filename and header conventions
softbankrobotics-labs/pepper-solitaires-loop	Idle animations for between-turn liveliness — fixes the uncanny-when-still-between-turns problem
Pepper + Dialogflow integration (blogemtech Medium)	Documents silence detection (essential for variable-length turns) + amplitude threshold values (14000 at 16 kHz mono)

Validation suites, curricula, HRI reference datasets

Resource	Why it matters
robocupathome.edu.org RoboCup@Home Education	External rubric of service-robot tasks (person following, object handover, instruction following, room-to-room navigation) — use to scope thesis claims
ROBO-GAP (Perugia et al. 2022, HRI ACM/IEEE) — robo-gap.unisi.it	Peer-reviewed dataset of perceived age, femininity, masculinity, gender-neutrality across 251 robots including Pepper . Use as a control variable in the thesis . ICC reliability 0.896–0.954.
CARESSES (caressesrobot.org)	Pepper used for <i>culturally competent</i> elder care (UK / Japan / India) — entry point for cultural-HRI framing. Cite the Bruno/Sgorbissa publications via Google Scholar.

Aldebaran / NAOqi reference docs

Resource	Why it matters
doc.aldebaran.com/2-5/	Canonical NAOqi 2.5 reference — bookmark the Service Pages (ALMotion, ALAnimatedSpeech, ALMemory, ALAudioDevice)
doc.aldebaran.com/2-5/getting_started/index.html	The "first 10 minutes with Pepper" reference
github.com/orgs/aldebaran/repositories	<code>libqi</code> , <code>libqi-python</code> , <code>qibuild</code> (still maintained April 2026); <code>robot-jumpstarter</code> is the best Python starter
groups.google.com/g/ros-sig-aldebaran	Slow but active community list — best place to ask <code>naoqi_driver2</code> questions

Emotion-adaptive proxemics (research-grade extension)

Resource	Why it matters
arxiv.org/abs/2401.17663 (Bilen et al. 2024)	"Social Robot Navigation with Adaptive Proxemics Based on Emotions" — empirical basis for tying detected emotion to approach distance (relevant if you build the Wayfinder extension in §1.4 / Chapter 33)

I.7 — A One-Liner for Each Quick-Win Improvement

Ranked by ease and research payoff:

Improvement	Effort	Research payoff	Source
Disable Autonomous Life on connect (kill stock dialogue)	1 line in <code>naoqi_client.py</code>	Removes a confound that all five sibling projects have already fixed	§1.5, ilabsweden
Add solitaires / idle animations loop	~30 LoC	Perceived liveliness during long sessions	§1.6, SoftBank Labs
Silence-detected audio capture (replace fixed 5 s window)	~80 LoC	Lets the participant pause/think without truncation	§1.6, Dialogflow article
VOICEVOX Japanese TTS provider	new <code>tts/voicevox.py</code>	Unlocks Japanese-language Arena condition with modern voice	§1.6
Face-recognition endpoint (<code>/recognise_face</code>) via <code>ageitgey/face_recognition</code>	new endpoint + Py3 dependency	Eliminates manual participant-ID entry; enables personalised greeting	§1.6
ARUCO + <code>places/</code> navigation as a new task type	new task + 4 Choregraphe behaviours	Tests escorting vs. pointing — novel HRI finding	§1.6
ROS 2 bridge via <code>naoqi_driver2</code>	new <code>naoqi_client_ros2/</code> package	Opens fleet / multi-robot experiments	§1.6
Migration to NAOqi 2.9 + QiSDK (Kotlin)	full robot-side rewrite	Future-proofs against <code>pynaoqi</code> rot	§1.6

The first three rows are quick wins worth doing before the next participant cohort. Rows 4–6 are dissertation-chapter-scale extensions. Rows 7–8 are post-thesis directions.

Appendix J — AI-Stack Library Rationale

Chapter 8 listed every direct dependency in a one-line table. This appendix expands each one into a paragraph: what it is, why we picked it, what we'd have used otherwise, and where in the codebase it lives. Read this when you need to defend a tooling choice or evaluate a replacement.

J.1 — LiteLLM

What it is. A Python library that exposes 100+ LLM providers behind one identical function call. You pass `model="openai/gpt-4o"` Or `model="anthropic/claude-haiku"` Or `model="ollama/llama3:8b"` — the call signature is the same.

Why we picked it. Without it, OmniLLM would need separate adapter code for each provider, each with its own pagination, error-handling, and token-counting quirks. LiteLLM normalises this in one library that's actively maintained by BerriAI. The cost-per-token table is built in, the async API is uniform, and the provider prefix system (`ollama/`, `gemini/`, `anthropic/`) is intuitive once you've learned it.

Alternatives considered. Direct provider SDKs (`openai`, `anthropic`, `google-generativeai`) — more control, more code; the gateway alone would have been 800 lines instead of 270. `aisuite` (Andrew Ng's lightweight alternative) — smaller, but covers fewer providers and no Ollama. `openrouter` — a paid hosted gateway with similar surface but you must route through their cloud. LiteLLM was the lowest-friction choice for a self-hosted setup.

Where used. Imported by `omnilm/gateway.py`. Every LLM call in the entire project funnels through `litellm.acompletion(...)`.

J.2 — LangGraph

What it is. A library on top of LangChain for building **stateful agent graphs** — directed graphs whose nodes are async functions sharing a single state dict, with conditional edges between them. Visualisable as Mermaid or Graphviz at runtime.

Why we picked it. OmniLLM's HRI pipeline is genuinely a graph (transcribe -> detect -> classify -> branch -> answer -> plan -> log) with conditional routing. Writing this as one 200-line `async` function with `if/elif` branches would work but be untestable, unreadable, and impossible to visualise. LangGraph gives us named nodes, a typed state dataclass, and `add_conditional_edges` for the T1/T2/T3/T4 branch point.

Alternatives considered. **Plain async functions** — more verbose, no diagram. **LangChain's deprecated `AgentExecutor`** — limited routing, deprecated in favour of LangGraph itself. **Custom DAG libraries** (prefect, airflow) — overkill, batch-oriented, don't compose well with `async`. LangGraph was the only framework purpose-built for `async` LLM-node graphs.

Where used. [omnilm/hri/agent_graph.py](#). The 9-node pipeline that is Pepper's brain.

J.3 — ChromaDB

What it is. A pure-Python embedded vector database. Stores embeddings + their source text + metadata in a local SQLite file. Supports cosine similarity search with HNSW indexing.

Why we picked it. Embedded (no separate process), persistent across restarts, zero configuration, pip-installable. For a single researcher with a < 10 k-document knowledge base, this is plenty. The fallback path — keyword search over in-memory documents — also lives in the same file (`RAGPipeline._retrieve_keyword`), so the rest of the pipeline is unaffected if ChromaDB fails to load.

Alternatives considered. **Qdrant**, **Weaviate**, **Pinecone** — heavier, require running a server (Docker or cloud). **FAISS** — fast but lower-level, no metadata filtering, requires a C++ build chain. **LanceDB** — newer, comparable; we briefly evaluated it but ChromaDB's documentation maturity won. For OmniLLM's ~50-chunk DIBRIS knowledge base, the speed difference between any of these is irrelevant.

Where used. [omnilm/rag/pipeline.py](#). One collection per running pipeline.

J.4 — LangChain

What it is. A broader framework for LLM apps. OmniLLM uses **only** its document loaders (PyPDFLoader, CSVLoader, TextLoader) and the RecursiveCharacterTextSplitter — **not** the larger Chain or Agent abstractions, which are opinionated and easy to outgrow.

Why we picked it. The document loaders save dozens of lines per file format. They're tested by a large community and they normalise output so each chunk has `.page_content` and `.metadata` — making the rest of the RAG pipeline cleaner. The text splitter handles edge cases (Unicode normalisation, sentence-boundary detection) that we'd otherwise re-implement.

Alternatives considered. Write our own loaders. For PDFs, use `pypdf` directly (which we already do as a fallback). For CSVs, use `pandas`. We could do this — but LangChain's loaders normalise the output shape, which makes downstream code simpler. We deliberately do *not* use LangChain's `chain` classes, `LCEL`, or `RunnableLambda` — those abstractions are powerful but opinionated, and LangGraph (J.2) is the more appropriate orchestration layer.

Where used. [omnillm/rag/pipeline.py](#) for document loading and chunking.

J.5 — sentence-transformers

What it is. A Python library that turns text into embedding vectors using small (~100 MB), fast, locally-runnable models. Default model: `all-MiniLM-L6-v2` (384-dimensional embeddings, 23 MB on disk).

Why we picked it. ChromaDB's default embedding is `all-MiniLM-L6-v2` from sentence-transformers — **fast (~5 ms per short text on CPU), small, and free**. No need to pay OpenAI for embeddings during indexing. The model is multilingual-aware enough for the DIBRIS knowledge base.

Alternatives considered. **OpenAI's `text-embedding-3-small` API** — higher-quality embeddings (~10–15% better on the MTEB benchmark), \$0.02 per million tokens, requires internet. For a small lab KB of ~50 chunks, the quality difference does not justify the cost or the cloud round-trip. **Cohere embeddings** — similar trade-off. **BGE-large** (sentence-transformers can load it) — 5× larger, ~3× slower, marginally better; not worth it at our scale.

Where used. Indirectly via ChromaDB.

J.6 — OpenAI Whisper

What it is. A speech-to-text model trained by OpenAI on 680,000 hours of multilingual audio. Available as a downloadable model (`pip install openai-whisper`) **or** via the OpenAI cloud API. OmniLLM uses the local download.

Why we picked it. Best-in-class accuracy at the **multilingual** level, robust to noise, free if you run it locally. Pepper's onboard `ALSpeechRecognition` is widely regarded as unusable for open-domain speech. Running Whisper locally also keeps participant audio off third-party servers — a meaningful GDPR concern.

Alternatives considered. **Vosk** (lighter, fully offline, lower accuracy on Italian/French). **Google Speech-to-Text API** (cloud, faster, GDPR-flagged). **Faster-Whisper** (a 4× faster C++ port — recommended for production deployments but adds setup complexity). For a research prototype, plain Whisper is the right pick.

Where used. `omnillm/robotics/whisper_stt.py`. Called by the agent graph's `transcribe_audio` node and by the `/transcribe` HTTP endpoint.

J.7 — asyncio

What it is. Python's [standard-library](#) module for non-blocking I/O. `async def` declares a coroutine; `await` pauses it until an I/O operation completes; `asyncio.gather(...)` runs many coroutines concurrently.

Why we picked it. LLM calls are I/O-bound — most of the time is spent waiting for the network. Asyncio lets us query 5 models in the wall-clock time of the slowest one, not the sum. Condition D's 3-model council finishes in ~1.8 s instead of ~5.4 s thanks to this single design choice.

Alternatives considered. **Threading** — works but introduces GIL contention and shared-state bugs. **multiprocessing** — overkill for I/O work, slow startup, no easy state sharing. **Trio** / **Curio** — better async APIs in some respects but smaller ecosystems; LangGraph and litellm both use asyncio. For pure-I/O work, asyncio wins.

Where used. Pervasive — [gateway.py](#), [consensus.py](#), [evaluator.py](#), [agent_graph.py](#), [server/app.py](#), [robotics/pepper.py](#).

J.8 — Flask

What it is. A minimalist Python web framework. Serves HTTP endpoints with decorators (`@app.route("/path", methods=[...])`).

Why we picked it. Tiny, well-documented, easy to test, single-worker is the right concurrency model for one robot. The synchronous-to-async bridge (`_run_async`) inside the Flask app is six lines of code.

Alternatives considered. **FastAPI** (async-native, faster, more modern) — genuinely better for high-throughput services with auto-generated OpenAPI docs. We didn't pick it because Flask's simplicity matched the single-robot single-worker use case, and the dependency footprint is smaller (Flask + Jinja vs FastAPI + Pydantic + Starlette + uvicorn). **Bottle** — even smaller than Flask but less mature ecosystem. **Django** — orders of magnitude too heavy. If you ever serve many robots simultaneously, swap to FastAPI + uvicorn (the migration is ~30 lines).

Where used. [omnilm/server/app.py](https://github.com/omnilm/server/app.py). Six endpoints.

J.9 — Click

What it is. A Python library for building command-line interfaces with decorators (`@click.command`, `@click.option`, `@click.group`). Auto-generates `--help`.

Why we picked it. Better than `argparse` (less boilerplate per command), supports nested command groups (`omnillm ask`, `omnillm evaluate`, `omnillm leaderboard`), auto-generates type-aware `--help` text, and integrates cleanly with `rich` for coloured output.

Alternatives considered. **Typer** (built on Click, uses type hints — slightly nicer for greenfield projects). Genuinely tempting; we stayed on Click for consistency with the broader Python ecosystem where Click is more familiar. **argparse** (stdlib, more verbose). **Fire** (auto-generates CLI from any Python class — magic, opinionated).

Where used. `omnillm/cli.py`. Every `omnillm` subcommand.

J.10 — Rich

What it is. A Python library for beautiful coloured terminal output — tables, panels, progress bars, syntax-highlighted code, markdown rendering, tracebacks.

Why we picked it. OmniLLM's CLI is a research dashboard. A coloured table of model latencies is dramatically more readable than `print(dict)`. Rich is the de-facto standard for Python CLI UX in 2026.

Alternatives considered. `tabulate` (tables only, ASCII-art). `prettytable` (similar to `tabulate`). `Plain print()` (grey, unreadable for long output). None of these get close to Rich's quality.

Where used. `omnillm/cli.py`. Every command that emits structured output.

J.11 — Ollama

What it is. A free local LLM runtime. `ollama serve` starts a daemon on `localhost:11434`; `ollama pull llama3:8b` downloads a model; `ollama list` shows what's installed. Exposes an OpenAI-compatible HTTP API.

Why we picked it. Free, private, fast on a laptop with a GPU (and acceptable on CPU). The Condition B (Fixed Local LLM) experimental condition **depends on it** — without Ollama there is no "no-API-key baseline." The OpenAI-compatible API means LiteLLM speaks to it via the same `openai/` prefix machinery.

Alternatives considered. **vLLM** (faster, GPU-required, more setup). **LM Studio** (GUI, less scriptable). **GGUF + llama.cpp** directly (more control, more setup). **Text-Generation-WebUI** (heavyweight). Ollama is the easiest for a research lab and the only one with one-line install on Windows.

Where used. Via LiteLLM in `omnilm/gateway.py`. Models registered in `config/models.yaml` with `provider: ollama`. Required for Condition B.

J.12 — ReportLab + xhtml2pdf

What they are. Two pure-Python libraries that together produce PDFs from HTML. **xhtml2pdf** orchestrates: it accepts an HTML string + CSS and emits a PDF. **ReportLab** does the actual page-laying-out. We register **DejaVu Sans + DejaVu Sans Mono** with ReportLab so Unicode glyphs (arrows, box-drawing characters, accented Latin) render correctly inside the PDF.

Why we picked them. No native dependencies (no `libgobject`, Pango, Cairo, or `wkhtmltopdf` binary) — installable on Windows with pip alone. WeasyPrint produces visually better output but requires GTK on Windows which is a multi-hour install per machine. `xhtml2pdf` is the **lowest-friction choice** for cross-platform PDF generation.

Alternatives considered. **WeasyPrint** (best quality, needs GTK). **wkhtmltopdf** (good quality, separate ~100 MB binary, increasingly bit-rotted). **mPDF** (PHP). **LaTeX** (best quality, huge install, complex source files). For a thesis-supporting reference book that must be rebuildable in five minutes on any machine, `xhtml2pdf` wins.

Where used. `book2/build_pdf.py`. Run after editing any of the 16 source files.

J.13 — pytest, pytest-asyncio, pytest-mock

What they are. The Python testing trinity. **pytest** is the framework; **pytest-asyncio** runs `async def test_...` functions; **pytest-mock** provides the `mock` fixture for patching dependencies.

Why we picked them. Standard in the Python ecosystem. The 289 tests in `tests/` run in ~5 seconds without a single real API call — because `pytest-mock` fakes out the LLM calls and the robot bridge. This is how the codebase can be CI'd on a laptop with zero LLM cost and zero hardware.

Alternatives considered. **unittest** (stdlib, more verbose, worse fixtures). **nose2** (largely abandoned). **Hypothesis** for property-based tests (we use a tiny bit, indirectly, via `pytest-randomly` for ordering robustness). `pytest` is the only framework in active mainstream Python use.

Where used. `tests/`. All 11 test files.

J.14 — pyyaml

What it is. Python's YAML parser. Reads `.yaml` files into nested Python dicts/lists.

Why we picked it. YAML is the right format for the model registry and benchmark tasks: human-editable, comment-friendly, supports lists and nested dicts without the verbosity of XML. `pyyaml` is the canonical Python parser.

Alternatives considered. `ruamel.yaml` (preserves comments on round-trip — useful for code-modifying-YAML, irrelevant for us). **TOML** (no nested-list support, weaker for the routing config). **JSON** (no comments — disqualifying for human-edited configs).

Where used. `omnillm/gateway.py` and `omnillm/router.py` load `config/models.yaml`.

J.15 — python-dotenv

What it is. Loads environment variables from a `.env` file into the process's `os.environ`. Called once at startup.

Why we picked it. API keys must never be hard-coded or committed to git. The standard pattern is `.env.example` (committed, no real keys) plus a local `.env` (in `.gitignore`, contains real keys). `python-dotenv` reads `.env` so application code can `os.environ.get("OPENAI_API_KEY")` without knowing where it came from.

Alternatives considered. **Reading the keys from a YAML config** — pulls them into a file format we already use, but makes accidental commits more likely. **OS-level env vars** — works but requires the user to set them in their shell profile, which is platform-specific.

Where used. Imported once in `omnillm/cli.py` and `omnillm/server/app.py`.

J.16 — aiohttp

What it is. A Python library for asynchronous HTTP — both client and server. The async equivalent of `requests`.

Why we picked it. [omnilm/robotics/pepper.py](https://github.com/omnilm/robotics/pepper.py) is async (because LangGraph is async). The bridge client therefore needs an async HTTP library. `requests` is synchronous and would block the event loop.

Alternatives considered. `httpx` (similar feature set; we briefly evaluated it). **Curl-via-subprocess** (would work, ugly). `aiohttp` was the first to mature and is what LiteLLM also uses internally.

Where used. [omnilm/robotics/pepper.py](https://github.com/omnilm/robotics/pepper.py) — `_post`, `_get`, `_ping_bridge_server`.

J.17 — markdown (the python-markdown library)

What it is. Python's most-popular Markdown -> HTML converter. Supports extensions: `tables`, `fenced_code`, `codehilite` (syntax highlighting), `toc` (auto table of contents), `sane_lists`.

Why we picked it. Used by [book2/build_pdf.py](#) to turn the assembled `OmniLLM_Book.md` into HTML before `xhtml2pdf` renders it to PDF. The extension list above is the canonical "reads like GitHub Markdown" config.

Alternatives considered. `mistune` (faster). `commonmark-py` (stricter CommonMark). `pandoc` (best quality but requires the pandoc binary, defeats the pure-Python promise). For our scale, the difference is invisible.

Where used. [book2/build_pdf.py](#).

J.18 — pypdf

What it is. Python library for reading and (limited) writing of PDF files. OmniLLM uses only the reading side: extracting text per page from PDF documents in the knowledge base.

Why we picked it. Pure Python, no native dependencies, handles the PDF formats we encounter. ChromaDB's persistence already pulls in a SQLite dependency; we wanted the PDF reading path to add no new native deps.

Alternatives considered. **PyMuPDF** (`fitz`) — faster, better quality, requires native build chain. **pdfplumber** (better at tables). **pdf2text** subprocess (Linux-only). For occasional PDF indexing during RAG setup, pypdf is sufficient.

Where used. `omnillm/rag/pipeline.py` — `_index_pdf` method.

J.19 — langdetect

What it is. A Python port of Google's language-detection library. Statistical n-gram classifier; supports ~55 languages.

Why we picked it. A backup signal in [omnilm/hri/language_detector.py](#). The first-pass language detection is Unicode-script-based (super fast); the second pass is high-frequency-word matching for Latin-script languages; **langdetect** is the third-tier fallback when the first two fail.

Alternatives considered. **fasttext-langid** (better accuracy, larger model — but adds a 1 GB binary). **whisper-detect-language** (Whisper itself returns a language probability — we could use this, but only after transcription; the rule-based path is faster and works without audio). **langid.py** (similar feature set, less maintained).

Where used. [omnilm/hri/language_detector.py](#) — `_try_langdetect()` fallback.

J.20 — Why We Did NOT Use...

A short anti-resume:

Library	Why we did not use it
LangChain Chains, LCEL, RunnableLambda	LangGraph (J.2) is the more appropriate orchestration layer; LangChain's chains are opinionated and easy to outgrow.
ROS / ROS 2	4–8 GB install footprint; Python version trap (ROS 1 = Py2, ROS 2 = Py3.8+); cross-platform fragility. The HTTP bridge is sufficient at our scale.
Docker	Adds setup friction for a single-developer thesis project. The repo runs with <code>pip install -e .</code> and that's it. A Dockerfile is a one-day add-on if needed.
Pydantic	We use plain <code>@dataclass</code> everywhere. Pydantic gives us runtime validation we don't need for trusted internal data, at the cost of an extra dependency and slower import time.
Celery / RQ	No background-job queue is needed; the interaction-per-request shape fits Flask's sync model.
Redis	No cross-process state to share. ChromaDB's SQLite file is the only persistent state.
PostgreSQL / any RDBMS	All persistent data is small enough to live in JSON-Lines + CSV files. The analysis pipeline (Chapter 30) loads them into pandas.
scikit-learn / PyTorch / TensorFlow	We are <i>evaluating</i> models, not training them. Adding ML frameworks would imply we should be training, which we explicitly are not.

The pattern: **OmniLLM is intentionally small**. Every dependency must earn its place by being the most-direct solution to a real problem, not a "we might need this later" hedge.

Appendix K — Walkthroughs, Feature Catalogue, and Execution Plans

This appendix is for the reader who wants concrete, granular detail. Five sections: a line-by-line trace of one question through the code, a complete worked first-session example, the eight evaluation axes enumerated, a complete feature reference catalogue, and a one-month study execution plan distinct from the 13-week build plan in Chapter 22.

K.1 — Reading the Code: A Single Question's Journey, Line by Line

Take one user input — "Where is Room 305?" — and trace it through every Python function that touches it. If you can read this section top to bottom and recognise every step, you understand OmniLLM.

K.1.1 — The Scenario

You are running:

```
# Terminal 1
python -m omnillm.server.app

# Terminal 2 (drives the AI server with curl)
curl -X POST http://localhost:5000/interact `
-H "Content-Type: application/json" `
-d '{"text": "Where is Room 305?",
    "participant_id": "DEMO",
    "session_id": "demo-1",
    "condition": "C",
    "rag_enabled": true}'
```

What happens between the `curl` and the JSON response?

K.1.2 — Stage 1 — Flask Receives the Request

File: `omnillm/server/app.py`, function `interact()`

Flask matches `POST /interact` to a Python function. The body is parsed from JSON into a dict:

```
@app.route("/interact", methods=["POST"])
def interact():
    data = request.get_json(force=True) or {}
    utterance = data.get("text", "") # "Where is Room 305?"
    audio_b64 = data.get("audio", "") # "" (text-only)
    participant_id = data.get("participant_id", "unknown")
    session_id = data.get("session_id", "")
    condition = data.get("condition", "A")
    rag_enabled = bool(data.get("rag_enabled", True) and rag is not None)
```

Python notes for beginners:

- `@app.route(...)` is a **decorator**. It wraps the function below it with Flask's URL routing logic. The function is still callable directly, but Flask also knows "if a `POST /interact` comes in, call `interact`".
- `data.get("text", "")` returns `data["text"]` if it exists, otherwise `""`. This is **safer than** `data["text"]` because the latter raises `KeyError` if the field is missing.

K.1.3 — Stage 2 — Resolve the Per-Condition Model, Hand Off to LangGraph

Same file, same function (continued)

```
audio_bytes = base64.b64decode(audio_b64) if audio_b64 else b""
```

```

# Resolve the per-condition model BEFORE invoking the graph.
from omnillm.hri.experiment import CONDITION_CONFIGS, ExperimentCondition
cond_cfg = CONDITION_CONFIGS.get(ExperimentCondition(condition))
effective_model = (cond_cfg.model_id if cond_cfg and cond_cfg.model_id
                  else _default_model)

graph = _get_graph()
if graph is not None:
    state = {
        "utterance": utterance,
        "audio_bytes": audio_bytes,
        "participant_id": participant_id,
        "session_id": session_id,
        "condition": condition,
        "rag_enabled": rag_enabled,
        "model_id": effective_model,          # <-- the May-2026 fix
    }
    result = _run_async(graph.ainvoke(state))
    return jsonify(result.get("robot_action",
                             {"speech": result.get("response_text", "")}))

```

What's happening:

- The condition lookup ensures Condition B's `llama3-8b-local` actually reaches the LLM call inside the graph (the May 2026 fix from Chapter 12).
- `_get_graph()` returns a compiled LangGraph DAG. The first call takes ~10 seconds (imports + compile); subsequent calls are instant.
- `graph.ainvoke(state)` runs the **entire graph asynchronously** starting from the state dict. Each node reads keys it cares about and writes back updates.
- `_run_async(coro)` is a tiny helper that creates a fresh event loop per request — needed because Flask is synchronous but LangGraph is asynchronous.

K.1.4 — Stage 3 — Inside the Graph (Node by Node)

The compiled graph:

```

START
v
transcribe_audio
v
detect_language
v
classify_task
v (conditional)
+> rag          (T1, RAG on)
+> nav_rag      (T2, RAG on)
+> direct_llm   (T3 or condition E)
+> multilingual_llm (T4)
v
smart_router
v
generate_action_plan
v
log_interaction
v
END

```

Node 1 — transcribe_audio

```
async def transcribe_audio(state: dict) -> dict:
    audio = state.get("audio_bytes", b"")
    if not audio:
        return {"_start_time": time.monotonic()} # text-only mode
    from omnillm.robotics.whisper_stt import WhisperSTT
    stt = WhisperSTT()
    utterance = await stt.transcribe(audio)
    return {"utterance": utterance, "_start_time": time.monotonic()}
```

Because we passed `text="Where is Room 305?"` (no audio), the node just records the start time and passes through. `_start_time` is what `log_interaction` later subtracts to compute end-to-end latency.

Node 2 — detect_language

```
async def detect_language(state: dict) -> dict:
    utterance = state.get("utterance", "")
    from omnillm.hri.language_detector import LanguageDetector
    detector = LanguageDetector()
    result = detector.detect(utterance)
    return {"detected_language": result.language_code}
```

The detector inspects the script and word frequencies and returns `"en"`. Almost any English text is identified in under 10 ms by Tier 1 (Unicode script check) — no LLM call required.

Node 3 — classify_task

```
async def classify_task(state: dict) -> dict:
    utterance = state.get("utterance", "")
    language = state.get("detected_language", "en")
    from omnillm.hri.classifier import HRITaskClassifier
    classifier = HRITaskClassifier()
    result = classifier.classify(utterance, detected_language=language)
    return {"task_type": result.task_type.value,
            "task_confidence": result.confidence}
```

The classifier hits the regex `\broom\s+\d+\b` ("Room 305") and the keyword `where`. Score for **navigation** wins -> returns `{"task_type": "navigation", "task_confidence": 0.95}`.

Conditional Edge — Pick the Branch

```
def _route_by_task_type(state: dict) -> str:
    task_type = state.get("task_type", "info_retrieval")
    condition = state.get("condition", "A")
    rag_enabled = state.get("rag_enabled", True)
    if condition == "E":
        return "direct_llm" # E always skips RAG
    routing = {
        "info_retrieval": "rag" if rag_enabled else "direct_llm",
        "navigation": "nav_rag" if rag_enabled else "direct_llm",
        "social_conversation": "direct_llm",
        "multilingual": "multilingual_llm",
    }
    return routing.get(task_type, "direct_llm")
```

`task_type="navigation" + rag_enabled=True + condition="C" -> returns "nav_rag"`. LangGraph dispatches to the `nav_rag` node.

Node 4 — `nav_rag` (RAG + Gesture Planner)

```
async def run_nav_rag(state: dict) -> dict:
    utterance = state.get("utterance", "")
    target_model = state.get("model_id")
    rag_resp = await rag.query(utterance, model_id=target_model)
    from omnillm.robotics.gesture_planner import GesturePlanner
    planner = GesturePlanner()
    gesture, led = planner.plan("navigation", rag_resp.answer)
    return {
        "rag_context": "\n\n".join(f"[{c.source}]: {c.text}"
                                   for c in rag_resp.retrieved_chunks),
        "rag_chunks": rag_resp.retrieved_chunks,
        "rag_faithfulness": rag_resp.faithfulness_score,
        "response_text": rag_resp.answer,
        "model_id": rag_resp.model_id,
        "latency_ms": rag_resp.latency_ms,
        "gesture": gesture,
        "led_color": led,
    }
```

What `rag.query()` does internally:

1. Convert "Where is Room 305?" into an embedding vector via sentence-transformers.
2. Search ChromaDB for the 4 most similar chunks from the DIBRIS knowledge base.
3. Build an augmented prompt: `system + "Use only this context: ..." + user`.
4. Call `gateway.query(model_id, messages)` — this returns the LLM's answer.
5. (Optional) Score the answer's faithfulness with a judge LLM.

The result is something like:

```
"Room 305 is on the third floor of the DIBRIS building. Take the
elevator on your left, then turn left at the corridor."
```

Then `planner.plan("navigation", answer)` sees the word "left", picks gesture `point_left`, picks LED `#00AAFF` (calm navigation blue).

Node 5 — `smart_router`

For `condition="C"`, the router fires:

```
from omnillm.router import SmartRouter, RoutingStrategy
router = SmartRouter()
decision = router.route_for_hri_task(
    hri_task_type="navigation",
)
target = decision.model_id # e.g. "openai-gpt4o-mini" per hri_task_routing
resp = await gateway.query(target, messages)
return {"response_text": resp.content, "model_id": resp.model_id, ...}
```

The router reads `config/models.yaml`'s `hri_task_routing.navigation`. The answer overrides what `nav_rag` produced. (For conditions A, B, E the `smart_router` node is a no-op — the previous answer wins.)

Node 6 — generate_action_plan

```
async def generate_action_plan(state: dict) -> dict:
    return {
        "robot_action": {
            "speech": state["response_text"],
            "gesture": state["gesture"],
            "emotion_led": state["led_color"],
            "metadata": {
                "task_type": state["task_type"],
                "model_id": state["model_id"],
                "rag_enabled": state["rag_enabled"]},
        }
    }
```

This is just packaging — the heavy work is done.

Node 7 — log_interaction

```
async def log_interaction(state: dict) -> dict:
    total_latency = (time.monotonic() - state["_start_time"]) * 1000
    logger.log_interaction(
        session_id=state.get("session_id", ""),
        participant_id=state.get("participant_id", ""),
        condition=state.get("condition", "A"),
        task_type=state.get("task_type"),
        utterance=state.get("utterance", ""),
        response=state.get("response_text", ""),
        model_id=state.get("model_id", ""),
        latency_ms=total_latency,
        # ...
    )
    return {"latency_ms": total_latency}
```

The logger appends an InteractionRecord to disk (JSON-Lines) and to the in-memory list.

K.1.5 — Stage 4 — Flask Returns the JSON

Back in app.py:

```
return jsonify(result.get("robot_action", ...))
```

The dict is serialised to JSON and returned with HTTP 200:

```
{
  "speech": "Room 305 is on the third floor of the DIBRIS building. Take the elevator on your left, then turn left",
  "gesture": "point_left",
  "emotion_led": "#00AAFF",
  "metadata": {
    "task_type": "navigation",
    "model_id": "openai-gpt4o-mini",
    "rag_enabled": true
  }
}
```

K.1.6 — Stage 5 — What Pepper Does With It

If a real Pepper is at the other end, naoqi_client.py would:

```
def _execute_action(self, action):
    speech = action.get("speech", "")
```

```
gesture = action.get("gesture")
led_color = action.get("emotion_led")
if led_color: self._set_leds(led_color)
if gesture and gesture in GESTURE_TO_BEHAVIOR:
    self._run_behavior_async(GESTURE_TO_BEHAVIOR[gesture])
if speech: self._speak(speech)
```

- Eyes fade to #00AAFF over 300 ms.
- `ALBehaviorManager.runBehavior("animations/Stand/Gestures/Explain_8")` starts the point-left gesture in a non-blocking thread.
- `ALAnimatedSpeech.say(speech, {"bodyLanguageMode": "contextual"})` speaks the answer with synchronised arm motion.

K.1.7 — The Whole Picture

That is one question, end-to-end. Sub-2-second latency (~1.4–1.6 s typical for Condition A/C), all of it logged, every model swap configurable, every step testable in isolation.

If you can find the code for each step in the repository without re-reading this section, you understand OmniLLM.

K.2 — A Complete First Experimental Session — Worked Example

A pretend participant P003 walks into the DIBRIS / Sgorbissa lab. We run them through all five conditions of one slot of the Latin square. Every command you'd type, every JSON you'd see, every questionnaire score you'd enter — front to back.

K.2.1 — Before the Participant Arrives

```
# 0. Pre-flight check
cd C:\Users\akshi\OneDrive\Desktop\OmniLLM
.\env\Scripts\Activate.ps1

omnillm models # all 19 models load
pytest tests/ -q | Select-Object -Last 3 # 289 tests pass

# 1. Make sure Pepper is awake (real robot at DIBRIS)
C:\Python27\python.exe -c "from naoqi import ALProxy; ALProxy('ALMotion', '192.168.1.100', 9559).wakeUp()"

# 2. Start the AI server (Terminal 1)
python -m omnillm.server.app --host 0.0.0.0 --port 5000
```

Expected server log lines:

```
INFO:omnillm.rag.pipeline:Indexed 12 chunks from lab_info.txt
INFO:omnillm.rag.pipeline:Indexed 15 chunks from faq.txt
INFO:omnillm.rag.pipeline:Knowledge base loaded -- 49 total chunks
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
```

```
# 3. Start the NAOqi bridge (Terminal 2, Python 2.7)
C:\Python27\python.exe omnillm\server\ naoqi_bridge_server.py `
  --robot-ip 192.168.1.100 --robot-port 9559 `
  --bind 0.0.0.0 --bridge-port 6000
```

Confirm both up:

```
curl http://127.0.0.1:5000/status
curl http://127.0.0.1:6000/ping
```

You also have, on the desk:

- 5 paper questionnaire forms (one per condition)
- 1 final pairwise-preference sheet
- 2 consent forms
- 1 GDPR notice
- The Latin-square sheet that says "P003: conditions C, D, E, A, B in that order"

K.2.2 — Participant Arrives — Briefing (5 minutes)

"Thank you for joining. Today you'll talk with Pepper in five different modes. Each mode has four short interactions. After each mode I'll hand you a brief paper questionnaire. The whole session takes about 30 minutes. You can stop at any time without giving a reason. There are no right or wrong answers — we're studying Pepper, not you. Please sign here..."

Hand over the consent form and GDPR notice. Note demographics: P003 speaks English and Italian.

K.2.3 — Run the Experiment Driver

```
# Terminal 3
.\env\Scripts\python.exe scripts\pepper_demo\run_subject_experiment.py `
--participant P003 `
--server http://127.0.0.1:5000 `
--bridge http://127.0.0.1:6000
```

Progress on screen as conditions cycle (abridged):

```
Subject experiment -- participant=P003 session=578b023b
Server: http://127.0.0.1:5000
Robot bridge: http://127.0.0.1:6000

[01/20] C / T1_info_retrieval: What time does the lab open?
-> The lab opens at 08:30 on weekdays...
model=openai-gpt4o-mini path=graph latency=1430 ms
robot: ok

[02/20] C / T2_navigation: Where is the Pepper room at DIBRIS?
-> The Pepper room is at the end of the ground-floor corridor...
model=openai-gpt4o-mini path=graph latency=1590 ms
robot: ok

[03/20] C / T3_social: Hello Pepper, how are you today?
-> Hello! I'm feeling bright and cheerful today, thank you!
model=claude-haiku path=graph latency=920 ms
robot: ok

[04/20] C / T4_multilingual: Ciao Pepper, dove si trova la stazione di Brignole?
-> Ciao! La stazione di Brignole si trova a Genova...
model=claude-haiku path=graph latency=1980 ms
robot: ok

--- [pause for Condition C questionnaire] ---
Press Enter when ready to continue...
```

You hand the participant the Condition C questionnaire:

```
+-----+
| CONDITION C   PARTICIPANT P003 |
+-----+
| Strongly disagree 1 2 3 4 5 6 7 Strongly agree |
+-----+
| Q1 The robot's answers were accurate.      [_] [_] [_] [X] [_] [_] [_] -> 4 |
| Q2 The robot was natural to talk to.        [_] [_] [_] [_] [_] [X] [_] -> 6 |
| Q3 I trust the information given to me.      [_] [_] [_] [_] [_] [X] [_] -> 6 |
| Q4 The robot's gestures were appropriate.    [_] [_] [_] [_] [X] [_] [_] -> 5 |
| Q5 The robot responded quickly enough.      [_] [_] [_] [_] [_] [X] [_] -> 6 |
+-----+
```

P003 ticks 4/6/6/5/6. Continue with D, E, A, B in their assigned order. At each condition boundary, hand the matching questionnaire, collect after ~60 seconds, press Enter.

K.2.4 — The Pairwise Preference at Session End

After all 20 interactions, ask three verbal questions:

"1. Which of the five Peppers did you prefer overall?" "2. Which one felt most natural to talk to?" "3. Which one would you most trust to give correct information?"

P003 answers: "Overall I preferred C. Most natural — also C. Most trustworthy — D, the one that took longer." Record on the final sheet.

This produces seven pairwise records: C wins over A, B, D, E (overall); C wins over A, B, D, E (naturalness); D wins over A, B, C, E (trust). These feed `EloScorer.update_pairwise(...)` later.

K.2.5 — Save Everything Within the Hour

```
# Outputs are already at:
# results/subject_run_P003_<timestamp>.json
# results/subject_run_P003_<timestamp>.csv

# Move to per-participant folder
New-Item -ItemType Directory -Path "results/by_participant/P003"
Move-Item "results/subject_run_P003_*.json" "results/by_participant/P003/"
Move-Item "results/subject_run_P003_*.csv" "results/by_participant/P003/"

# Digitise the questionnaires
python -c '@'
from omnillm.utils.questionnaire import (
    InteractionQuestionnaire, PairwisePreference, QuestionnaireCollector)
c = QuestionnaireCollector()
c.add_interaction_response(InteractionQuestionnaire(
    session_id="578b023b", participant_id="P003", condition="A",
    accuracy=6, naturalness=5, trust=6,
    gesture_appropriateness=5, response_speed=7))
# ...repeat for B, C, D, E...
c.add_pairwise_preference(PairwisePreference(
    session_id="578b023b", participant_id="P003",
    condition_a="C", condition_b="A", preferred="C"))
# ...etc for all pairwise records...
c.save("results/by_participant/P003/questionnaire.json")
'@
```

K.2.6 — What the JSON Logs Look Like

A typical `InteractionRecord` row in the JSON file:

```
{
  "timestamp": "2026-06-12T14:23:51.842Z",
  "session_id": "578b023b-09af-4da3-a047-e742e25c5913",
  "participant_id": "P003",
  "condition": "A",
  "task_type": "info_retrieval",
  "utterance": "What time does the lab open?",
  "response": "The lab opens at 08:30 on weekdays.",
  "model_id": "openai-gpt4o-mini",
  "latency_ms": 1437.3,
  "input_tokens": 412,
  "output_tokens": 13,
  "cost_usd": 0.0000687,
  "rag_enabled": true,
  "rag_faithfulness": 0.94,
  "rag_chunk_count": 3,
  "judge_score": 0.91,
  "task_success": true,
```

```
"language": "en",  
"gesture_used": "nod"  
}
```

After 15 participants × 5 conditions × 4 tasks, you have **300 such rows** plus 75 questionnaire rows plus ~45 pairwise preferences. That is your dataset.

K.2.7 — Final Backup Step

```
# Copy results/ to a backup location at the end of every day  
Copy-Item -Recurse -Force "results\" "D:\thesis-backup\results-2026-06-12\"
```

Data loss after a participant has gone home is the worst-case scenario in HRI studies. Don't be the person it happens to.

K.3 — The Eight Evaluation Axes (Enumerated)

OmniLLM's built-in benchmark tasks (`tasks/sample_tasks.py`) span eight axes chosen to be **orthogonal**: a model that scores high on one does not necessarily score high on the others. These axes power the `omnillm evaluate` command and the leaderboard categories.

#	Axis	What it measures	Sample tasks
1	Reasoning	Multi-step logic, mathematics	Bat-and-ball puzzle; syllogism; Gambler's Fallacy
2	Knowledge	Factual accuracy	Capital cities; speed of light; DNA discoverers
3	Code	Generate, debug, explain code	<code>is_prime</code> ; FizzBuzz; binary search bug
4	Instruction Following	Strict format / constraints	Numbered list of 5; pure JSON output; exact 3 sentences
5	Safety	Refusal of harmful / PII requests	Refuse hacking; refuse misinformation; redact PII
6	Robot-Readiness	Structured JSON for robot control	Generate NAO action plan; parse voice command; ROS2 Nav2 goal
7	Latency	Time-to-first-token	One-sentence greeting; "Answer with only the number"
8	Cost-Efficiency	Quality at minimum tokens	Define "recursion" in one sentence; Yes/No questions

K.3.1 — The Three Judge Patterns (Recap)

Pattern	When to use it	Output
Referenceless (G-Eval)	No gold answer; open-ended	Score 0–1 + reasoning
Reference-Based	You know the right answer	Score 0–1 + reasoning
Pairwise	Comparing two models	Winner: A / B / tie

K.3.2 — The Twelve Per-Interaction Metrics (HRI)

Automatically recorded for every Embodied LLM Arena interaction:

1. **Latency** — end-of-utterance to start-of-robot-speech, in ms.
2. **Input tokens** — prompt token count.
3. **Output tokens** — completion token count.
4. **Cost (USD)** — calculated from `models.yaml` pricing.
5. **RAG retrieval scores** — cosine similarity of top-k chunks.
6. **RAG faithfulness** — LLM-as-Judge of grounding (0–1).
7. **Hallucination flag** — word-overlap heuristic.
8. **LLM-as-Judge quality** — referenceless score (0–1, optional).
9. **Language detected** — ISO 639-1 code.
10. **Task classification** — T1/T2/T3/T4 + confidence.
11. **Model used** — the actual `model_id` (matters for Conditions C and D).
12. **Gesture used** — what Pepper actually did.

All twelve fields land in `InteractionRecord` and export cleanly to CSV.

K.3.3 — Composite Scoring (the `BEST_VALUE` Formula)

```
value = 0.50 * quality + 0.30 * cost_score + 0.20 * latency_score
```

Where:

- `quality` $\in [0, 1]$ from LLM-as-Judge or running mean of past evaluations
- `cost_score` = $\max(0, 1 - \text{cost} / 0.05)$ (normalised against \$0–\$0.05 per query)
- `latency_score` = $\max(0, 1 - (\text{latency_ms} - 500) / 9500)$ (normalised against 500ms–10s)

This is the formula the smart router uses for `BEST_VALUE` strategy. The 50/30/20 weighting was the most stable across all four HRI task types in development; it lives in `_calculate_value_score` and is editable.

K.4 — Complete Feature Reference Catalogue

Every shipped feature, organised by category. Use this as a checklist when comparing OmniLLM against another HRI framework, or when answering "what does this project actually do?"

K.4.1 — Core LLM Features

- Unified `LLMGateway.query(model_id, messages)` across **19 registered models** from 6+ providers (OpenAI, Anthropic, Google, DeepSeek, Ollama, openai-compatible)
- `LLMGateway.query_multiple(model_ids, messages)` — parallel queries via `asyncio.gather`
- Per-call cost calculation in USD from the `config/models.yaml` registry
- Latency measurement with `time.perf_counter()` high-resolution timer
- Graceful per-call error handling — errors return as `ModelResponse(error=...)` rather than raising
- `litellm.drop_params = True` — automatically strips unsupported parameters (e.g. `temperature` for o-series models)
- Provider prefix routing (`ollama/`, `gemini/`, `anthropic/`, `openai/`)

K.4.2 — Smart Routing Features

- Six routing strategies: `BEST_QUALITY`, `LOWEST_COST`, `LOWEST_LATENCY`, `BEST_VALUE`, `LOCAL_PREFERRED`, `TASK_TYPE`
- Composite `BEST_VALUE` score (quality 50% + cost 30% + latency 20%)
- Hard constraints: `budget_usd`, `max_latency_ms`
- Fallback model list in `config/models.yaml` (`routing.fallback_models`)
- Per-HRI-task routing via `routing.hri_task_routing` block
- Static default scores in `SmartRouter._DEFAULT_SCORES` for the cold-start case
- Self-updating quality table from past `omnillm evaluate` results
- `route_by_complexity()` — short prompts -> cheap model; long -> quality model

K.4.3 — Consensus / Council Features

- Three synthesis strategies: `majority_vote`, `weighted`, `synthesis` (default)
- Default 3-model council (configurable)
- Position-bias mitigation via response shuffling on synthesis
- Graceful per-model degradation (council continues if one model fails)
- Cost-aware: configurable per-call budget cap

K.4.4 — Evaluation Features (LLM-as-Judge)

- **Referenceless (G-Eval)** — score open-ended responses
- **Reference-Based** — score against a known gold answer

- **Pairwise** — winner: A / B / tie, with position-bias swap-and-aggregate
- 8 evaluation axes (Reasoning, Knowledge, Code, Instruction-Following, Safety, Robot-Readiness, Latency, Cost-Efficiency)
- Configurable judge model (default GPT-4o-mini)
- Used internally by RAG pipeline for faithfulness scoring

K.4.5 — ELO Scoring Features

- Standard ELO maths (K=32 below 2000, K=16 above)
- Per-category leaderboards: `overall`, `reasoning`, `embodied_hri`, plus the four HRI task types
- `update_pairwise(model_a, model_b, winner, category=...)`
- `leaderboard(category=...)` returns sorted ratings
- Persistent JSON file across runs
- Initial rating 1000 per category

K.4.6 — RAG Features

- ChromaDB-backed semantic retrieval (HNSW cosine)
- Keyword-search fallback when ChromaDB unavailable
- Document loaders for `.txt`, `.csv`, `.pdf` (via `pypdf`)
- Recursive character text splitter (`chunk_size` 512, `overlap` 64)
- Configurable `top_k` (default 4)
- LLM-as-Judge faithfulness scoring (0–1)
- Lightweight hallucination heuristic (answer-word coverage)
- Per-call `model_id=` override (May 2026 addition)
- Persistent SQLite-backed ChromaDB store

K.4.7 — HRI Features

- Rule-based task classifier (T1–T4) with optional LLM fallback
- 3-tier language detector: Unicode script, n-gram, `langdetect`
- `LanguageDetector.recommended_model` per detected language
- 5 experimental conditions (A–E) with declarative `CONDITION_CONFIGS`
- `ParticipantSession` dataclass with UUID session IDs
- `ExperimentManager` for Latin-square assignment (optional)
- Per-condition model resolution at `/interact` endpoint (May 2026 fix)
- Multilingual retry-with-backup-model in T4 node

K.4.8 — LangGraph Agent Pipeline Features

- 9 nodes, all async

- `_merge_state` wrapper preserves prior fields through every node
- Conditional edge based on `(task_type, condition, rag_enabled)`
- Fully visualisable as Mermaid / Graphviz
- Lazy LangGraph import (server still runs without it via fallback)
- End-to-end latency measurement via `_start_time` field

K.4.9 — Robot Bridge Features

- Abstract RobotBridge ABC with 6 async methods
- Pepper concrete implementation with 3-mode auto-detection (`server / direct / stub`)
- aiohttp async HTTP client
- Choregraphe virtual-robot port auto-discovery
- Three trigger modes (`text, touch, vad`) in `naoqi_client.py`
- Face tracking via `ALTracker` (`--track-face`)
- Two topologies (Pepper-polls and AI-server-drives)

K.4.10 — Robotics Helper Features

- `GesturePlanner` — rule-based (`task_type + response_text`) -> (`gesture, LED`)
- Direction-pattern regex matching for navigation gestures
- Content-triggered gesture overrides (`hello -> wave, goodbye -> wave_goodbye`)
- 7-colour eye-LED palette tied to interaction mode
- Local whisperSTT (privacy-preserving, free, ~700ms on CPU)
- `parse_llm_to_action()` — LLM JSON -> `RobotAction`
- `parse_nav2_goal()` — for future ROS2 integration

K.4.11 — Server Features

- Flask AI server with 6 HTTP endpoints (`/interact, /transcribe, /evaluate, /health, /status, /export`)
- Stdlib-only NAOqi bridge server (Python 2.7-compatible)
- Three trigger modes in NAOqi client
- Automatic fallback to direct gateway if LangGraph unavailable
- CORS-off by default (same-LAN only)
- Environment-variable-based config (`OMNILLM_DEFAULT_MODEL, OMNILLM_KNOWLEDGE_BASE`)

K.4.12 — Logging / Reporting Features

- `ExperimentLogger` with append-only JSON-Lines persistence
- `InteractionRecord` dataclass — 20 fields per interaction
- `QuestionnaireCollector` — Likert + Godspeed + Pairwise + Observer

- `CostTracker` — per-model running USD spend
- `export.py` — JSON ↔ CSV ↔ Markdown converter
- `summary_by_condition()` for the per-condition Likert means table
- `pairwise_win_rates()` for the ELO updates

K.4.13 — CLI Features

- `omnillm models` (filterable by cloud/local)
- `omnillm ask` — one or many models or `--all`
- `omnillm route` — six strategies with optional budget/latency constraints
- `omnillm council` — three synthesis strategies
- `omnillm compare` — pairwise with position-bias swap
- `omnillm evaluate` — full 8-axis benchmark sweep
- `omnillm leaderboard` — per-category ELO leaderboard
- `omnillm costs` — per-model spend
- `omnillm export` — CSV / Markdown / JSON

K.4.14 — Test Suite Features

- 289 tests covering every public function
- All LLM calls mocked — no API keys or real network required
- `pytest-asyncio` for async test functions
- `pytest-mock` for the `mocked` fixture
- ~5 second total run-time
- CI-friendly (Linux, macOS, Windows)

K.4.15 — Documentation Features

- This 250-page book (rebuildable from Markdown via pure-Python pipeline)
- DejaVu-based PDF with Unicode glyph coverage
- 18 chapters + 11 appendices
- Three-reader structure (Owner / Beginner / Academic) consistent across every chapter

K.5 — One-Month Execution Plan (Distinct From the 13-Week Build Plan)

Chapter 22 gave a 13-week build plan for someone replicating OmniLLM from a blank repo. This section gives a one-month execution plan for someone whose OmniLLM is already built and who wants to run the experimental study at the lab.

K.5.1 — Week 1 — Final Infrastructure Setup (Days 1–7)

Day 1–2 — Pepper ↔ AI Server Bridge verification. Confirm `/health` returns OK against the real Pepper at DIBRIS. Run the Day-Zero checklist (Chapter 21) end-to-end with the author as P000. Catch any infra bugs at this stage rather than mid-study.

Day 3–4 — OmniLLM + LiteLLM full sweep. Confirm `omnilm models` lists all 19 models, that each model with an API key returns a non-error `omnilm ask` response, that Ollama returns from `llama3-8b-local`. Test the smart router (`omnilm route --strategy TASK_TYPE`) and LLM-as-Judge (`omnilm evaluate`).

Day 5–6 — RAG Pipeline verification. Open the DIBRIS knowledge base files (`lab_info.txt`, `faq.txt`, `university_map.txt`) and verify they reflect current lab state. Update any outdated facts. Re-index. Test retrieval with `curl -X POST .../interact -d '{"text":"What time does the lab open?","condition":"A"}'`.

Day 7 — End-to-end pilot pass. Run the full 20-interaction matrix on the real Pepper with yourself as P000. Verify every condition routes to the expected model. Check latencies fall in the expected range (A: ~1.4s, B: ~2.8s, D: ~3.1s).

K.5.2 — Week 2 — Protocol Refinement and Participant Prep (Days 8–14)

Day 8–9 — Gesture / Speech Refinement. Watch the pilot recording (if available). Are gestures crisp? Is Pepper's speaking rate too fast? Tune `ALAnimatedSpeech` parameters. Confirm the 7-colour LED palette communicates the intended mode.

Day 10–11 — Final Questionnaire and Protocol. Print 20 sets of paper questionnaires (5 conditions per participant × 15 participants + spares). Finalise the experimenter script (the briefing speech). Print Latin-square assignment sheets for each participant slot.

Day 12–13 — Pre-Study Pilots With Lab Colleagues. Run 2–3 pilots with friendly volunteers from the lab. Note: do not include these in the final N. Identify and fix any latency issues, Whisper misfires, gesture-sync problems, or protocol clarification needs.

Day 14 — Ethics & GDPR Paperwork. Confirm Comitato Etico approval is in hand (if required). Have consent forms printed. Set up the participant-recruitment email and book the lab schedule.

K.5.3 — Week 3 — Run Experiments (Days 15–21)

Day 15–21 — Participant Sessions. Aim for **2–3 participants per day**. Each session ≈ 40–60 minutes including briefing, the 20 interactions, paper questionnaires, pairwise preference, debrief, and turnover. Target: **15 participants** by end of week.

Day	Participants
Day 15 (Mon)	P001, P002
Day 16 (Tue)	P003, P004
Day 17 (Wed)	P005, P006, P007
Day 18 (Thu)	P008, P009
Day 19 (Fri)	P010, P011
Day 20 (Mon)	P012, P013
Day 21 (Tue)	P014, P015

Reserve days 22–24 as buffer for reschedules / drop-outs.

Daily routine:

1. Pre-session: Pepper power-on, server-up, ping check.
2. Per-session: briefing -> driver -> questionnaires -> pairwise -> debrief.
3. Post-session: backup results/by_participant/Pxxx/ immediately.
4. End-of-day: Copy-Item -Recurse results\ D:\thesis-backup\results-<date>\.

K.5.4 — Week 4 — Analysis and Writing (Days 22–30)

Day 22–24 — Data Analysis. Open notebooks/analysis.ipynb. Load all 15 participants' CSVs into pandas. Compute per-condition Likert means with 95% CIs. Run the repeated-measures ANOVA. Compute pairwise win-rates. Build the embodied ELO leaderboard.

Day 25–28 — Write. The thesis structure:

- Introduction & Related Work (use Appendix G citations)
- System Architecture (figures from Appendix E + Part II)
- Experimental Design (Part V chapters 23–25)
- Results (your tables from Day 22–24)
- Discussion (refer to Part VI for future scope; compare against literature in Appendix H)
- Conclusion

Day 29–30 — Polish and Submit. Generate the final figures. Cross-check every citation. Run a final spell-check. Submit to your supervisor for review. Address feedback within 2–3 days.

K.5.5 — Common Failure Modes During the Study Month

Failure	Mitigation
Participant cancels last-minute	Have a backlog of 3–5 alternates on stand-by. Recruit through the department mailing list as a rolling pool.
Pepper drops connection mid-session	The driver logs and continues; if it recurs, restart bridge + driver in <60s. If it happens twice in one session, abort and reschedule.
OpenAI/Anthropic outage	Most outages are <30 minutes. Schedule sessions later that day. If sustained, run Conditions B + locally-routed C only.
Whisper transcription fails for a participant with a strong accent	Bump <code>model_size</code> to "small" in <code>whisper_stt.py</code> for that participant only.
Ethics committee comes back with revision requests	Build in 2-week buffer before Day 15. Approval-related delays are the #1 cause of missed thesis deadlines.
Your laptop battery dies mid-session	Always plug in. Have a spare charger in the bag.

K.5.6 — When to Stop

You will be tempted to add features forever. The thesis writes itself when you can answer **three** questions with data:

- Does smart routing beat fixed (H2)?
- Does RAG help (H3)?
- Does embodiment change the ranking (H1)?

Everything else is decoration. Add it after submission.